

Security Operations Lab — Complete Reference

Contents

Security Operations Lab — Complete Reference	5
From First Setup to Advanced Identity Security Practice	5
How to use this document	5
Who this is for	6
The philosophy behind this document	6
A note on how this lab differs from a training course lab	6
Document structure at a glance	6
Part 1: Lab Architecture Overview	7
What this lab actually is	7
Master architecture diagram	7
What's actually connected vs not yet (read this before anything else)	7
Why the lab is structured this way	8
Part 2: Environment Setup End-to-End	8
2.1 Toolchain overview	9
2.2 Homebrew	9
2.3 Docker Desktop	10
2.4 QEMU	11
2.5 Terraform and Azure CLI	12
2.6 GitHub CLI (gh)	12
2.7 Node.js, npm, and wrangler	13
2.8 Python 3, and why venvs per project (not a global environment)	13
2.9 Verify your setup — full checklist	14
Part 3.1-3.3: Wazuh, TheHive + Cortex, LocalStack	15
3.1 Wazuh	15
What it is and the principle behind it	15
Paid vs free tier reality	16
When to use this / when NOT to use this	16
Internal architecture	16
Install and configuration as used in this lab	16
The MySQL pipeline, as a concrete worked example	18
The Cloudflare Pages pipeline, as a second worked example	19
Troubleshooting	19
How Wazuh fits into the overall lab architecture	19
3.2 TheHive + Cortex	20
What it is and the principle behind it	20
Paid vs free tier reality	20
When to use this / when NOT to use this	20
Internal architecture	20
Install and configuration as used in this lab	20

Troubleshooting	22
How TheHive + Cortex fits into the overall lab architecture	22
3.3 LocalStack	22
What it is and the principle behind it	22
Paid vs free tier reality	23
When to use this / when NOT to use this	23
Internal architecture	23
Install and configuration as used in this lab	23
Troubleshooting	25
How LocalStack fits into the overall lab architecture	25
Part 3.4-3.6: Suricata, MySQL, Cloudflare Pages + Workers/Functions	25
Part 3.4: Suricata — Network Intrusion Detection	26
What a network IDS actually does	26
Signature-based detection vs anomaly detection	26
Why Suricata is “the obvious next thing to feed into Wazuh” — and why it isn’t wired in yet	26
When to use this / when NOT to use this	27
Part 3.5: MySQL as a Monitored, Attacked Lab Target	27
Why a real MySQL container	27
general_log vs error_log — and why that distinction is a real production tradeoff	27
Wazuh’s built-in MySQL decoder and rules	28
The prematch problem, and how the relay bridges it	28
The verified result	30
When to use this / when NOT to use this	30
Part 3.6: Cloudflare Pages + Workers/Functions	30
Static hosting vs edge compute — the distinction that matters here	30
The lab’s real setup	30
Why Free tier has no Logpush, and what that means practically	30
The wrangler-tail limitation: bound to a deployment ID	31
The relay and Wazuh ingestion — and why this needed a custom rule but no custom decoder	31
Verified result	31
When to use this / when NOT to use this	33
Part 3.7-3.9: QEMU/QuartzVM/Windows VM, Terraform + Azure, Sigma + KQL (Detection-as-Code)	33
Part 3.7: QEMU, QuartzVM, and the Windows Server VM	33
Hypervisor acceleration vs software emulation	33
The disposable-lab philosophy vs what actually got built	34
Unattended installation via autounattend.xml	34
The real bug: a specialize-pass component name mismatch	34
Why BIOS/IDE instead of UEFI/virtio	35
When to use this / when NOT to use this	35
Part 3.8: Terraform + Azure	35
State: the core concept, and why remote state matters	35
The bootstrap chicken-and-egg problem	36
Module composition	36
Terraform Cloud vs local backend	36
“Syntax valid” is not “actually deploys correctly” — and none of this has been applied	36
When to use this / when NOT to use this	37
Part 3.9: Sigma + KQL — Detection-as-Code	37
What Sigma is and why it exists	37
The real platform-translation problem this lab actually hit	38
attack-mapping.csv and MITRE ATT&CK tagging as a discipline	38
When to use this / when NOT to use this	38

Part 4: Identity Security Deep Dive	40
4.1 Core identity concepts: what an “identity” actually is	40
Identity object types	40
Entra ID / Azure AD vs AWS IAM: two different vocabularies for the same problem	41
4.2 The Joiner-Mover-Leaver (JML) lifecycle	42
A real failure mode from this lab: unverified contractor provisioning	42
Detective vs preventive controls, and why this distinction is the whole lesson	42
JML and Mover/Leaver in this lab’s other content	43
4.3 GUID/Object triage methodology	44
Why this ambiguity exists	44
The real investigative methodology	44
Why this matters beyond one runbook	45
4.4 Conditional Access, MFA, and PIM	45
4.5 RBAC and least privilege	47
Role assignment scope in Azure	47
Common over-privileging failure modes	48
4.6 Federation and identity providers (IdP)	48
Entra ID as an IdP, and why Auth0 was planned as a second one	48
4.7 BEC and phishing as identity-adjacent threats	49
Case 1: invoice fraud via authenticated bulk-mail services	49
Case 2: executive impersonation via external personal email	49
Why these are identity security problems, not just email problems	49
Part 5: Detection Engineering Principles	50
5.1 Sigma rule anatomy	50
Where Sigma’s single-rule model genuinely runs out	51
5.2 KQL fundamentals	51
The pipeline problem: why conversion isn’t automatic for every log source	52
5.3 MITRE ATT&CK mapping as a discipline	53
5.4 False positive tuning philosophy	53
5.5 The platform translation problem	54
Why “just convert it” isn’t trivial	54
Why forcing the fit would be worse than admitting the gap	55
The generalizable lesson	55
Part 6: Incident Response & Triage Workflows	56
6.1 The alert → investigation → case → resolution lifecycle	56
The honest gap in this lab	56
When to use a dedicated case management system / when not to	58
6.2 Real case walkthroughs	58
Walkthrough 1: The GUID triage runbook (detection-engineering/guid-triage/)	58
Walkthrough 2: The two BEC/phishing cases (detection-engineering/bec-phishing/)	59
Walkthrough 3 (hypothetical — planned, not run yet): T1078.004 and T1059.001	59
Walkthrough 4: The real live-fired alerts from this lab session	60
6.3 A generic triage runbook template	61
1. Context and scope	61
2. Initial hypothesis	61
3. Evidence gathering (the “who/what/how many” of triage)	61
4. Confirm or deny	61
5. Containment decision	61
6. Documentation	61
Part 7: SOAR & Automation Principles	62
7.1 When to automate vs. keep a human in the loop	62

7.2 Playbook design walkthroughs	62
Playbook 1: auto-enrich-signin-alert — no approval gate, and why	62
Playbook 2: disable-compromised-user — Teams approval gate, and why here specifically	63
Playbook 3: isolate-infected-device — selective isolation, and why un-isolation is manual-only	63
What “designed but not deployed” honestly means	65
7.3 A general SOAR playbook design checklist	65
Trigger conditions	65
Required permissions/scopes (principle of least privilege for the automation’s own identity)	65
Approval gates	66
Rollback path	66
What stays manual, and why	66
Part 8: Cloud Security & Infrastructure as Code	66
8.1 Terraform + Azure: security principles beyond “how IaC works”	66
Secrets hygiene in IaC: why <code>.tfstate</code> and <code>.tfvars</code> must never be committed	66
gitleaks as a second-layer safety net — and its real, documented limitation	67
Least-privilege auth for IaC: <code>az login</code> vs. long-lived service principal secrets	67
8.2 Case study: security-header baseline at the Application Gateway	68
The principle: centralize the control at a chokepoint	69
The honest limitation	69
8.3 Case study: WAF-in-front-of-APIM, detection mode pilot	70
Detection mode vs. prevention mode	70
Why this staged-rollout pattern matters	70
8.4 AWS side: IAM privilege escalation detection against LocalStack	71
IAM privilege escalation: what the detections are looking for	71
Why LocalStack instead of a real AWS account for this kind of testing	72
Part 9: End-to-End Practical Labs	72
Lab 1: Stand up Wazuh [FREE] locally via Docker	72
Goal	72
Prerequisites	72
Steps	73
Expected result	74
What it teaches	74
Lab 2: Wire a real Cloudflare Pages site’s traffic into Wazuh with a custom detection rule	74
Goal	74
Prerequisites	74
Steps	74
Expected result	75
What it teaches	76
Lab 3: Wire a real MySQL container’s logs into Wazuh using its built-in <code>mysql_log</code> decoder/rules	76
Goal	76
Prerequisites	76
Steps	76
Expected result	77
What it teaches	77
Lab 4: Build a Windows Server 2022 evaluation VM via QEMU with a fully unattended install	77
Goal	77
Prerequisites	77
Steps	77
Expected result	79
What was NOT verified — an honest, stated gap	79
The open, unresolved tension worth naming	79
What it teaches	79

Lab 5 (NOT YET DONE — a documented next step, not a completed lab): install Sysmon + a Wazuh agent, run one real Atomic Red Team test	79
Status	79
Goal (planned)	80
Prerequisites this depends on	80
What it would teach, once done	80
Part 10: Real Troubleshooting Log	80
10.1 The Wazuh certificate generator permissions bug	80
10.2 The Windows autounattend.xml specialize-pass component error	81
10.3 The Cloudflare API token permission-scoping saga	82
10.4 The wrangler tail deployment-ID binding limitation	83
10.5 The MySQL log-format mismatch with Wazuh's decoder	83
10.6 The repo-wide markdown-header convention drift	84
Part 11: Glossary	85
A	85
B	85
C	85
D	85
F	86
G	86
H	86
I	86
J	86
K	87
L	87
M	87
O	87
P	87
R	87
S	88
T	88
W	88
X	88
Part 12: Appendix — Paid vs. Free Summary Table	89
Notes on accuracy and drift	94

Security Operations Lab — Complete Reference

From First Setup to Advanced Identity Security Practice

How to use this document

This is a single, complete reference for a real, working, self-hosted security operations lab — built on one Mac, using a mix of free open-source tools and free tiers of commercial products, deliberately kept close to zero recurring cost. It is written to take a reader from “I have never installed any of this” through to “I understand the principle behind every tool, why it was chosen over the alternative, and what I would do differently in a real production environment.”

It is organized in the order you would actually build the lab, followed by the identity security and detection engineering theory that gives the practical work its meaning, followed by incident response, automation, and

cloud security material, followed by fully reproducible practical labs, a real troubleshooting log from actually building this, a glossary, and a paid-vs-free summary table.

Read it front to back if you are new to this field. Use it as a reference if you already know the concepts and need the exact commands, the exact config, or the exact reasoning behind a specific decision in this lab.

Who this is for

Any security persona: a SOC analyst learning to triage; a detection engineer learning to write and test detection logic; an identity/IAM engineer learning how identity governance actually breaks in practice; a cloud security engineer learning infrastructure-as-code security patterns; an incident responder learning a real triage methodology; a student or career-changer trying to build genuine, defensible hands-on experience instead of certificate-only knowledge.

The philosophy behind this document

No shortcuts. Every tool section explains the principle behind the tool, not just the commands to run it — the goal is that a reader who finishes a section understands why the tool is built the way it is, not just how to operate it. Every claim of something working is backed by what was actually verified, not assumed. Every gap, every unfinished piece, every tool that was tried and abandoned, is documented honestly rather than smoothed over — a lab that only shows finished, successful work teaches less than one that shows the real process, including the parts that are still open questions.

Paid products are marked clearly wherever they appear, and the free-tier limits that actually matter are spelled out — the aim throughout was to build something close to zero-cost, and where a paid product was still worth discussing (because it's what real employers actually run), that's noted honestly rather than pretended around.

A note on how this lab differs from a training course lab

Training courses give you a lab environment that's already built for you, usually pre-wired, usually disposable, usually forgotten the moment the course ends. This lab is the opposite: built from scratch, kept running, extended over time, with real bugs hit and real fixes applied — the actual bugs and fixes are documented in Part 10 specifically because that debugging process is itself the most transferable skill in this whole document. Anyone can follow a script that works. Learning to diagnose why it doesn't, the first time, is the actual job.

Document structure at a glance

- Part 1 — Lab architecture overview, with a master diagram of every component and how it connects (or, honestly, doesn't yet connect) to every other component.
- Part 2 — Environment setup end-to-end: getting the host machine ready.
- Part 3 — Tool-by-tool deep dives: what each tool is, the principle behind it, paid vs free, when to use it and when not to, install/config, and real troubleshooting.
- Part 4 — Identity security deep dive: the theory that gives the identity-focused parts of this lab their meaning.
- Part 5 — Detection engineering principles: Sigma, KQL, MITRE ATT&CK, false-positive discipline, and the real platform-translation problem this lab hit.
- Part 6 — Incident response and triage workflows, walked through using this lab's real case studies.
- Part 7 — SOAR and automation principles, walked through using this lab's real playbook designs.
- Part 8 — Cloud security and infrastructure-as-code, walked through using this lab's real Terraform exercises.
- Part 9 — End-to-end practical labs: fully reproducible, numbered, with exact commands and exact expected results.

- Part 10 — A real troubleshooting log: the actual bugs hit while building this lab, and how they were diagnosed and fixed.
- Part 11 — Glossary.
- Part 12 — Appendix: a complete paid-vs-free summary table across every tool and service used.

Part 1: Lab Architecture Overview

What this lab actually is

A single MacBook (Apple Silicon, 16GB RAM) running a mix of self-hosted, containerized open-source security tools alongside a small number of free-tier cloud services, plus a public GitHub portfolio documenting the detection engineering, cloud security, and identity security work built on top of it. Nothing in this lab costs money to run in its current state — that was a deliberate constraint, not an accident, because the goal was to build genuine, defensible hands-on experience without needing an employer’s budget or a training course’s pre-built environment.

The lab is not one finished system. It is several independently useful pieces, some fully wired together and proven end-to-end with real traffic and real alerts, some built but not yet connected to anything else, and some still open questions. This document is honest about which is which — see the diagram below, and the “what’s actually connected vs not yet” section immediately after it.

Master architecture diagram

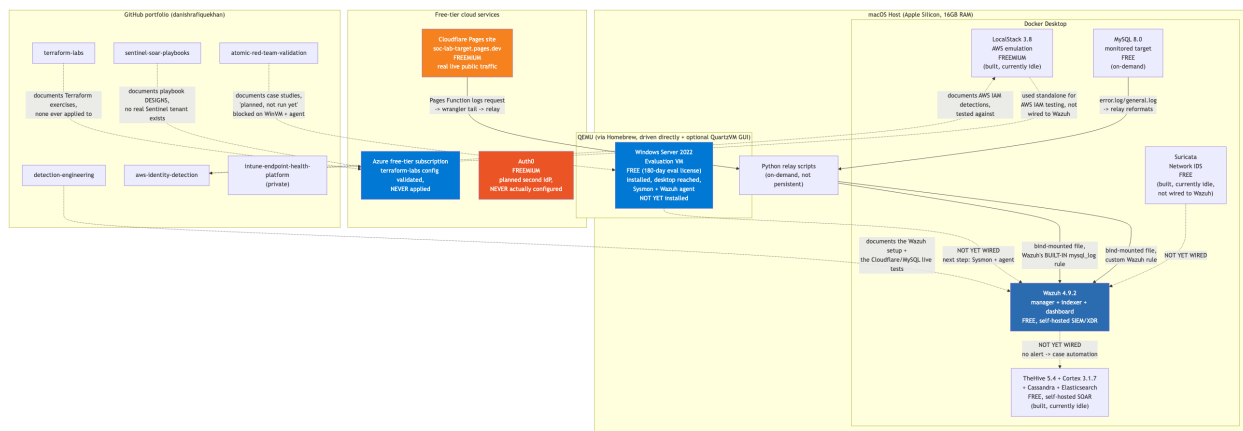


Figure 1: Diagram

Solid arrows are real, verified, currently-working data flows. Dashed arrows are either genuinely absent connections (documented gaps) or “this repo documents/tests against this thing, but there’s no live data flow” relationships. Reading the dashed arrows honestly is as important as reading the solid ones — a lab diagram that only shows solid arrows is lying by omission.

What’s actually connected vs not yet (read this before anything else)

Actually working, verified with real traffic, right now:

- Cloudflare Pages site → Python relay → Wazuh (custom rule, fires on /login.html hits)
- MySQL container → Python relay → Wazuh (built-in mysql_log rule, fires on failed auth)

Built and running, but not wired to anything else:

- TheHive + Cortex — a real case management/SOAR stack, sitting idle, not receiving alerts from Wazuh

- LocalStack — used standalone for AWS IAM testing content, not feeding any SIEM
- Suricata — network IDS running, not forwarding anything to Wazuh yet

Built, but the next connection is the actual point of building it and hasn't happened yet:

- The Windows Server VM — installed and reachable at a desktop, but has no Sysmon and no Wazuh agent installed on it yet. Until that happens, it cannot generate the telemetry the atomic-red-team-validation repo's case studies need to become real, run, evidenced work instead of "planned, not run yet" documentation.

Documented as a plan, never actually built:

- Auth0 as a second identity provider — a dashboard was set up, no working application credentials were ever configured.
- Any terraform apply against the real Azure subscription — every exercise in terraform-labs is syntax-valid and terraform validate-clean, and none has ever created a single real Azure resource.

Why the lab is structured this way

Three deliberate architectural decisions run through everything else in this document, and understanding them up front makes every later section make more sense:

1. Wazuh, not Sentinel, is the live SIEM — and the Sigma/KQL detection content stays written for Sentinel anyway. Sentinel costs real money to run meaningfully; Wazuh is free and self-hosted. But Wazuh's architecture (agents + manager, built for host and network telemetry) is a genuinely poor fit for cloud identity provider log schemas like Entra ID sign-in logs — forcing that content onto Wazuh just to claim "it runs somewhere" would be a fake fit, not a real one. So the detection content that's meant to demonstrate Sentinel/KQL skill stays written for Sentinel's schema, syntax-validated but never fired against real Sentinel data (no free real Sentinel tenant exists yet — the Microsoft 365 Developer Program's E5 trial is the actual next step, not something blocked by cost). Meanwhile Wazuh gets its own, separate detection content (the Cloudflare and MySQL rules) written specifically for what Wazuh actually is. Part 5 covers this "platform translation problem" in full depth, because it's one of the most transferable lessons in the whole lab.
2. Everything that touches real traffic is disclosed as synthetic, and audited for real-data leaks before going public. The Cloudflare site has an on-page banner disclosing it's a lab fixture. Every IP in every committed sample uses documentation-reserved ranges. A full confidentiality audit (current files and entire git history, across every repo) was run before treating any of this as safe to be public, and it found and fixed one real leak — a specific former employer's tool name, genericized before that repo could go public. This isn't paranoia; it's a real discipline anyone building a public portfolio from real work experience needs, and it's worth doing deliberately rather than trusting that nothing slipped through.
3. Gaps stay visible instead of getting quietly built around. TheHive/Cortex not being wired to Wazuh, no terraform ever being applied, Auth0 never getting finished — none of these get hidden by only documenting the parts that work. A lab that shows you the finished, polished 20% teaches you a fraction of what one that shows the real, still-in-progress 100% does.

Part 2: Environment Setup End-to-End

This section walks through setting up the host machine for this lab from a completely clean macOS install, on Apple Silicon. Every command below is real and was actually used to build this environment — none of it is hypothetical. Where a step has a known failure mode, that failure mode is documented alongside the fix, because "it just works" is rarely true the first time and pretending otherwise doesn't help anyone reproducing this.

The host for this lab is a MacBook, Apple Silicon (M-series), 16GB RAM, running macOS. That RAM ceiling matters throughout this section and the rest of the document — Wazuh's three containers, TheHive's four

containers, LocalStack, Suricata, MySQL, and an x86_64 Windows VM under software emulation are all candidates to run concurrently, and 16GB does not comfortably hold all of them at once. Part of “setting up the environment” is accepting that you will be starting and stopping stacks deliberately, not leaving everything running.

2.1 Toolchain overview

Before the command-by-command walkthrough, here is the shape of what you’re installing and why it’s layered this way. Each layer depends on the one below it being healthy — a Docker Desktop that hasn’t been launched yet will make every container command below it fail in a way that looks like a Docker problem but is actually a “the daemon isn’t running” problem.

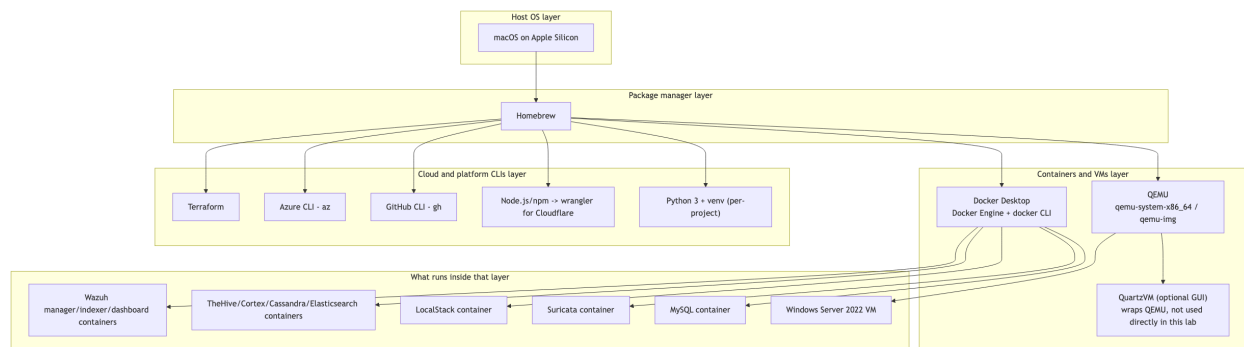


Figure 2: Diagram

The important read of this diagram: Homebrew is the single install path for almost everything else. Docker Desktop and QEMU are peers — both sit directly on Homebrew, both are “run untrusted/isolated things” layers, but they solve different problems (containers for Linux-native services like Wazuh; full x86_64 VM emulation for a Windows guest that needs its own kernel, not a container). The cloud CLIs (Terraform, az, gh, wrangler via npm) don’t depend on Docker or QEMU at all — they’re a separate, parallel install track that happens to also come through Homebrew (except wrangler, which comes through npm).

2.2 Homebrew

Homebrew [FREE] (open source, no paid tier) is macOS’s de facto package manager and the entry point for nearly everything else in this section.

Install it fresh (if not already present):

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

What this does: downloads and runs Homebrew’s official install script, which places the Homebrew prefix at /opt/homebrew on Apple Silicon (not /usr/local — that’s the Intel-Mac path, and mixing the two up is a common source of “brew installed but command not found” confusion when following instructions written for Intel Macs).

After install, the script itself prints the exact lines needed to add Homebrew to your shell PATH — run them, then verify:

```
eval "$(/opt/homebrew/bin/brew shellenv)"
brew --version
brew doctor
```

brew shellenv prints the environment variables (PATH, MANPATH, etc.) Homebrew needs and eval-ing it applies them to the current shell. Add the eval "\$(/opt/homebrew/bin/brew shellenv)" line to your ~/.zshrc (zsh is the default macOS shell) so every new terminal session picks it up automatically —

without this, brew-installed tools work in the shell you ran the installer in but vanish in every new terminal, which looks like a broken install but is actually just a missing PATH entry.

`brew doctor` audits your Homebrew install for common misconfigurations (conflicting PATH entries, permission issues on Homebrew's directories) and is worth running any time something installed via brew "isn't found" — its output is usually specific enough to fix directly.

2.3 Docker Desktop

Docker Desktop [FREEMIUM] (free for personal use and small business under Docker's current terms; paid subscription required for larger companies) is how every containerized piece of this lab runs: Wazuh's three containers, TheHive's four, LocalStack, Suricata, and the MySQL target.

Install via Homebrew cask:

```
brew install --cask docker
```

This downloads and installs the Docker Desktop .app — it does not start it, and it does not put a working docker daemon on your machine yet. This is the single most common trip-up in this whole setup process, worth stating plainly:

Docker Desktop must be manually launched at least once before anything Docker-related works. Installing the cask puts `Docker.app` in `/Applications` and symlinks the `docker/docker-compose` CLI binaries onto your PATH, but the actual container engine (a lightweight Linux VM Docker Desktop manages internally) only starts when you open the app — via Spotlight, open `-a Docker`, or clicking the icon — and it takes a real number of seconds to finish starting (the whale icon in the menu bar animates until it's ready).

Until that VM is up, every docker command that needs the daemon fails with a socket-connection error that has nothing to do with your actual command — for example:

```
$ docker info
```

```
Cannot connect to the Docker daemon at unix:///Users/you/.docker/run/docker.sock. Is the docker da
```

This looks like a broken install. It usually just means Docker Desktop's app hasn't been launched yet, or hasn't finished starting. The fix:

```
open -a Docker
```

then wait (10-30 seconds is typical, longer on first launch or after a macOS update) and confirm readiness with:

```
docker info
```

Once `docker info` returns a full daemon info block instead of the socket-connection error, the daemon is live. Verify the CLI and Compose plugin versions:

```
docker --version
```

```
docker compose version
```

Note the lab's compose stacks (Wazuh's `wazuh-docker`, TheHive's `compose file`) use `docker compose` — the integrated docker CLI subcommand shipped with Docker Desktop — not the older standalone `docker-compose` Python tool. If you find yourself following an older Wazuh tutorial that references `docker-compose` (hyphenated), substitute `docker compose` (space) throughout; the hyphenated tool is deprecated and may not even be present on a fresh Docker Desktop install.

A second real constraint worth flagging now, because it shapes how you'll operate this lab day to day: with 16GB of host RAM, running Wazuh (3 containers) + TheHive's stack (4 containers, including two JVM-heavy services — Cassandra and Elasticsearch) + LocalStack + Suricata + MySQL simultaneously will contend hard for memory. Docker Desktop's own Resources settings (Settings → Resources) let you cap how much RAM/CPU the Docker VM itself is allowed to use — check this setting if containers start getting OOM-killed

or the whole Mac becomes sluggish; it's not unusual to need to explicitly `docker compose down` one stack before bringing another one up on a 16GB machine.

2.4 QEMU

QEMU [FREE] (GPL, no paid tier) is the machine emulator/virtualizer used in this lab to build the Windows Server 2022 evaluation VM from raw ISO, without going through a heavier commercial virtualization product.

Install via Homebrew:

```
brew install qemu
```

Verify both binaries this lab actually uses are on PATH — `qemu-img` for disk image creation/management, and the `x86_64` system emulator specifically (Homebrew's `qemu` formula installs emulators for many target architectures; the Windows guest needs the `x86_64` one):

```
qemu-img --version
qemu-system-x86_64 --version
which qemu-img qemu-system-x86_64
```

Both should resolve to paths under `/opt/homebrew/bin/`. If `qemu-system-x86_64` is missing while `qemu-img` is present, it usually means an incomplete or corrupted brew install — `brew reinstall qemu` resolves it.

A note on why this lab uses raw `qemu-system-x86_64` commands directly rather than a GUI: QuartzVM (github.com/danishrafiquekhan/quartzvm), a Tauri/Rust desktop app the lab's author also built as a separate open-source project, exists specifically to wrap QEMU behind a "create → snapshot → destroy" disposable-lab UI. It's mentioned here because it's part of the same author's toolkit and worth knowing about, but the actual Windows VM build documented in this lab was driven directly via raw QEMU commands, not through QuartzVM's GUI — useful to know if you're trying to reproduce this lab exactly rather than through the GUI tool.

The core command shape used to boot the Windows Server 2022 VM (covered in full, with the unattended-install answer file and the firewall/RDP bug fix, in the identity/VM-focused section of this document) looks like:

```
qemu-system-x86_64 \
  -machine pc \
  -m 4096 \
  -smp 2 \
  -drive file=win2022.qcow2,if=ide \
  -cdrom WindowsServer2022.iso \
  -drive file=autounattend.iso,if=ide,media=cdrom \
  -boot d
```

Two deliberate choices worth flagging at the setup-planning stage, before you even get to the install: `-machine pc` (the older i440fx BIOS-boot chipset, not q35/UEFI) and `-drive ...,if=ide` (IDE disk/CD-ROM controllers, not virtio) were both chosen for maximum guest driver compatibility — Windows Server's installer recognizes IDE/BIOS hardware out of the box, whereas virtio disk/network controllers need drivers injected during setup that Windows doesn't ship natively. This trades performance for "it boots without extra steps," which is the right tradeoff for a lab VM you're building once, not a production hypervisor guest. And because this is an `x86_64` guest running on an Apple Silicon (arm64) host, QEMU falls back to TCG (software) emulation rather than hardware-accelerated virtualization — genuinely slow, expect over an hour for a full Windows Server install, not a misconfiguration to debug.

2.5 Terraform and Azure CLI

Terraform [FREE] (open source CLI; Terraform Cloud has a free tier used in one of the terraform-labs exercises, with paid tiers above it) and the Azure CLI [FREE] (the CLI tool itself is free; what it manages — Azure resources — is billed separately, and the lab's Azure subscription is a free-tier account with a spending cap) are installed for the terraform-labs repo's exercises.

```
brew install terraform azure-cli
```

Verify:

```
terraform -version  
az version
```

Authenticate the Azure CLI interactively — this lab's own documented convention (per the terraform-labs README) is to authenticate only via interactive `az login`, never long-lived service principal secrets checked into anything:

```
az login
```

This opens a browser window for interactive sign-in and, on success, prints the subscription(s) the authenticated account can see as JSON to the terminal. Confirm the active subscription matches the intended free-tier one before running anything:

```
az account show
```

Terraform itself needs no separate login step for Azure — it authenticates through the same Azure CLI session via the `azurerm` provider's `use_cli = true` behavior (or by default, when no explicit credentials block is set and an active `az login` session exists). Confirm a `.tf` directory is ready with:

```
terraform init  
terraform validate
```

`terraform init` downloads the provider plugins a configuration references (the `azurerm` provider, and for the Terraform Cloud exercise, remote backend configuration) into a local `.terraform/` directory. `terraform validate` checks syntax and internal consistency without touching any real cloud resources — safe to run repeatedly, and the right first check after any edit. Per this lab's own honestly-documented status: every exercise in terraform-labs passes `fmt/validate` clean, but none have ever been `terraform apply`'d against the real Azure subscription. That's a deliberate, stated gap, not an oversight in this walkthrough — `apply` needs its own review of the spending cap and MFA state before it's run for real, which is outside the scope of "get your environment set up."

2.6 GitHub CLI (gh)

GitHub CLI [FREE] is used throughout this lab's workflow for pushing the portfolio repos and managing pull requests/issues from the terminal rather than the browser.

```
brew install gh
```

Authenticate:

```
gh auth login
```

This launches an interactive prompt: choose `GitHub.com`, choose `HTTPS` or `SSH` for your preferred git protocol, and choose the browser-based device-code authentication flow (simplest — it opens `github.com` in your browser, shows an 8-character code, you paste it in, and the CLI session authenticates once you approve). Alternatively, `gh auth login --with-token < token.txt` accepts a pre-generated personal access token piped in, useful for non-interactive/scripted setups, but the interactive browser flow is the simpler default for a one-time human setup.

Verify:

```
gh auth status
```

This should report the authenticated account and confirm the token's scopes. From here, `gh repo clone`, `gh pr create`, `gh repo create --public`, etc. all work without further credential setup — this is how this lab's public portfolio repos got created and pushed.

2.7 Node.js, npm, and wrangler

Node.js and npm [FREE] are needed for one specific purpose in this lab: running wrangler, Cloudflare's CLI, which manages the Cloudflare Pages site (`soc-lab-target.pages.dev`) and — critically — tails its live request logs for the Wazuh JSON-log pipeline described elsewhere in this document.

```
brew install node
node --version
npm --version
```

Install wrangler globally, or use `npx` to run it without a global install — this lab uses the global install for convenience since it's invoked frequently during live-traffic testing sessions:

```
npm install -g wrangler
wrangler --version
```

Authenticate wrangler to Cloudflare:

```
wrangler login
```

This opens a browser-based OAuth consent flow. Worth calling out explicitly because it was a real, documented lesson from building this lab: this OAuth login flow grants wrangler a broad, pre-defined set of scopes sufficient for typical Pages/Workers management out of the box. The alternative — hand-creating a scoped Cloudflare API token via the dashboard's custom token permission-group UI — is easy to under-scope by accident. During this lab's setup, the first hand-created token was scoped only for R2 storage (wrong for Pages entirely), and two subsequent attempts were missing the specific "Pages: Edit" permission group before a working token was finally assembled. The `wrangler login` OAuth flow sidestepped all of that by requesting a broader, correct scope in one step. For a single-operator lab like this one, prefer `wrangler login` over hand-scoping a token unless you specifically need a narrowly-scoped token for a CI/automation context.

Once authenticated, the command this lab relies on for live traffic capture (covered in depth in the Cloudflare/live-traffic section of this document) is:

```
wrangler pages deployment tail <deployment-id> --format=json
```

Flagging one real limitation now, at the setup stage, because it affects how you'd operate this day to day: this command binds to a specific deployment ID, not "the current live deployment" — every time the Pages site is redeployed, the tail command targeting the old deployment ID stops receiving traffic and needs to be restarted against the new ID. There is no setup fix for this; it's a real, current limitation of `wrangler pages deployment tail` to plan around operationally.

2.8 Python 3, and why venvs per project (not a global environment)

Python 3 is used across this lab for the relay scripts (MySQL log relay, Cloudflare traffic relay) and the `llm-triage/log-correlation` tooling in the detection-engineering repo. macOS ships a system Python, but do not use it for project work — install a current Python via Homebrew instead:

```
brew install python@3.12
python3 --version
```

The general principle this lab follows, and that this section recommends regardless of which specific scripts you're running: use a fresh virtual environment (venv) per project, never a single global site-packages environment for everything.

Why this matters concretely, not just as best-practice folklore: this lab spans several genuinely different Python dependency sets — a relay script using nothing but the standard library plus a couple of small parsing libraries, `llm-triage` scripts depending on Anthropic's Python SDK, and the `intune-endpoint-health-platform` repo's `duplicate-asset-reconciliation` script depending on `pandas` for fuzzy matching. Installing all of that into one global Python environment means every project's dependencies (and their transitive version constraints) share one namespace — a `pandas` version bump for one project can silently break an unrelated script that imported a different `pandas` API surface, and there's no clean way to answer "what does this specific script actually need to run" months later. A dedicated `venv` per project directory keeps each project's dependency graph isolated, reproducible via a per-project `requirements.txt`, and disposable — deleting a stale project's `venv` directory cleans it up completely with zero risk to any other project.

The standard pattern, run from inside each project directory:

```
cd ~/securitylab/<project-dir>
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt # or pip install <packages>, then pip freeze > requirements.txt
```

`python3 -m venv .venv` creates an isolated Python environment (its own `site-packages`, its own `pip`) in a `.venv` directory local to the project. `source .venv/bin/activate` puts that environment's `python/pip` first on `PATH` for the current shell session — your prompt typically grows a `(.venv)` prefix as a visual confirmation it's active. From that point, `pip install` only affects this project's isolated environment, not the system or any other project's `venv`. `deactivate` (no arguments) exits back to the normal shell environment when you're done.

A `.gitignore` entry for `.venv/` in every project is worth calling out explicitly here too — `venvs` are large, machine-specific, and trivially regenerable from `requirements.txt`, so they should never be committed.

2.9 Verify your setup — full checklist

Run this block top to bottom in a fresh terminal after completing every step above. Each line should succeed and print a version or status, not an error. This is the fastest way to catch a `PATH` problem or an unlaunched Docker Desktop before you're mid-way through standing up a stack and confused about why it's failing.

```
# Homebrew itself
brew --version

# Docker – CLI presence, then daemon health
docker --version
docker compose version
docker info # fails with a socket error if Docker Desktop's app isn't running

# QEMU
qemu-img --version
qemu-system-x86_64 --version

# Terraform + Azure CLI
terraform --version
az version
az account show # confirms an active az login session

# GitHub CLI
gh --version
gh auth status # confirms authenticated
```

```
# Node/npm + wrangler
node --version
npm --version
wrangler --version
wrangler whoami           # confirms authenticated to Cloudflare

# Python
python3 --version
python3 -m venv --help | head -1  # confirms the venv module is available
```

If `docker info` fails: launch Docker Desktop (open `-a Docker`) and wait for the whale icon in the menu bar to stop animating, then retry.

If `az account show` errors with “Please run ‘az login’”: the CLI is installed correctly but no session exists yet — run `az login` and retry.

If `gh auth status` or `wrangler whoami` report not-authenticated: re-run the respective `login` command from the sections above — both use browser-based flows and are quick to redo.

If any brew-installed binary reports “command not found” despite a successful `brew install`: it’s almost always a missing `eval "$(/opt/homebrew/bin/brew shellenv)"` in your shell’s startup file (`~/.zshrc` on the default macOS shell) rather than a failed install — confirm with `echo $PATH | grep -o /opt/homebrew/bin` before reinstalling anything.

With every line in this checklist passing, the host is ready for the tool-by-tool stack setup covered in Part 3.

Part 3.1-3.3: Wazuh, TheHive + Cortex, LocalStack

This section covers three of the tools running in this lab’s local stack (`~/securitylab/`) in real depth: Wazuh (the SIEM/XDR at the center of the lab), TheHive + Cortex (the SOAR/case-management layer, built but not yet wired to Wazuh), and LocalStack (AWS emulation for the `aws-identity-detection` local lab). For each, this section covers what the tool actually is and the architectural principle behind it, the honest paid/free reality, concrete when-to-use/when-not-to guidance, the real install and configuration steps as used in this lab, real troubleshooting, and how the tool fits into the lab’s overall data flow.

3.1 Wazuh

What it is and the principle behind it

Wazuh [FREE] (Apache 2.0, fully self-hosted, no paid tier — the commercial offering is paid support/cloud hosting, not a feature-gated version of the software) is a SIEM/XDR platform built around an agent-plus-manager architecture: lightweight agents run on monitored hosts (or, as in this lab, log sources are fed in without a traditional agent via file-based log collection), forward events to a central manager, and the manager decodes, rules-matches, and indexes those events for search and alerting.

The architectural principle worth understanding, because it explains nearly every design decision downstream: Wazuh was built for host and network telemetry — file integrity monitoring, log collection from installed applications and OS-level sources (syslog, Windows Event Log, audit logs), rootkit/malware detection, vulnerability detection against installed software inventories, and configuration assessment (CIS benchmarks). Its decoder/rule engine is designed around the shapes those telemetry sources take: syslog-like text lines, structured-but-flat log formats, and well-known application log formats it ships pre-built decoders for (Wazuh ships decoders for things like MySQL’s log format out of the box — used directly in this lab, see below). This is fundamentally different from what a cloud identity provider’s audit log looks like — a deeply nested JSON object like an Entra ID sign-in log, with dozens of fields describing conditional access policy evaluation results, risk detections, and device compliance state. Wazuh can ingest

arbitrary JSON (it auto-flattens top-level JSON keys into `data.<field>` for rule matching when you set `<log_format>json</log_format>` on a log source — used in this lab for the Cloudflare Pages traffic pipeline), but it has no purpose-built understanding of what a sign-in log's fields mean the way Microsoft Sentinel's `SignInLogs/AuditLogs` table schema does, with native KQL operators tuned for exactly that data.

Paid vs free tier reality

Fully free and open source — no artificial tier limits, no feature paywall, no user-count cap. The only “paid” surface is Wazuh Inc.’s optional commercial support/cloud-hosted offering, not used in this lab. This is a big part of why it was chosen for the live lab at all.

When to use this / when NOT to use this

Use Wazuh when the telemetry is genuinely host/network-shaped: endpoint logs, file integrity events, application logs with a known text/line format (or well-formed JSON), and you want a real, running, queryable SIEM without paying for one. This lab’s MySQL monitoring and Cloudflare Pages traffic monitoring are both good fits for exactly this reason — they’re log-line and JSON-request telemetry respectively, and both were wired up with zero or minimal custom decoder work.

Do NOT reach for Wazuh as the place to build out cloud identity provider detection logic. This is the single most important honest judgment call in this lab’s design, worth restating explicitly: the Sigma rules and KQL queries in the `detection-engineering` repo that target identity telemetry (Entra ID sign-in logs, audit logs — things like impossible travel, MFA fatigue, privilege escalation via role assignment) were deliberately not ported into Wazuh’s native rule XML format, even though Wazuh was the SIEM actually running in this lab and porting them would have made for a more “complete-looking” local demo. The reasoning: Wazuh’s rule engine, decoder model, and field-matching semantics are built for the shape of host/network logs, not for the deeply nested, policy-evaluation-heavy shape of a cloud IdP’s sign-in event. Forcing an Entra sign-in log through Wazuh’s JSON auto-decode and writing pattern-matching rules against `data.*` fields would technically run, but it would not reflect how anyone actually builds identity detection content in practice — that content is written against Sentinel’s `SignInLogs/AuditLogs` schema and KQL because that’s the tool built for that telemetry shape, with native understanding of conditional access results, risk levels, and location/device correlation that Wazuh has no equivalent concept of. Keeping the Sigma/KQL content targeted at Sentinel’s schema, and not pretending Wazuh is an adequate substitute SIEM for identity telemetry, is the more honest and more professionally representative choice — even though it means the live Wazuh lab and the identity detection content in the portfolio don’t currently talk to each other. That gap is real and stated plainly, not smoothed over.

Internal architecture

The manager is the decode-and-detect engine; the indexer (OpenSearch-based) is the storage/search back-end; the dashboard is the web UI on top of the indexer. All three run as separate containers in the official single-node `wazuh-docker` compose setup, communicating over the compose network.

Install and configuration as used in this lab

The lab uses the official `wazuh-docker` repository’s single-node compose setup, cloned to `~/securitylab/wazuh-docker`:

```
cd ~/securitylab
git clone https://github.com/wazuh/wazuh-docker.git -b v4.9.2
cd wazuh-docker/single-node
```

Before bringing the stack up, Wazuh’s containers require TLS certificates for inter-component communication (manager-to-indexer, dashboard-to-indexer), generated by a dedicated `cert-generator` step defined in the compose file. This is where a real, non-obvious bug shows up on first run:

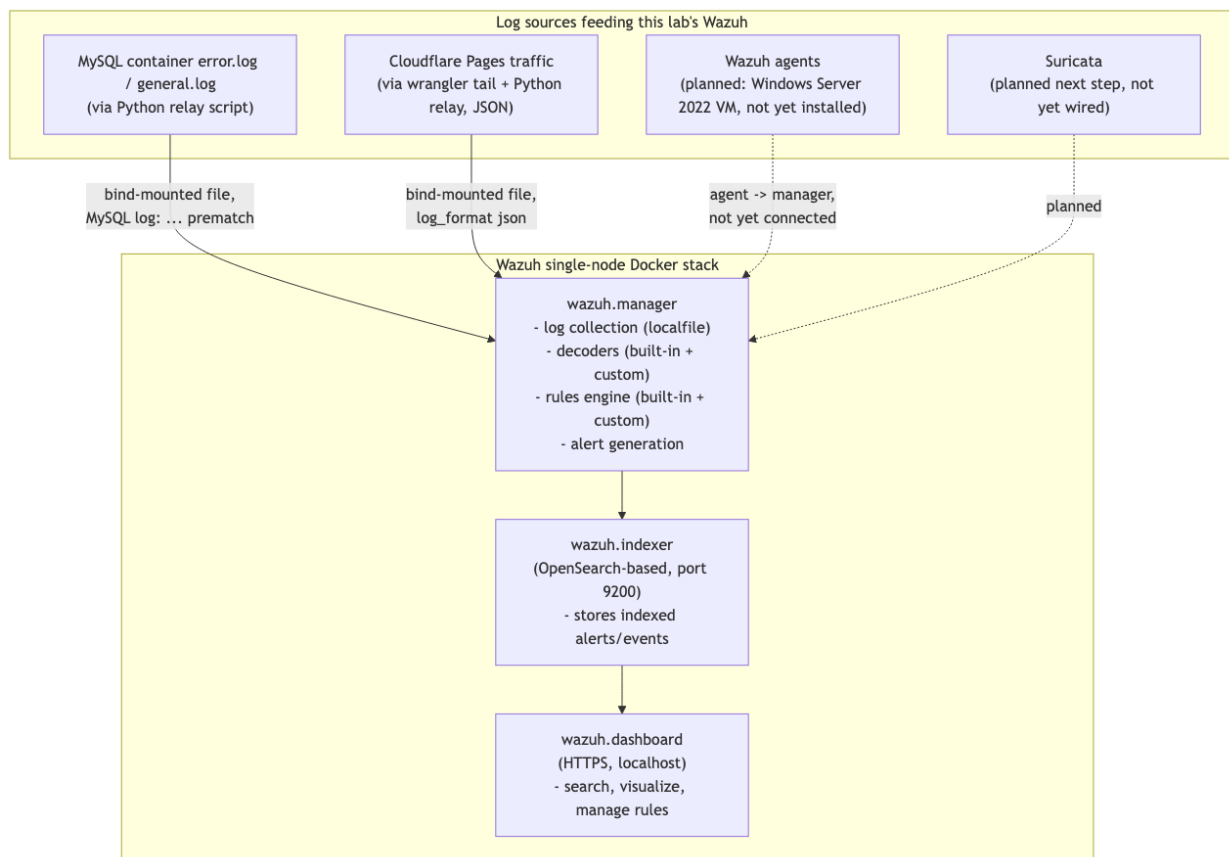


Figure 3: Diagram

The cert-generator permissions bug. The wazuh-docker single-node compose setup runs a one-shot generate-indexer-certs.yml-style cert-generator container that writes certificate files into a local config/wazuh_indexer_ssl_certs/ directory (bind-mounted from the host). On some Docker Desktop configurations — this shows up especially with certain host filesystem permission/ownership defaults on macOS — the cert-generator container's write attempts into that bind-mounted directory fail or produce certs owned in a way the subsequent wazuh.indexer and wazuh.dashboard containers can't read at startup, and those containers then fail to become healthy, typically surfacing as the indexer or dashboard container repeatedly restarting or exiting with a TLS/cert-load error in its logs rather than a clear permissions message up front.

The fix used in this lab: ensure the target certs directory exists and is writable by the container's expected UID before running the cert-generator step, and if certs were already partially generated with wrong ownership, clear and regenerate them fresh:

```
# from wazuh-docker/single-node
rm -rf config/wazuh_indexer_ssl_certs
mkdir -p config/wazuh_indexer_ssl_certs
chmod -R 777 config/wazuh_indexer_ssl_certs # permissive for the one-shot generator step; lab
```

```
docker compose -f generate-indexer-certs.yml run --rm generator
```

Once the generator step completes cleanly (its container exits with status 0, and cert files — CA, node certs, admin certs — appear in config/wazuh_indexer_ssl_certs/), bring the main stack up:

```
docker compose up -d
```

Confirm all three containers report healthy:

```
docker compose ps
```

Expect wazuh.manager, wazuh.indexer, and wazuh.dashboard all in an Up (healthy) state — the indexer in particular can take longer than the other two to pass its health check on first start, since it's doing OpenSearch cluster initialization.

Access the dashboard at <https://localhost> (self-signed cert — the browser will warn, this is expected for a local lab) and the indexer API directly at `:9200`. The default credentials are **admin / SecretPassword**. This is explicitly and deliberately documented here as a “change this” item, not a real secret — any real or long-running deployment of this stack must change the default indexer/dashboard credentials (Wazuh documents the process via its wazuh-indexer internal users config) before it's exposed to anything beyond localhost, and even for a localhost-only lab, leaving defaults in place is a habit worth breaking early.

The MySQL pipeline, as a concrete worked example

This lab stood up a real MySQL 8.0 container (soc-lab-mysql) specifically as a monitored/attacked target, with general query logging and error logging both enabled. Wazuh ships built-in decoders and rules for MySQL's log format already — ruleset/decoders/0150-mysql_decoders.xml and ruleset/rules/0295-mysql_rules.xml — so no custom Wazuh rule was written for this source; the built-in ruleset was sufficient.

The integration challenge wasn't detection logic, it was getting the log lines into the shape Wazuh's built-in decoder expects. A small Python relay script tails MySQL's error.log/general.log, reformats matching lines to fit Wazuh's expected MySQL log: ... prematch format, and appends them to a file bind-mounted into the wazuh.manager container, where a <localfile> block configured with <log_format>syslog</log_format> picks them up:

```
<localfile>
  <log_format>syslog</log_format>
  <location>/var/ossec/logs/mysql/relayed.log</location>
</localfile>
```

Verified live in this lab: four deliberate wrong-password login attempts against the MySQL container correctly fired Wazuh's built-in rule 50106 ("MySQL: authentication failure", level 9), which — because it's a built-in rule, not a custom one — came with Wazuh's out-of-the-box regulatory compliance mapping already attached (PCI DSS, GDPR, HIPAA, NIST 800-53 control references appear directly on the alert). This is a good concrete illustration of the "use Wazuh for what it's built for" principle above: because MySQL log format is squarely inside Wazuh's home turf, this integration needed a small format-adaptor script and zero custom detection logic.

The Cloudflare Pages pipeline, as a second worked example

A real public Cloudflare Pages site (`soc-lab-target.pages.dev`, Cloudflare Pages [FREEMIUM] — static hosting is free, but continuous log streaming/Logpush needs a paid plan, which is exactly why this lab uses `wrangler pages deployment tail` as a workaround instead) logs every real request via a Pages Function (`functions/_middleware.js`) using `console.log` — method, path, client IP via `cf-connecting-ip`, country, user-agent. Since the free tier has no log retention, a Python relay parses `wrangler pages deployment tail --format=json` output live and appends it to a file Wazuh watches with `<log_format>json</log_format>` — Wazuh auto-flattens top-level JSON keys into `data.<field>` for rule matching here, so no custom decoder was needed, but (unlike the MySQL case) a custom rule was, since Wazuh has no built-in decoder or ruleset for arbitrary web request JSON the way it does for MySQL. A custom local rule (id 100011, level 7) fires on any request to `/login.html`, framed as "possible scanner/credential-testing activity" given the page does nothing real. Verified live with real curl traffic producing real alerts.

Troubleshooting

- Containers never reach **healthy**: check `docker compose logs wazuh.indexer` first — indexer startup failures (often cert-related, see above) cascade into dashboard failures since the dashboard depends on a healthy indexer. Don't debug the dashboard container first; work bottom-up.
- Cert-generator permission errors: covered above — clear the certs directory, ensure it's writable, rerun the generator before `docker compose up`.
- Default credentials: `admin/SecretPassword` must be treated as a "day one, change this" item — worth doing even before you consider the lab "done," since it's a five-minute fix and a real bad habit to leave in place.
- JSON log source producing no alerts: confirm `<log_format>json</log_format>` is set (not `syslog`) on that specific `<localfile>` block, and confirm the custom rule's XPath/field references match the actual `data.<field>` names Wazuh flattened the JSON into — check via the dashboard's alert JSON view on a raw (non-matching) event first to confirm field names before writing the rule.
- Resource pressure on a 16GB host: the indexer (OpenSearch) is the heaviest of the three containers; if the Mac becomes sluggish with multiple stacks running, this is usually the first place to check memory usage (`docker stats`).

How Wazuh fits into the overall lab architecture

Wazuh is the lab's central SIEM — the place host/network/application telemetry converges, gets decoded, and gets rule-matched into alerts. As of this document, it consumes MySQL logs and Cloudflare Pages traffic, with Suricata network IDS output and a Wazuh agent on the Windows Server VM both identified as the next real sources to wire in (neither done yet). It does not currently forward alerts anywhere — TheHive/Cortex are built but not wired to consume Wazuh alerts, which is a stated, real gap, not an oversight.

3.2 TheHive + Cortex

What it is and the principle behind it

TheHive [FREE] (TheHive 5 is open source under AGPL for self-hosted use — StrangeBee, the company behind it, also sells a commercial “TheHive:Cloud”/enterprise offering, but the self-hosted version run in this lab is free) is a case-management/SOAR platform: it exists to take raw alerts and turn them into structured, trackable cases with observables (IPs, hashes, domains, etc.), tasks, and an audit trail of analyst actions — the workflow layer a SIEM like Wazuh doesn’t provide on its own. Cortex [FREE], same licensing model, is TheHive’s companion enrichment/response engine: it runs “analyzers” (enrichment lookups against threat intel sources, sandboxes, reputation services) and “responders” (automated actions) against observables submitted to it, either manually or triggered from TheHive.

The architectural principle: a SIEM’s job is detection (turn raw telemetry into an alert); a SOAR’s job is response workflow (turn an alert into a tracked investigation with enrichment, assignment, and resolution status). They’re deliberately separate concerns with separate tools in this lab’s design — Wazuh does not attempt to be a case-management system, and TheHive does not attempt to be a log-search/detection engine. TheHive’s own architecture reflects a document-oriented workflow (cases, tasks, observables, as first-class linked objects) backed by a search index for querying across cases — hence Cassandra (the durable document/case store) plus Elasticsearch (search) as its two backing data stores, a heavier footprint than a typical single-database app, but one that maps directly to “store structured case data durably, and make it fully searchable” as two genuinely different jobs.

Paid vs free tier reality

Self-hosted TheHive 5 and Cortex are both free and fully functional with no artificial feature gating for this lab’s purposes — the commercial offering from StrangeBee is a hosted/managed version plus enterprise support, not a more-capable version of the self-hosted software for core case-management functionality.

When to use this / when NOT to use this

Use TheHive+Cortex when you actually have a volume or complexity of alerts that benefits from structured case tracking and repeatable enrichment workflows — multiple analysts, an audit-trail requirement, or enrichment lookups you want automated (Cortex analyzers) rather than manually run every time. Its value is entirely dependent on having something feeding it alerts in the first place; a SOAR platform with nothing wired into it is just an empty case-tracking UI.

Do NOT stand this up expecting it to add value on day one of a small lab — and this is the honest, stated state of this lab right now: TheHive/Cortex are built and their containers exist, but they are currently stopped/ idle, and — more importantly — not actively wired to Wazuh’s alerts. That’s a known, documented gap, not a hidden one. The realistic sequence for a lab this size is: get a SIEM producing real alerts first (done, via Wazuh), then build the alert-forwarding integration (Wazuh has a native TheHive connector/webhook path for this), and only then does running TheHive continuously start paying for its own resource cost. Running it idle “because it’s part of the architecture” without a live feed is a fair thing to note in a portfolio as planned/next-step work, but running it disconnected from real alerts on a resource-constrained 16GB host with no current benefit is not worth the standing memory cost.

Internal architecture

Install and configuration as used in this lab

TheHive’s stack was brought up previously via Docker Compose, defining Cassandra, Elasticsearch, Cortex, and TheHive as four services on a shared compose network — Cassandra and Elasticsearch as the backing stores, Cortex as a separate service TheHive is configured to talk to via its API URL and an API key generated in Cortex’s own first-run setup, and TheHive itself as the top-level UI/API service.

The general shape of that compose bring-up:

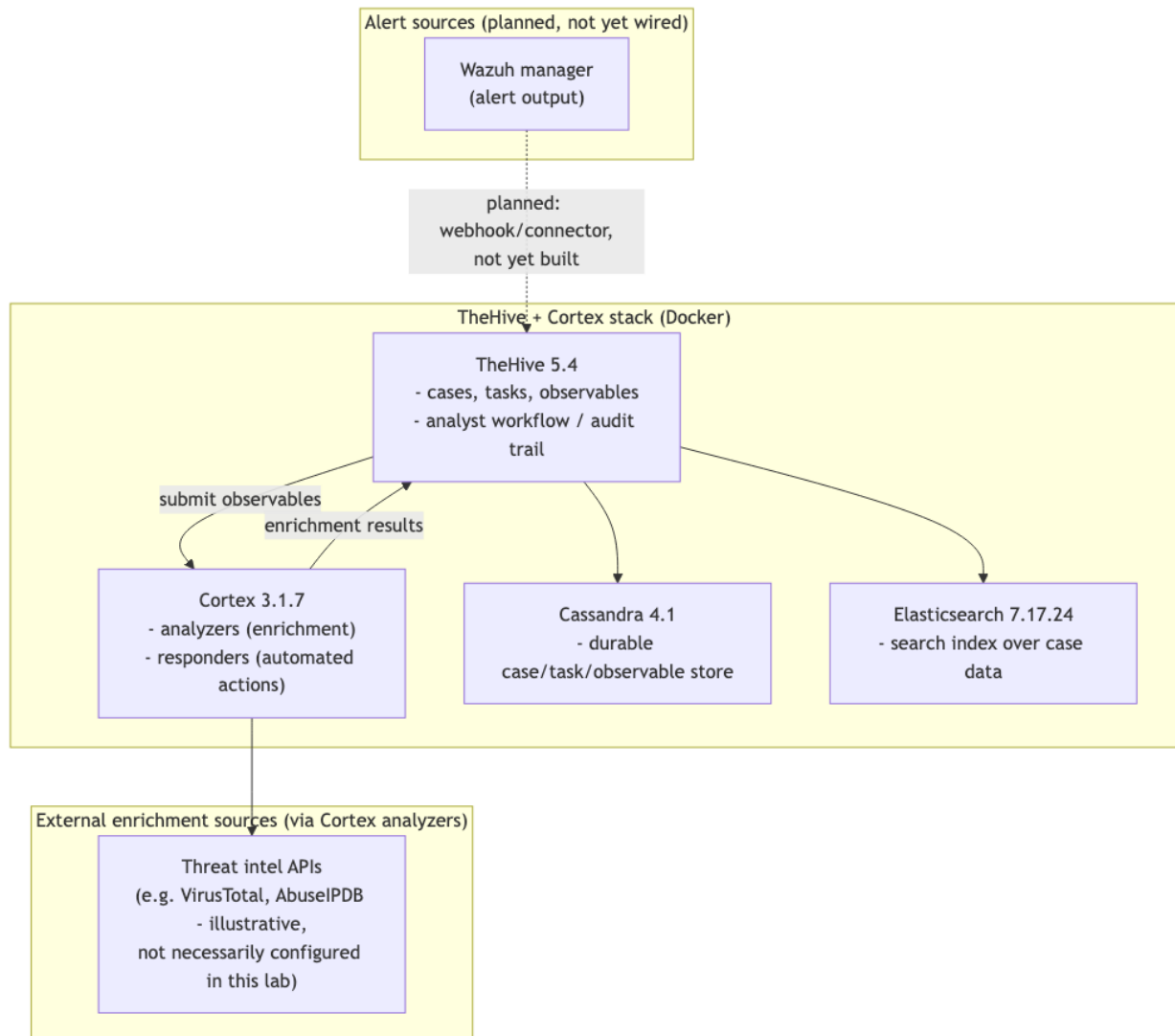


Figure 4: Diagram

```
cd ~/securitylab/thehive-project # illustrative path, matches this lab's actual layout convention
docker compose up -d
docker compose ps
```

First-run configuration order matters and is a real, non-obvious part of getting this stack working: Elasticsearch and Cassandra need to be healthy before Cortex and TheHive can successfully connect to their respective backing stores, and Cortex itself needs its initial admin account and organization created through its own web setup wizard before an API key can be generated for TheHive to use to talk to it. The practical sequence: bring the stack up, wait for Cassandra/Elasticsearch to report healthy, open Cortex's UI first and complete its setup wizard (creates the initial superadmin, then an organization, then an analyzer-running user with an API key), then configure that key into TheHive's connector settings for Cortex integration, then finally open TheHive's own UI and complete its first-run setup (initial org, initial admin user).

Default credentials are another explicit "must change" item here, same principle as Wazuh: both TheHive and Cortex prompt for initial admin/org setup on first launch rather than shipping a fixed default login, which is a better default than Wazuh's fixed admin/SecretPassword — but whatever password is set during that first-run wizard needs to be treated with the same seriousness as any other credential, not left as a throwaway lab password if this stack is ever exposed beyond localhost.

Troubleshooting

- Elasticsearch fails to start / exits immediately: Elasticsearch 7.x has real host-level requirements — most commonly `vm.max_map_count` needing to be raised (a classic Elasticsearch-in-Docker issue, not specific to this stack) — check container logs for a `max virtual memory areas vm.max_map_count [65530] is too low` message, which is the concrete symptom if this hasn't been set on the Docker Desktop VM.
- Cortex can't reach Elasticsearch/TheHive can't reach Cassandra: usually a startup-ordering issue — confirm both backing stores show healthy in `docker compose ps` before troubleshooting the app-layer services that depend on them.
- Containers "up" but the UI never becomes reachable: check that the expected ports aren't already bound by another stack — this is a real risk in a lab running several Docker Compose projects side by side, and worth checking with `docker ps` across all running containers, not just this stack's `docker compose ps`.
- x86 emulation slowness is not the cause of any of the above: worth noting for clarity — TheHive's containers are typically available as native arm64 images or run fine under Docker Desktop's transparent emulation without the severe slowness seen in the Windows QEMU VM; if this stack feels slow, it's far more likely genuine resource contention (Cassandra and Elasticsearch are both JVM-heavy) on a 16GB host running multiple stacks at once than architecture emulation overhead.

How TheHive + Cortex fits into the overall lab architecture

Intended role: the case-management/response layer downstream of Wazuh — Wazuh alerts would forward into TheHive as cases, Cortex would enrich observables extracted from those alerts. Actual current state: built, capable, currently idle, and explicitly not yet wired to Wazuh's alert output. This is the most honestly-stated gap in the lab's architecture and worth presenting as exactly that in this document — a real next step, not a finished integration.

3.3 LocalStack

What it is and the principle behind it

LocalStack [FREEMIUM] (the Community edition used in this lab is free and open source, covering core services like IAM, S3, and CloudTrail emulation sufficient for this lab's purposes; LocalStack Pro is a paid tier adding more services and features, not needed here) emulates AWS service APIs locally, so code and detection logic written against real AWS APIs (the AWS SDK, aws CLI) can run against a local emulated

endpoint instead of a real AWS account. The principle: for testing detection logic — e.g. “does this IAM privilege-escalation pattern actually get logged the way I expect, and does my detection query actually catch it” — you need to generate the shape of real AWS API activity and its logs, but you do not need (and for cost/safety reasons, usually do not want) to do that against a billed, real AWS account, especially when deliberately generating privilege-escalation-shaped activity for test purposes.

Paid vs free tier reality

LocalStack Community [FREE] covers the core services this lab’s `aws-identity-detection` work needs — IAM, and enough surrounding service emulation to exercise privilege-escalation-relevant API calls. LocalStack Pro is a paid subscription adding broader service coverage and features like advanced persistence — not required for this lab’s stated purpose.

When to use this / when NOT to use this

Use LocalStack when you want to exercise AWS API call patterns and their logging/detection surface without touching a real, billed AWS account — exactly the `aws-identity-detection` use case here: testing that IAM privilege-escalation detection logic actually fires against realistic API call sequences, safely and repeatably, without needing a real AWS account or worrying about cost or blast radius from deliberately-risky test actions.

Do NOT treat LocalStack’s emulation as a perfect stand-in for real AWS behavior, especially for identity/IAM edge cases — service emulation fidelity varies by service and by LocalStack edition, and IAM policy evaluation semantics in particular are complex enough that subtle real-AWS behavior (specific deny-precedence edge cases, service-specific resource policies, actual CloudTrail event field completeness) is not guaranteed to be bit-for-bit replicated locally. LocalStack is the right tool for building and dry-running detection logic against realistic API call shapes cheaply; it is not a substitute for eventually validating that same detection logic against a real AWS account’s real CloudTrail output before trusting it in production. This lab is honest that its `aws-identity-detection` content has been validated against LocalStack, not against a real AWS account — the same “don’t overclaim” discipline applied elsewhere in this document (e.g. `terraform-labs` never having been apply’d for real).

Internal architecture

LocalStack’s architecture is a single container exposing one edge endpoint (by default port 4566) that internally routes requests to the correct emulated service implementation based on the AWS API being called — from the client’s perspective it looks like one AWS-shaped endpoint, which is exactly the point: existing AWS CLI/SDK code needs only an endpoint-URL override to run against it, not a rewrite.

Install and configuration as used in this lab

```
cd ~/securitylab/aws-identity-detection-lab # illustrative path matching this lab's layout co
docker compose up -d localstack
docker compose ps
```

A typical LocalStack compose service definition looks like:

```
services:
  localstack:
    image: localstack/localstack:3.8
    ports:
      - "4566:4566"
    environment:
      - SERVICES=iam,sts,cloudtrail,s3
      - DEBUG=0
```

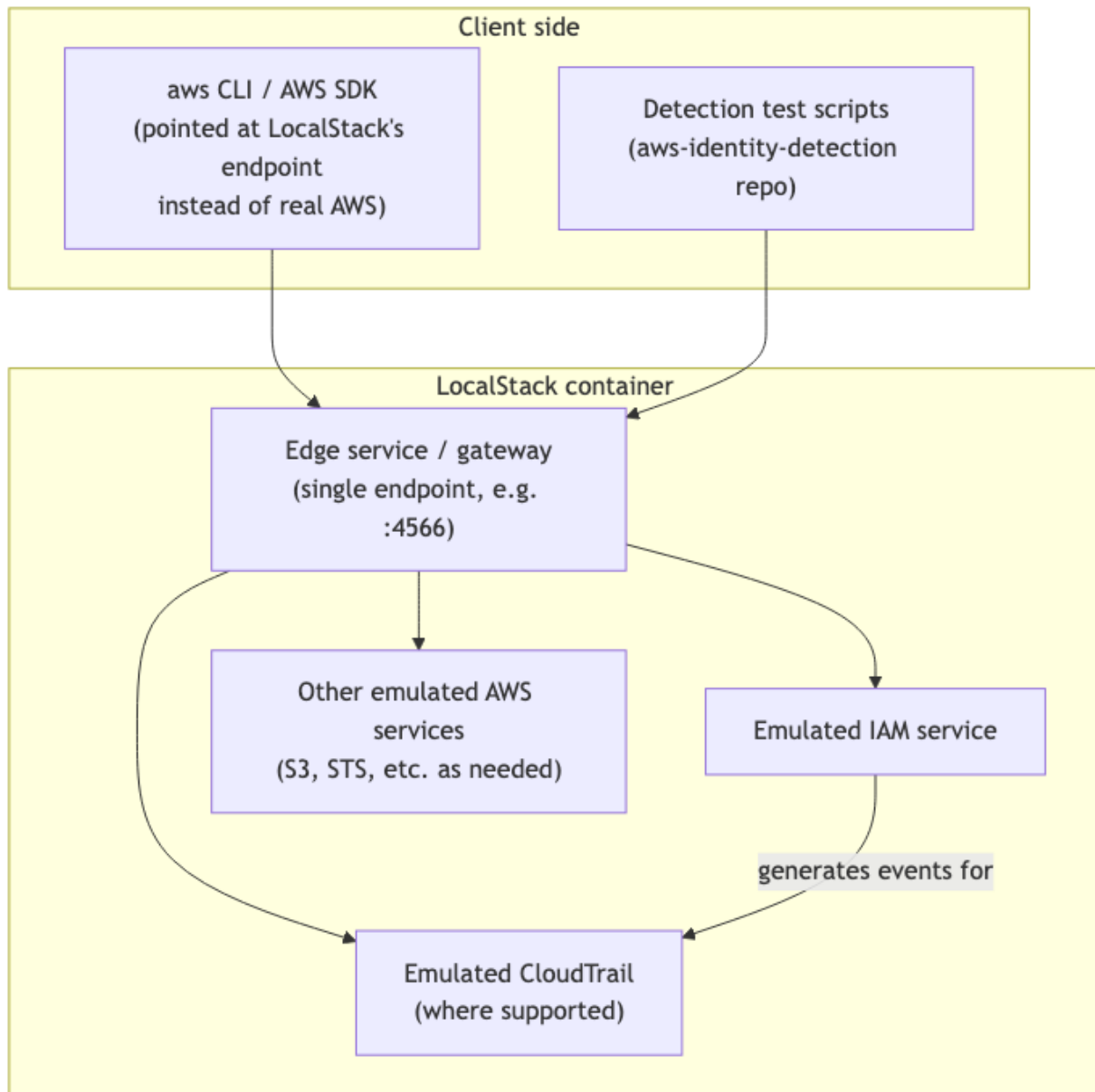


Figure 5: Diagram


```
volumes:
- "./localstack-data:/var/lib/localstack"
```

Point the AWS CLI at it explicitly per-command with `--endpoint-url`, or configure a dedicated profile so it's not accidentally used against real AWS:

```
aws --endpoint-url=http://localhost:4566 iam create-user --user-name test-privesc-user
aws --endpoint-url=http://localhost:4566 iam list-users
```

Using a distinct named AWS CLI profile (e.g. `aws configure --profile localstack` with dummy access keys — LocalStack does not validate real AWS credentials by default) is worth doing deliberately, specifically to avoid the real risk of a copy-pasted command accidentally running against a real AWS account because the default profile was active. This is a genuine operational safety habit worth stating plainly, not a hypothetical concern.

Troubleshooting

- **docker info**/connection-refused on port 4566: same root cause category as Docker Desktop issues elsewhere in this document — confirm the container is actually running (`docker compose ps`) and the port mapping matches what your CLI/SDK is targeting.
- A service call returns “not implemented” or an unexpected error: check that the specific AWS service is listed in the `SERVICES` environment variable for the container (Community edition emulates a defined set of services — an unlisted service simply isn’t available) and check LocalStack’s own compatibility notes for that specific API action, since emulation completeness varies per action within a service, not just per service.
- Detection logic passes against LocalStack but needs re-validation before being trusted: not a bug to “fix,” but the correct honest framing to carry forward — LocalStack validation is a real, useful step, not a final one, for exactly the IAM-policy-nuance reasons above.
- Apple Silicon note: LocalStack ships native arm64-compatible images, so this is one of the few pieces of this lab’s stack that does not suffer the x86 emulation slowness the Windows Server VM does under QEMU — worth explicitly distinguishing from the QEMU case so the two aren’t conflated as “the same kind of slow.”

How LocalStack fits into the overall lab architecture

LocalStack is scoped narrowly and intentionally: it exists solely to support the `aws-identity-detection` repo’s local lab, letting AWS IAM/privilege-escalation detection content be exercised against realistic API call shapes without a real AWS account. It does not feed Wazuh, TheHive, or any other part of this lab’s stack — it’s a self-contained test harness for one specific repo’s detection content, currently stopped/idle between active development sessions, which is the appropriate operating mode for a tool whose job is “be available when testing that specific content,” not “run continuously as part of the lab’s live telemetry pipeline.”

Part 3.4-3.6: Suricata, MySQL, Cloudflare Pages + Workers/Functions

This part covers three very different kinds of monitored target in the lab: a network intrusion detection engine that is built but not yet wired in (Suricata), a database that was stood up specifically to be attacked and watched (MySQL), and a real, live, public web target whose logs have to be smuggled out of a platform that doesn’t want to give them to you for free (Cloudflare Pages + Functions). Read together, they make a useful point: how much detection work a target needs depends entirely on how close its native log format already is to what your SIEM expects. MySQL needed zero custom Wazuh rules. Cloudflare needed a custom rule but no custom decoder. Suricata, if and when it gets wired in, will need real decoder work because its EVE JSON schema is nothing like either of the other two. That contrast is the throughline of this part.

Part 3.4: Suricata — Network Intrusion Detection

What a network IDS actually does

Suricata [FREE] (open source, GPLv2, maintained by the Open Information Security Foundation) is a network intrusion detection and prevention engine (NIDS/NIPS). At the level that matters for this lab, a network IDS sits somewhere it can see raw traffic — a mirrored switch port, a tap, a container network namespace, a host interface in promiscuous mode — and inspects packets as they cross the wire, independent of whatever the endpoint itself logs or doesn't log. This is a fundamentally different vantage point from everything else in this lab so far. Wazuh's other inputs (MySQL logs, Cloudflare Function logs) are all application-layer, after-the-fact records: something already decided to write a log line, and Wazuh consumes that decision. A NIDS doesn't wait for an application to decide to log anything. It sees the packets themselves, so it can catch things no application log would ever mention — a port scan, a C2 beacon using a protocol nobody instrumented, a plaintext credential crossing the wire in a protocol that was never supposed to carry it in the clear.

Signature-based detection vs anomaly detection

Suricata (like its predecessor/sibling Snort) is primarily a signature-based engine, though it supports some anomaly-style features. The distinction matters and is worth being precise about:

Signature-based detection matches traffic against a database of known patterns — a specific byte sequence in a payload, a specific combination of protocol fields, a known-bad IP or domain, a regex against a URL. Suricata uses rules written in a specific syntax (largely compatible with Snort's rule language), for example a simplified illustrative rule shape:

```
alert tcp any any -> $HOME_NET 3306 (msg:"Possible MySQL brute force";  
  flow:to_server; content:"|4d 79 53 51 4c|"; threshold:type both, track by_src,  
  count 5, seconds 60; sid:1000001; rev:1;)
```

(This is an illustrative example of the rule shape, not a rule that has been written, loaded, or tested against Suricata in this lab — Suricata is currently stopped/idle, as noted below.) The strength of signature-based detection is precision: when a signature fires, you know exactly what matched and why, which makes triage fast and false-positive tuning tractable. The weakness is coverage: a signature-based engine can only catch what someone already wrote a signature for. Zero-days, novel malware, and attackers who deliberately avoid known IOCs sail straight through.

Anomaly-based detection (statistical baselining, machine-learning classifiers, protocol-anomaly detection) instead tries to learn what “normal” traffic looks like for an environment and flag deviations from that baseline — a host that suddenly starts talking to a hundred new destinations, a protocol used at a volume or time-of-day it's never used at before. The strength is that it can catch genuinely novel attacks with no prior signature. The weakness is noise: baselines drift, “normal” is messy in a real network, and anomaly detectors are notorious for high false-positive rates unless heavily tuned — which is itself a nontrivial ongoing labor cost, not a one-time setup cost.

In production, the two approaches are complementary, not competing: signature-based engines like Suricata catch known-bad fast and cheap; anomaly/behavioral layers (increasingly, this is what “XDR” vendors sell as their differentiator on top of a base like this) catch what signatures miss, at the cost of more tuning and more analyst judgment on each alert.

Why Suricata is “the obvious next thing to feed into Wazuh” — and why it isn't wired in yet

Suricata's container (jasonish/suricata:latest) was built previously in this lab and is currently stopped/idle — it has not been running during this document's session, and it is not currently feeding Wazuh anything. This is a deliberate, honestly-documented gap, not an oversight being glossed over.

The reasoning for calling it “the obvious next thing” is architectural, not aspirational hand-waving: every other source Wazuh currently ingests in this lab (MySQL logs, Cloudflare Function logs) is an application log —

something a specific piece of software chose to write down. Suricata would give Wazuh its first network-layer visibility: traffic Wazuh currently cannot see at all, regardless of what any application on top of that traffic decides to log. Concretely, Suricata emits an EVE JSON log (`eve.json`) with event types like `alert`, `dns`, `http`, `tls`, `flow` — Wazuh's `<log_format>json</log_format>` localfile mechanism (the same mechanism already proven out for the Cloudflare relay, see Part 3.6 below) is the natural ingestion path, since EVE JSON is already structured JSON Wazuh can auto-decode into `data.<field>` keys without a custom decoder. What would be needed is custom Wazuh rules that match on Suricata's specific field names (`alert.signature`, `alert.category`, `src_ip`, `dest_port`, etc.) — Wazuh ships no built-in Suricata ruleset the way it does for MySQL, so this would look architecturally like the Cloudflare integration (custom rule, no custom decoder) rather than the MySQL one (fully built-in). That parallel is exactly why it's called out as “next”: the pattern is already proven twice in this lab, just not yet applied to network traffic.

When to use this / when NOT to use this

Use a NIDS like Suricata when you need visibility into traffic your endpoints and applications don't or can't log — lateral movement between hosts that never touches an instrumented application, C2 beaconing, protocol misuse, traffic to/from known-bad infrastructure. Don't expect it to replace endpoint or application logging — it has zero visibility into what happens inside an encrypted TLS session past the handshake/SNI (without TLS interception, which this lab does not do and which carries its own significant tradeoffs), and it says nothing about what a user did once authenticated. In this lab specifically: do not describe Suricata as “deployed” or “detecting” anything right now — it is a stopped container with real prior build work behind it, not a live control.

Part 3.5: MySQL as a Monitored, Attacked Lab Target

Why a real MySQL container

A real MySQL 8.0 container (`soc-lab-mysql`) — [FREE] (MySQL Community Edition, GPL) — was stood up in this lab session specifically to be a monitored and attacked target, not a functional application backend. The point was to generate real authentication log lines from real failed login attempts and prove those lines reach Wazuh and fire a real rule — end to end, not simulated.

`general_log` vs `error_log` — and why that distinction is a real production tradeoff

MySQL ships (at minimum) two separate logging subsystems relevant here, and conflating them is a common mistake:

- `error_log` — logs server-level events: startup/shutdown, crashes, and, critically for this lab, authentication failures (failed login attempts show up here as `Access denied for user '...'@'...'`). This log is bounded in practice — it only writes on events that actually matter (errors, warnings, connection failures), so its volume is proportional to how much is going wrong, not to overall traffic.
- `general_log` (the general query log) — logs every single statement the server executes: every `SELECT`, every `INSERT`, every connection and disconnection, from every client, successful or not. This is enabled in this lab (both general query logging and error logging are on for `soc-lab-mysql`) specifically for lab visibility.

The reason this distinction matters far beyond this lab is a real, recurring production tradeoff: `general_log` gives you complete audit visibility — literally every query anyone ran — but at unbounded, traffic-proportional cost. On a busy production database, the general query log can dwarf the actual data volume of the database itself in short order, because it captures full statement text for every single query, all the time. That has three concrete costs: disk (the log file grows without bound if not rotated aggressively), I/O performance (every query now incurs an extra synchronous or buffered write to the log target, which for `FILE` output competes with the database's own I/O), and potentially very serious data-exposure risk (query text often contains literal values — including, in a badly designed application, plaintext credentials or PII passed as query parameters — sitting in a log file with whatever access controls that file happens to have, which are usually much looser

than the database's own access controls). This is precisely why `general_log` is off by default in MySQL and why production DBAs enable it only surgically — for a bounded diagnostic window, or filtered to specific users/statement types — rather than leaving it on indefinitely. In this lab it's left on because the lab's entire purpose is generating visible activity to detect, and disk/performance cost is a non-issue at this scale; a security engineer reading this should not walk away thinking “just turn `general_log` on in production,” they should walk away understanding why that's a real decision with a real cost, made deliberately here because the tradeoff doesn't bite at lab scale.

Wazuh's built-in MySQL decoder and rules

Wazuh ships built-in support for MySQL's log format out of the box, in two files inside its default ruleset:

- `ruleset/decoders/0150-mysql_decoders.xml` — the decoder(s) that parse a MySQL log line into fields Wazuh's rule engine can match against.
- `ruleset/rules/0295-mysql_rules.xml` — the rule definitions that fire based on what the decoder extracted, including rule ID 50106, “MySQL: authentication failure” (level 9).

This is the single biggest practical difference between the MySQL integration and the Cloudflare integration described in Part 3.6: no custom Wazuh rule or decoder was needed for MySQL at all. Wazuh already speaks MySQL's log dialect natively. The only integration work required was getting MySQL's actual log lines into the exact shape Wazuh's existing decoder is written to expect.

The prematch problem, and how the relay bridges it

Wazuh's built-in MySQL decoder is written around a specific prematch pattern — conceptually, it expects to see a log line that begins with a literal string resembling `MySQL log:` followed by the actual event text (this is the decoder's anchor for “this line is MySQL, hand it to the MySQL decoding logic”). Real MySQL log output — whether from `error.log` or `general.log` — does not naturally start with that string. MySQL writes its own native format (timestamped, with its own prefix conventions per log type), and it has no setting that reformats its output to match an arbitrary third-party SIEM's expected prematch string.

This is the gap a small Python relay script exists to bridge. The relay's job, conceptually, is:

1. Tail MySQL's `error.log` and `general.log` files continuously (the same tailing-a-growing-file pattern used elsewhere in this lab, e.g. the Cloudflare wrangler `tail` relay, though this one reads local files directly rather than a streaming CLI).
2. Filter/match each new line against the patterns that indicate an event Wazuh's decoder cares about — in particular, authentication failure lines (`Access denied for user ...`).
3. Reformat each matching line by prepending the literal `MySQL log:` prefix (and otherwise preserving the original event text) so the resulting line is byte-for-byte shaped the way Wazuh's built-in `0150-mysql_decoders.xml` decoder is written to expect.
4. Append the reformatted line to a separate output file that is bind-mounted into the Wazuh manager container's filesystem.
5. Wazuh's manager, via a `<localfile>` block configured with `<log_format>syslog</log_format>` pointed at that bind-mounted file path, picks up each new appended line, runs it through the decoder pipeline, and — because the line now matches the expected prematch — the MySQL decoder successfully parses it and hands it to the rule engine.

The relay is, in effect, a translation layer between “the log format the source software actually writes” and “the log format the SIEM's built-in content was written to expect.” This is an extremely common real-world pattern — very few security tools' out-of-the-box decoders match an arbitrary application's actual log output byte-for-byte, and the practical fix is almost always a small normalization step ahead of ingestion, not rewriting the SIEM's decoder.

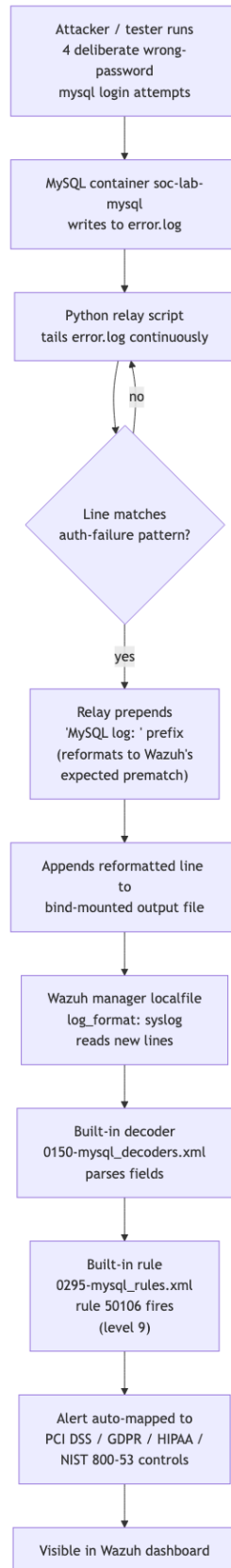


Figure 6: Diagram
29

The verified result

This was verified live, not simulated: 4 deliberate wrong-password login attempts against `soc-lab-mysql` produced 4 corresponding `Access denied` lines in MySQL's error log, which the relay picked up, reformatted, and appended to the bind-mounted file, which Wazuh's manager ingested and correctly decoded, causing rule 50106 ("MySQL: authentication failure", level 9) to fire. Because this rule already ships in Wazuh's default ruleset with compliance mappings attached, the resulting alert automatically carried PCI DSS, GDPR, HIPAA, and NIST 800-53 control references — no manual mapping work was required, purely because the rule was already tagged that way in Wazuh's shipped content. This is a useful, honest illustration of what "compliance mapping" means in a SIEM in practice: it's metadata attached to a rule ahead of time by whoever wrote the rule, surfaced automatically when the rule fires — not something computed dynamically from the alert content, and not something this lab built itself.

When to use this / when NOT to use this

Use built-in vendor decoders/rules whenever the target software is common enough that the SIEM vendor already wrote content for it — MySQL, SSH, sudo, Windows Security Event Log, and similar high-prevalence sources usually have mature built-in coverage in Wazuh, and reinventing that coverage from scratch is wasted effort. Do NOT assume built-in coverage means zero integration work, though — as shown here, the log source still has to actually reach the SIEM in the format the decoder expects, and bridging "real log format" to "decoder-expected format" is very often still necessary, via a relay/normalization step exactly like the one built here. Do not enable full general query logging on anything that isn't a disposable lab target without deliberately weighing the disk, I/O, and data-exposure costs described above.

Part 3.6: Cloudflare Pages + Workers/Functions

Static hosting vs edge compute — the distinction that matters here

Cloudflare Pages [FREEMIUM] (static hosting is free; Logpush/log retention requires a paid plan — the specific limit that matters most for this lab, explained below) is, at its core, a static site host: it serves pre-built HTML/CSS/JS files from Cloudflare's edge network, with no server-side execution per request by default. Cloudflare Pages Functions (built on the same underlying platform as Cloudflare Workers [FREEMIUM], free tier with a daily request-count cap) add edge compute on top of that static hosting — small JavaScript functions that execute at Cloudflare's edge, in front of or alongside the static content, for every matching request. This is the same static-vs-dynamic distinction that shows up everywhere in web architecture, just pushed all the way out to the CDN edge instead of living in an origin server: static hosting alone cannot observe, log, or react to individual requests beyond what the CDN's own access logs might capture (and on Free tier, as below, that's effectively nothing persistent); a Function can run arbitrary code on every request — including, as built here, logging it.

The lab's real setup

The live target is `soc-lab-target.pages.dev`, a real, public, Free-tier Cloudflare Pages site — three static pages (a home page, a dummy `/login.html` that logs to the browser console and does nothing else, a dummy `/contact.html`), every page tagged `noindex, nofollow` and carrying an on-page banner disclosing plainly that this is a synthetic lab fixture, not a real company, and collects nothing. A Cloudflare Pages Function at `functions/_middleware.js` runs on every real request to the site and logs method, path, client IP (via the `cf-connecting-ip` header Cloudflare attaches), country, and user-agent via `console.log`.

Why Free tier has no Logpush, and what that means practically

Cloudflare's Logpush product — continuous, durable streaming of request/log data out to an external destination (S3, a SIEM, etc.) — is a paid-plan feature. On Free tier, `console.log` output from a Pages Function

exists only transiently, surfaced through Cloudflare's real-time tailing tooling, with no built-in persistence and no way to query "what happened yesterday" after the fact. Practically, this means the Free tier gives you visibility into traffic happening right now, while you are actively watching, and nothing else — there is no log storage to fall back on, no historical query capability, nothing to backfill from if you weren't tailing at the moment a request happened. Any persistence has to be built by the user, which is exactly what this lab's Python relay does (see below): live-tail, capture, and durably append to your own file before Cloudflare's transient view of that request disappears.

The wrangler-tail limitation: bound to a deployment ID

Live traffic is captured via:

```
wrangler pages deployment tail --format=json
```

[FREE] (wrangler is Cloudflare's free CLI). The critical limitation, verified in this lab and worth stating plainly: `wrangler pages deployment tail` is bound to a specific deployment ID, not to the project or domain in general. Every time the Pages site is redeployed — including for something as small as a content typo fix — Cloudflare creates a new deployment ID, and the previously running tail session silently stops receiving new traffic because it's still attached to the old, now-stale deployment. This is a real, documented limitation that breaks the intuitive expectation of "persistent monitoring": you cannot start one tail process and assume it keeps working indefinitely across the site's lifecycle. In practice this means the tail process has to be restarted against the new deployment ID after every deploy, which is a manual step today in this lab — there is no automated "always follow the latest deployment" mode being used here. Anyone building this for real, persistent monitoring would need to script deployment-ID discovery and tail-process restart around every deploy, or move to a paid plan with Logpush instead of relying on `wrangler tail` at all.

The relay and Wazuh ingestion — and why this needed a custom rule but no custom decoder

The `wrangler pages deployment tail --format=json` output is parsed by a Python relay script and appended to a file that Wazuh watches with `<log_format>json</log_format>` configured on the corresponding `<localfile>` block. Wazuh's JSON log format handling auto-decodes top-level JSON keys into `data.<field>` namespacing for rule matching — so `{"method": "GET", "path": "/login.html", "ip": "203.0.113.7", ...}` becomes matchable as `data.method`, `data.path`, `data.ip`, and so on, with no custom decoder required. This is a direct contrast with the MySQL integration in Part 3.5: MySQL required reformatting the log line (via the relay) to fit an existing decoder, because Wazuh ships a purpose-built MySQL decoder and ruleset. Cloudflare's Pages Function log is arbitrary JSON with field names nobody at Wazuh anticipated, so there is no purpose-built decoder to fit — but because it's already well-formed JSON, Wazuh's generic JSON handling parses its structure for free. What Cloudflare's traffic does need, that MySQL's didn't, is a custom rule — Wazuh has no pre-existing knowledge of what a request to `/login.html` on this specific site should mean, because no vendor ships detection content for one lab's own bespoke web app. A custom local rule (id 100011, level 7) was written to fire on any request to `/login.html`, framed as "possible scanner/credential-testing activity."

This pairing — MySQL (built-in decoder + built-in rule, custom reformatting relay only) vs Cloudflare (auto-parsed JSON, custom rule, no decoder needed) — is a genuinely useful teaching contrast about how detection engineering effort is distributed. The work is never uniform across sources; it shifts entirely based on how close the target's native output already is to (a) a format the SIEM's parser understands structurally, and (b) a format the SIEM's rule content already has opinions about semantically. A source can need heavy help on one axis and none on the other, as both of these prove in opposite ways.

Verified result

This was verified live with real `curl` traffic against the public site producing real alerts in Wazuh — a genuine end-to-end path from an actual HTTP request, through Cloudflare's edge, through the tail-and-relay pipeline, into a firing custom rule.

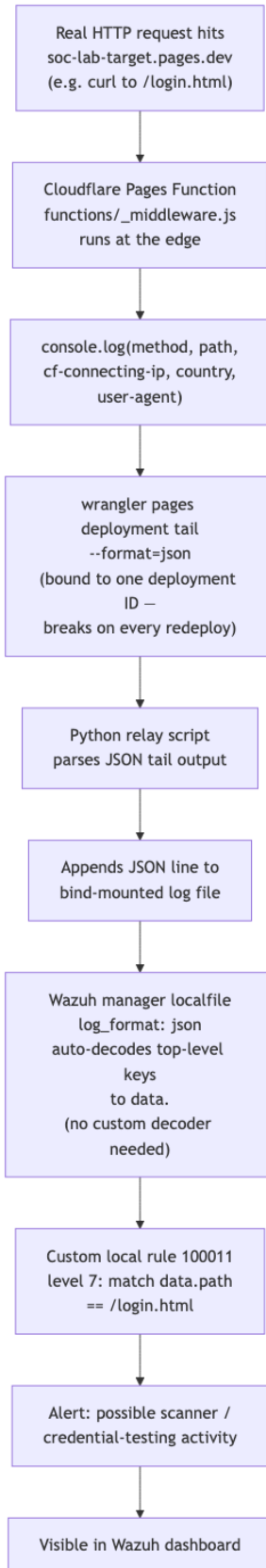


Figure 7: Diagram
32

When to use this / when NOT to use this

Use Cloudflare Pages Functions when you want edge-level request visibility or edge-level logic (auth checks, redirects, header injection, and here, logging) without standing up and paying for an origin server. Do NOT treat Free-tier wrangler tail as a substitute for real persistent log retention in anything beyond a lab — the deployment-ID binding alone makes it unsuitable for continuous production monitoring without real automation wrapped around it, and Logpush (paid) or shipping logs via a Worker to an external store on every request are the actual production answers. When deciding whether a custom Wazuh decoder is needed for a new JSON-emitting source, check first whether it's already well-formed JSON (often no decoder needed, per this section) versus a bespoke text format (usually needs one) — but expect to write a custom rule almost every time the source is your own bespoke application rather than well-known third-party software, since no SIEM vendor ships content for logic nobody but you wrote.

Part 3.7-3.9: QEMU/QuartzVM/Windows VM, Terraform + Azure, Sigma + KQL (Detection-as-Code)

This part covers three pieces of the lab that are, on the surface, about completely different technical domains — hardware virtualization, cloud infrastructure-as-code, and portable detection rule authoring — but share one theme worth naming up front: each of them is built to a real, honest, partial state, and each is a useful lesson in what “done” actually means at different layers. The Windows VM installs and boots but sits in tension with its own tooling's design philosophy. The Terraform code is syntax- clean but has never touched real cloud infrastructure. The Sigma rules are real and portable in theory but were only ever actually converted for one of the two backends this lab could target. None of that is failure — it's exactly the kind of honest state a real, evolving lab is normally in, and it's worth teaching as such rather than smoothing over.

Part 3.7: QEMU, QuartzVM, and the Windows Server VM

Hypervisor acceleration vs software emulation

QEMU [FREE] (open source, installed via Homebrew on the lab host) is a machine emulator and virtualizer. The distinction that actually determines whether a VM is usable or painfully slow is whether QEMU can use hardware-assisted virtualization or has to fall back to pure software emulation:

- Hardware acceleration — on Linux, KVM; on Windows, WHPX (Windows Hypervisor Platform); on macOS, Apple's HVF (Hypervisor.framework). These let the guest's CPU instructions execute close to natively on the host CPU, with the hypervisor only intervening for privileged operations (I/O, interrupts, memory management). This is what makes a VM feel close to bare-metal speed.
- TCG (Tiny Code Generator) — QEMU's built-in software emulation path, used when hardware acceleration for the guest's instruction set isn't available on the host. TCG translates guest instructions into host instructions in software, instruction by instruction, with none of the hardware-level shortcuts a real hypervisor gets. This is dramatically slower — often an order of magnitude or more — than hardware-accelerated execution.

The lab host is Apple Silicon (ARM64). HVF acceleration on Apple Silicon works for ARM64 guests — but the Windows Server 2022 Evaluation image is x86_64. There is no hardware acceleration path for running x86_64 guest instructions on an ARM64 host; Apple Silicon's CPU cannot natively execute x86_64 machine code, hardware-assisted or otherwise. So this VM necessarily runs under TCG software emulation — genuinely slow, which is exactly why the install took roughly an hour or more rather than the few minutes a hardware-accelerated install would take. This is worth stating plainly rather than glossing over: an Apple Silicon Mac running an x86_64 Windows guest via QEMU is doing full software translation of every instruction, not “a slower VM” in the way a resource-constrained hardware-accelerated VM is slower — it's an entirely different, much heavier execution model.

The disposable-lab philosophy vs what actually got built

QuartzVM — a real, open-source (MIT-licensed) Tauri/Rust desktop app the user built (github.com/danishrafiquekhan/quartzvm, installable via `quartzvm-releases`) — exists as a GUI layer over QEMU, explicitly designed around a “create → snapshot → destroy” disposable-lab philosophy: spin a VM up for a specific purpose, snapshot known-good states you want to return to, and destroy the VM entirely when you’re done, rather than maintaining long-lived VMs that accumulate drift and risk.

Worth stating plainly, because it’s a real and currently unresolved tension in this lab rather than a settled design decision: the actual Windows Server VM built in this session was not driven through QuartzVM’s GUI at all — it was built directly via raw `qemu-system-x86_64` and `qemu-img` commands. And the VM itself, as configured, looks much more like a persistent asset than a disposable one: it has a hardcoded administrator password baked into the unattended install answer file, and 10-boot autologon configured (the guest will automatically log in as the administrator for the first 10 boots without a password prompt). Neither of those is consistent with “disposable” — a hardcoded password and extended autologon are conveniences for a VM you expect to keep using and coming back to, not conveniences for something you intend to snapshot-and-destroy routinely. This is flagged here as an open, undecided question in the lab, not a mistake quietly fixed: does this VM get folded into QuartzVM’s disposable model going forward (rotate the password, drop autologon, start snapshotting it), or does it stay a longer-lived asset because Atomic Red Team validation work benefits from a stable, already-configured target? Both are legitimate answers; neither has been chosen yet.

Unattended installation via `autounattend.xml`

Rather than clicking through Windows Setup interactively, the VM was installed fully unattended using an `autounattend.xml` answer file. Understanding why this works requires understanding Windows Setup’s configuration passes — conceptually, a small number of distinct phases Windows Setup runs through, each of which can read different sections of the answer file:

- `windowsPE` — the earliest pass, running inside the WinPE (preinstallation environment) that boots from the installation media itself, before any OS is actually installed onto disk. This pass handles things like disk partitioning, language/locale selection for setup itself, and product key entry. Critically for this lab, this is also the pass during which Windows Setup scans all attached media — including removable media like a second CD-ROM/ISO — looking for a file literally named **`autounattend.xml`** at the root. If found, Setup automatically uses it to drive the rest of the install with zero interactive prompts. This is exactly why the answer file was built into its own small ISO (via macOS’s `hdiutil makehybrid`) and attached as a second CD-ROM device alongside the Windows installation media itself — Setup finds it automatically during this earliest pass without any manual pointing required.
- `specialize` — runs later, after the OS image has been laid down onto disk and the machine has rebooted into itself for the first time in a semi-configured state. This pass is where machine-specific and component-specific configuration happens — things like computer name, network configuration, and enabling/configuring individual Windows components (firewall behavior, specific services) via their component identifiers in the answer file.
- `oobeSystem` — the final pass, corresponding to the interactive “Out-Of-Box Experience” a normal user would see (account creation, region/keyboard confirmation, etc.) — except here, driven by the answer file instead of a human, including things like the local administrator account password and `FirstLogonCommands` (arbitrary commands to run automatically the first time a user logs on).

The real bug: a specialize-pass component name mismatch

A real, concrete failure was hit and fixed during this build. The original `autounattend.xml` included a component in the `specialize` pass intended to enable the Windows Firewall and configure it (with the eventual goal of allowing RDP access) via a `Networking-MPSSVC-Svc` component definition, using specific property names for that component’s settings. Windows Setup rejected the answer file at the `specialize` pass with an error to the effect of:

Windows could not parse or process the unattend answer file for pass [specialize]. The settings specified in the answer file cannot be applied.

This is a notoriously unhelpful class of Windows Setup error — it identifies which pass failed but not which property or line is actually wrong, because the underlying cause is usually a component property name or namespace that doesn't match what that specific Windows build's unattend schema actually expects (these schemas are versioned per Windows build and are easy to get subtly wrong from memory or from an example written for a different Windows version). Diagnosing this by trying to get the exact property names right for Networking-MPSSVC-Svc purely from memory/documentation turned out to be more fragile than it was worth. The actual fix was to remove that component from the specialize pass entirely and instead achieve the same end goal (firewall enabled, RDP allowed) with plain, well-documented netsh advfirewall commands placed in FirstLogonCommands during the oobeSystem pass instead — running ordinary command-line firewall configuration after the OS is already up, rather than fighting an XML schema for a component whose exact expected shape wasn't reliably known. This is a good general lesson: when a declarative configuration mechanism's exact schema is uncertain and the failure mode is opaque, falling back to an imperative command run after-the-fact can be the more reliable engineering choice, even though it's less "pure" than doing everything through the declarative answer file.

Why BIOS/IDE instead of UEFI/virtio

The VM was configured with QEMU's pc machine type (i440fx chipset, BIOS boot — not q35, which is the more modern UEFI-capable chipset), and its disk and CD-ROM devices are attached as IDE, not the higher-performance virtio paravirtualized device family QEMU also supports. This was a deliberate compatibility choice, not an oversight: virtio devices require guest-side drivers that Windows does not ship in-box — using virtio disk or network devices would have meant injecting third-party virtio driver packages (commonly the Fedora/Red Hat virtio-win driver ISO) into the Windows Setup process, either by slipstreaming drivers into the unattended install or manually loading them during setup's disk-selection screen. BIOS/IDE, by contrast, uses device types Windows has supported natively and driver-lessly for decades — the tradeoff is pure performance (IDE emulation is slower than virtio, particularly for disk I/O) traded directly for zero additional driver dependency, which matters more in a first-build context where the goal is "get a working, unattended, reproducible install" rather than "get the fastest possible VM."

When to use this / when NOT to use this

Use QEMU with TCG-only emulation when you specifically need a cross-architecture guest (x86_64 on ARM64 or vice versa) and can tolerate slow installs/boots in exchange for that flexibility — it is the only way to run this Windows Server image at all on this Apple Silicon host. Do NOT use TCG for anything where guest performance matters at runtime (this matters here mainly during install; once installed and idle, or running light agent workloads like Sysmon/a Wazuh agent, the VM is usable, just not fast). Use BIOS/IDE over UEFI/virtio specifically when driver compatibility and setup reliability matter more than raw performance — revisit that choice (move to virtio with injected drivers) if this VM's workload later becomes I/O-heavy enough that IDE emulation becomes the bottleneck. And treat the disposable-vs-persistent tension honestly rather than pretending it's resolved: this VM, as currently configured, needs a deliberate decision one way or the other before it's reused heavily, particularly given the hardcoded password.

Part 3.8: Terraform + Azure

State: the core concept, and why remote state matters

Terraform [FREE] (open source, HashiCorp) is declarative infrastructure-as-code: you describe the infrastructure you want in .tf files, and Terraform computes and applies the changes needed to make real infrastructure match that description. The mechanism that makes this possible is state — a JSON file (terraform.tfstate) that records what Terraform believes currently exists (resource IDs, attributes, dependency relationships) so it can diff "what's declared" against "what's real" on every run rather than blindly re-creating everything every time.

State can live locally (a `.tfstate` file on the machine running Terraform) or remotely (a shared backend — in the Azure ecosystem, typically an Azure Storage Account with a blob container configured as the backend). Remote state matters for two concrete reasons, not just “best practice” as an abstract label:

- Locking — a remote backend (when it supports locking, as Azure Storage does via blob leases) prevents two people or two automated runs from applying changes to the same infrastructure concurrently, which would otherwise corrupt state or produce conflicting real-world changes. Local state has no locking at all — nothing stops two terminal sessions on the same machine from racing each other.
- Shared source of truth — if state lives only on one person’s laptop, nobody else (and no CI/CD pipeline) can safely run Terraform against that infrastructure without first obtaining that exact state file, which doesn’t scale past a single operator and is fragile even then (lost laptop, disk failure — state gone, and with it Terraform’s entire understanding of what it manages).

The bootstrap chicken-and-egg problem

A specific, well-known wrinkle that `terraform-labs`’ own structure acknowledges (its numbered exercises include one specifically for “remote state Azure backend,” folder 02): to use a remote Azure Storage Account as your Terraform backend, that storage account itself has to already exist — and Terraform normally wants to be the tool that creates Azure resources. This is a genuine chicken-and-egg problem: you can’t declare the remote-state storage account as a Terraform-managed resource using that same backend, because the backend has to exist before Terraform can even initialize. The standard resolution is to create that one bootstrap resource (the storage account + container) manually or via a one-off local- state Terraform run, and only then configure subsequent Terraform configurations to point at it as their backend. This bootstrap step is inherently a bit manual/imperative even in an otherwise fully declarative workflow, and that’s normal, not a flaw in the approach.

Module composition

Terraform modules let a configuration be composed from reusable, parameterized units (a “networking module,” a “compute module”) rather than one flat sprawling file — `terraform-labs`’ folder 03 is specifically a networking+compute modules exercise. The principle is the same as function/library composition in software: define an interface (input variables, output values), implement the resource logic once, and reuse it across environments or exercises with different inputs, rather than copy-pasting resource blocks.

Terraform Cloud vs local backend

Terraform Cloud [FREEMIUM] (HashiCorp’s hosted service; a free tier exists for small teams, with paid tiers for governance/policy features at scale) provides remote state storage and remote plan/apply execution, plus features like a run history UI, policy-as-code gates, and team access controls — folder 04 in `terraform-labs` specifically explores Terraform Cloud integration. The tradeoff versus a plain Azure Storage Account backend: Terraform Cloud gives you a managed, richer experience (no need to build your own CI runner for `terraform plan/apply`, built-in state locking and history, a web UI) at the cost of depending on an external hosted service and, past the free tier, real cost. A plain remote backend (Azure Storage) gives you just state storage and locking — the minimum needed for team-safe state — with `plan/apply` still run from wherever you invoke the Terraform CLI (a laptop, a self-managed CI runner), which is more work to wire up but keeps everything inside infrastructure you already control.

“Syntax valid” is not “actually deploys correctly” — and none of this has been applied

This needs to be stated with complete honesty because it’s real and important: all seven **`terraform-labs`** exercise folders pass **`terraform fmt`** and **`terraform validate`** cleanly. None of them have ever been **`terraform apply`**’d against a real Azure subscription, despite a real free-tier Azure subscription existing (with MFA required, a spending cap set, and `az login-only` auth per the repo’s own README) for the entire duration of this project.

This is worth unpacking as two genuinely different levels of confidence, because conflating them is a common and consequential mistake:

- **terraform validate / fmt** confirms the configuration is syntactically well-formed HCL, that resource argument names and types match what the provider schema expects, and that references between resources resolve. This is a static, local, offline check. It catches typos, wrong argument names, and type mismatches. It does not contact Azure at all, and therefore cannot catch: insufficient IAM permissions on the executing identity, resource naming collisions in the target subscription, quota limits, region availability gaps for a specific SKU, misconfigured dependencies that are only wrong at apply time (a resource that references another resource's actual runtime-computed ID rather than a static value), or plain logical errors that are valid HCL but produce infrastructure that doesn't actually do what was intended.
- A successful **terraform apply** is the only thing that actually proves the configuration works against the real target platform — real permissions, real quotas, real naming rules, real inter-resource dependency ordering, all exercised for real.

terraform-labs sits entirely at the first level of confidence. That is explicitly, honestly documented in the repo itself as “untested-in-anger,” and it should be understood as exactly that here too — this is real, carefully-written infrastructure-as-code that has demonstrated it is well-formed, not that it works. A hiring manager or reviewer reading this portfolio should take that distinction at face value rather than assuming “clean Terraform repo” implies “deployed and proven”; the two are different claims, and this lab only makes the first one.

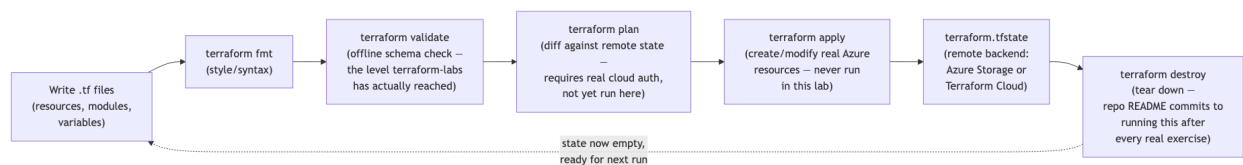


Figure 8: Diagram

When to use this / when NOT to use this

Use Terraform (over clicking through the Azure Portal) whenever infrastructure needs to be reproducible, reviewable (via pull requests against .tf diffs), or destroyed and recreated repeatedly — exactly the lab-exercise pattern terraform-labs is built around, where terraform destroy after every exercise is a stated intention. Use remote state (Azure Storage, or Terraform Cloud) the moment more than one person or more than one machine might ever run Terraform against the same infrastructure — local state does not scale past a single operator on a single machine. Do NOT present terraform validate-clean code as equivalent to proven, working infrastructure — say explicitly, as this lab does, that it has not been applied, and treat that as a concrete next step (with the bootstrap backend problem solved first) rather than a detail to omit.

Part 3.9: Sigma + KQL — Detection-as-Code

What Sigma is and why it exists

Sigma [FREE] (open source, community-maintained rule format and converter toolchain) is a generic, YAML-based signature format for SIEM detection rules — the stated goal of the project is “write a detection rule once, in a platform-agnostic way, then convert it to whatever your actual SIEM’s native query language is” (via sigma-cli/pySigma backends targeting specific platforms — Splunk SPL, Elastic Query DSL, Microsoft Sentinel KQL, and others). The value proposition, in theory, is that detection logic becomes portable and shareable across organizations and platforms the way source code is portable across machines with the same language runtime: a well-written Sigma rule expressing “flag sign-ins from impossible travel distances

within an implausible time window” should be convertible to whichever backend a given SOC actually runs, without each SOC having to invent that logic from scratch in their own platform’s syntax.

The real platform-translation problem this lab actually hit

This lab is a genuinely instructive, honest case of where that “convert once, run anywhere” promise runs into a real limit. The detection- engineering repo’s Sigma rules and their KQL conversions were written against Microsoft Sentinel’s specific table schema — SigninLogs, AuditLogs, the actual column names and semantics Entra ID sign-in and audit data land in inside a Sentinel workspace. That’s a deliberate, sensible choice: Sentinel is built for exactly this kind of cloud identity telemetry, with purpose-built tables, built-in identity connectors, and a KQL query surface designed around that data shape.

But the SIEM actually running live in this lab is Wazuh, not Sentinel — chosen for the live lab specifically because it’s free and self-hosted, while Sentinel is consumption-priced and no real Sentinel tenant exists for this project. Wazuh’s native rule format is not KQL at all — it’s an XML-based decoder/rule system (exactly what Parts 3.5 and 3.6 above walk through in concrete detail: <decoder> and <rule> blocks matching regex/field patterns against normalized log fields) built around log-based telemetry from hosts, applications, and network sources, not around cloud identity provider API schemas like Entra ID’s sign-in and audit logs.

The honest engineering conclusion reached in this lab — stated plainly rather than smoothed over — is that the identity-focused Sigma/KQL detection content in the **detection-engineering** repo was deliberately NOT converted into Wazuh’s native rule format. Forcing that conversion would mean writing Wazuh rules pretending to match against Entra ID sign-in/audit-log fields that Wazuh has no actual ingestion pipeline for in this lab — there is no Entra ID log source feeding Wazuh at all here. A Wazuh rule with no real matching data behind it is not a working detection; it’s a rule that looks plausible in a rules file and would fire on nothing, ever, in this environment. That would be fake coverage — content that exists to look complete rather than to actually detect anything — and this lab’s standard, demonstrated consistently elsewhere (the honest “stopped/idle” labels on Suricata/TheHive/LocalStack, the “never applied” label on Terraform), is to say plainly what’s real and what isn’t rather than paper over the gap. So: the Sigma/KQL content stays written for Sentinel’s schema, because that’s the platform it’s actually correct for, and it is documented as Sentinel-target content, not Wazuh-deployed content — a real, live SOC would either stand up a genuine Sentinel workspace with Entra ID log ingestion to run this content for real, or accept that running it meaningfully on Wazuh would require first building an entirely separate ingestion pipeline (an Entra ID audit log export mechanism feeding Wazuh, analogous in spirit to the MySQL/Cloudflare relays already built) before a native Wazuh rule would even have data to match against — and that pipeline does not exist in this lab.

attack-mapping.csv and MITRE ATT&CK tagging as a discipline

Independent of which backend a given rule targets, detection-engineering maintains an attack-mapping.csv tying each Sigma rule to specific MITRE ATT&CK technique IDs (e.g. T1078.004 for cloud account abuse, T1110 for brute force, and similar). This is a discipline worth calling out on its own: ATT&CK mapping isn’t decoration on top of a detection rule, it’s what lets a rule’s coverage be reasoned about systematically — which techniques does this ruleset actually cover, where are the gaps, which techniques does the organization have zero detection for at all. Maintaining that mapping as a structured, separate artifact (a CSV, queryable and diffable, rather than prose scattered across rule comments) is what makes coverage analysis and gap-finding tractable at any scale beyond a handful of rules — the same reason ATT&CK Navigator-style heatmaps are a standard SOC reporting artifact in real security programs.

When to use this / when NOT to use this

Use Sigma when detection logic genuinely needs to be portable across multiple SIEM backends, or when publishing/sharing detection content for others whose SIEM you don’t control — the platform-agnostic authoring step is real value even when only one backend ends up actually deployed, since it forces the underlying detection logic to be expressed clearly enough to convert at all. Do NOT treat “Sigma rule exists” as equivalent to “this detection is live and firing somewhere” — as this lab demonstrates directly, a Sigma

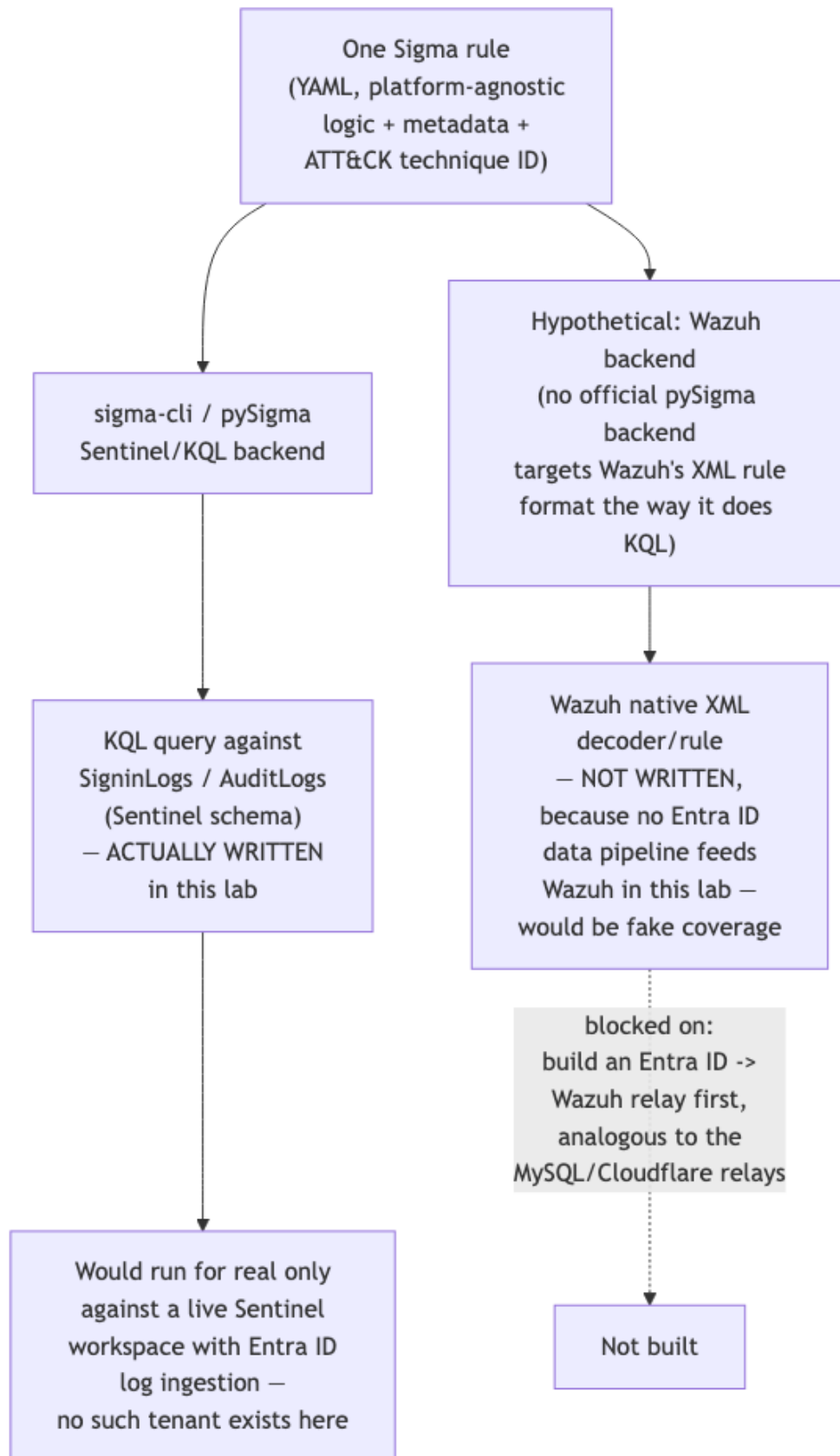


Figure 9: Diagram
39

rule converted to one backend's query language is only as real as that backend actually running with the right data feeding it, and content correctly scoped for one platform's schema (Sentinel's identity tables here) should not be force-converted to a structurally different platform (Wazuh's log-based rule engine) just to claim broader coverage — that produces rules that look like detections but detect nothing. Maintain an explicit ATT&CK mapping artifact (like `attack-mapping.csv`) as a first-class part of any detection content repository, not an afterthought, since it's what turns a pile of individually-reasonable rules into an analyzable coverage picture.

Part 4: Identity Security Deep Dive

Identity is the thread that runs through almost every other part of this lab. The detection rules in `detection-engineering` are mostly sign-in and audit-log rules. The IAM scripts in `intune-endpoint-health-platform` are Microsoft Graph API calls against directory objects. The `aws-identity-detection` repo is entirely about one identity misusing its own permissions. Even the phishing cases in `bec-phishing` are, underneath the email delivery mechanics, an attack on trust in an identity. This part exists to explain identity security properly — not as a glossary of terms, but as the reasoning a working analyst or engineer actually needs, grounded in what this lab's real content does and doesn't cover.

4.1 Core identity concepts: what an “identity” actually is

Before anything else, it's worth being precise about what “identity” means in a security context, because the word gets used loosely and that looseness causes real confusion later — Section 4.3 exists almost entirely because of it.

An identity is a principal that can be authenticated and then authorized to act. That's the whole definition. It sounds trivial, but the two halves matter and get conflated constantly:

- Authentication (AuthN) answers “who (or what) are you, and can you prove it?” It's the login event — a password check, an MFA challenge, a client certificate, a signed JWT. Authentication produces a token or session, not permission.
- Authorization (AuthZ) answers “now that I know who you are, what are you allowed to do?” It's evaluated separately, every time, against roles, policies, or scopes. A successfully authenticated identity with zero role assignments can do nothing — authentication succeeding says nothing about what happens next.

This separation is the single most important mental model in identity security, because almost every real-world identity incident is a failure in exactly one of these two stages, and the fix is different depending on which one failed. A password-spray attack (see the `password-spray.yml` Sigma rule discussed in Part 5) is an authentication-layer attack — the attacker is trying to become an identity. A privilege escalation, like the one the `aws-identity-detection` repo detects (an IAM principal creating a new user and immediately attaching `AdministratorAccess` to it), is an authorization-layer attack — the attacker already is an identity, and is manipulating what that identity (or a new one it creates) is allowed to do.

Identity object types

Not every identity is a human. A directory (Entra ID, AWS IAM, or any other identity store) holds several distinct kinds of principal, and detection/response logic has to treat them differently:

- User — a human, or a representation of one. Has a password or federated credential, can complete interactive MFA, has a manager, a job title, sign-in activity. This is the identity type `privileged_role_access_review.py` (in `intune-endpoint-health-platform`) is built around — it flags human directory-role members whose `signInActivity.lastSignInDateTime` is more than 30 days stale, because a human who's stopped signing in but still holds a privileged role is a live risk sitting unused.

- Group — not itself an identity that authenticates, but a container that grants authorization by membership. Role assignments are very commonly made to groups rather than individual users specifically so that adding/removing a user from a group is the access-control action, not editing role assignments directly. This matters for the JML lifecycle in 4.2 — group membership is usually the actual lever that access reviews and provisioning workflows pull.
- Service principal (Entra ID) / IAM role or user (AWS) — a non-human identity representing an application, script, or automated workflow. This is the type the Graph API scripts in this lab authenticate as themselves being run by (via device code flow, from a human's session) but is also the type an app registration creates when it needs to call an API unattended.
- Managed identity (Azure-specific) — a service principal that Azure creates and rotates the credential for automatically, tied to the lifetime of an Azure resource (a VM, a Function App, a Logic App). The entire point of a managed identity is that there is no secret for a human to manage, leak, or forget to rotate — Azure handles the credential material internally. The `sentinel-soar-playbooks` designs (`disable-a-compromised-user`, `isolate-a-device`) are written assuming the underlying Logic App would authenticate to Graph API and Defender for Endpoint via a managed identity rather than a stored client secret, precisely to avoid adding a new secret-management burden on top of an incident-response automation.
- Application / App Registration object (Entra ID) — the definition of an app (its permissions, redirect URIs, certificates), distinct from the service principal that represents an instance of it actually operating inside a specific tenant. This distinction is not academic — it is the exact distinction the GUID triage methodology in 4.3 depends on, because “App Registration exists but no corresponding Service Principal in this tenant” and “Service Principal exists” are two different, diagnostic states.

Entra ID / Azure AD vs AWS IAM: two different vocabularies for the same problem

This lab has real content in both worlds — `detection-engineering` and `intune-endpoint-health-platform` are Entra ID-native, `aws-identity-detection` is AWS IAM-native — so it's worth translating between them explicitly rather than assuming one vocabulary transfers cleanly to the other. They solve the same problem (who can do what) with different primitives:

Concept	Entra ID / Azure AD	AWS IAM
Human identity	User object	IAM User (increasingly discouraged in favor of federated roles)
Non-human identity	Service Principal / Managed Identity	IAM Role (assumed) or IAM User with access keys
“What can this identity do”	Role assignment (built-in or custom role) scoped to a resource	IAM Policy (JSON document) attached to a user/group/role
Grouping identities	Security Group / Microsoft 365 Group	IAM Group (users only — cannot contain roles)
Cross-account/cross-tenant access	Guest user (B2B), cross-tenant access settings	Trust policy on an IAM Role (a JSON document naming which principals may <code>sts:AssumeRole</code> into it)
Where authorization logic lives	Role assignment + Conditional Access policy, evaluated at sign-in and per-request	IAM Policy evaluated per-API-call, no separate “session risk” layer built in the same way
Central authorization query surface	Microsoft Graph API	AWS IAM API / CloudTrail (for what happened)

The AWS side has one architectural idea that Entra ID doesn't have a direct equivalent for: the trust

policy. An AWS IAM Role isn't just "a bundle of permissions" — it also carries a document specifying who is allowed to assume it in the first place (which accounts, which federated identity providers, under what conditions). This is why the privilege-escalation pattern documented in `aws-identity-detection/detections/privesc-detection.md` is dangerous: `iam:CreateUser` followed immediately by `iam:AttachUserPolicy` with `AdministratorAccess` doesn't need a trust policy at all, because the attacker is creating a brand-new identity and directly attaching permissions to it — no assumption step, no trust boundary to cross. That's exactly why this is one of the best-documented cloud privilege-escalation primitives (catalogued by tools like Rhino Security Labs' `pmapper`): it skips the one control (trust policy review) that AWS's role-assumption model would otherwise force an attacker through.

Entra ID's rough analog to a trust policy is Conditional Access plus cross-tenant access settings (covered in 4.4) — but it's evaluated at a different layer (sign-in time, continuously, with real-time signals like device compliance and location) rather than being a static document attached to the identity itself. Neither model is "better" in the abstract; they reflect the different shapes of the two clouds' control planes.

4.2 The Joiner-Mover-Leaver (JML) lifecycle

Joiner-Mover-Leaver is the standard identity governance framework for the three moments in an identity's life where access should change:

- Joiner — a new identity is provisioned. Access should be granted based on role, not copied from a convenient existing account ("just give them what Sarah has").
- Mover — an existing identity changes role, team, or department. Access should be re-evaluated — old access removed, new access granted — not simply additive. This is the stage most JML programs handle worst in practice, because "add access when someone moves" is an obvious ticket to file, and "remove the access they no longer need" is not, so it quietly doesn't happen.
- Leaver — an identity's relationship with the organization ends. Access should be revoked, ideally immediately and completely, not left to expire on its own or get caught later in an audit.

JML matters as a foundational concept because it's the framework that turns "identity security" from a point-in-time login problem into a lifecycle problem. Almost every stale-access finding an auditor produces — orphaned accounts, over-privileged movers, leavers who still have a valid session token three weeks after their last day — is a JML process gap, not a technology failure. The technology (Entra ID, IAM) will happily do exactly what it's told; JML is about whether what it's told is still correct.

A real failure mode from this lab: unverified contractor provisioning

The `intune-endpoint-health-platform` work includes a documented real process-gap pattern from the Joiner stage specifically: contractor accounts were being provisioned without an enforced identity-verification step first. In other words, the provisioning process had a step that was supposed to happen (verify who this contractor actually is before an account exists for them) but nothing in the technical process actually required it — it depended on a human doing it correctly, every time, without the system checking. This is a textbook Joiner-stage gap: the failure isn't in a login flow or a permission boundary, it's upstream of both, in the moment an identity is created in the first place.

This got caught, escalated, and led to an inventory cleanup — which is the useful part to actually learn from, because the shape of the fix matters more than the specific incident.

Detective vs preventive controls, and why this distinction is the whole lesson

The way this gap was caught was an audit script that flagged accounts missing verification attributes — after the fact, on a schedule, looking at what already existed. That is a detective control: it finds problems that have already happened. It's valuable — genuinely, an audit script that surfaces "these N contractor accounts have no recorded identity-verification step" is far better than not knowing — but it is not the same thing as fixing the problem, and it's important to be precise about why.

A detective control tells you a gate was skipped after someone has already walked through the gap. Every account it finds already existed, with real access, for however long it took the audit to run and someone to review the output. If that access happened to be sensitive, the exposure window is real and already happened by the time detection fires. This is why a detective-only response to a Joiner-stage gap is properly called a compensating control, not a fix: it compensates for the missing preventive control by catching its output, but it does nothing to stop the next instance of the same gap from occurring, and it depends on someone actually reviewing and acting on the audit output every time it runs.

A preventive control stops the gap from being creatable in the first place — in this case, that would mean the provisioning workflow itself refusing to create an enabled account (or refusing to activate/enable an already-created one) until the verification attribute is actually populated. Preventively, “contractor account exists” and “contractor identity has been verified” become the same fact, enforced at creation time, not two facts that a script has to reconcile later.

The reason this distinction is worth teaching carefully, not just naming: a detective control is usually faster and cheaper to build than a preventive one — an audit script that runs a Graph query and flags missing attributes (structurally similar to `orphaned_account_detection.py` in this same repo, which flags enabled accounts with no manager assigned by walking `/users` and checking `/users/{id}/manager` for a 404) can be written in an afternoon. Rebuilding a provisioning workflow so it structurally cannot create an unverified account is a real engineering and process change, often touching whatever system originates the Joiner event (HR system, ticketing system, or a manual request form) — and that’s exactly why organizations under time pressure land on the detective control and stop there, and why “we have a script that catches this” quietly becomes the permanent answer instead of the interim one. Recognizing that pattern — and pushing past it — is a real, teachable piece of identity governance judgment, not a theoretical nuance.

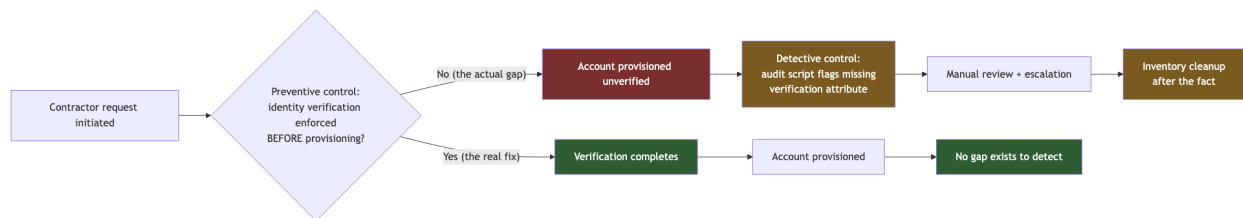


Figure 10: Diagram

JML and Mover/Leaver in this lab’s other content

The Leaver stage is the direct motivation for `orphaned_account_detection.py`: an enabled account with no manager assigned is exactly the kind of account a Leaver process should have caught and disabled, but didn’t — nobody owns it, which in practice means nobody notices it going stale, being targeted, or quietly retaining access it no longer needs. The script’s own output deliberately caveats this (“no manager isn’t proof of orphaning on its own — service accounts and some external guests legitimately have none”) which is itself good JML practice: a detective control’s output is a worklist for a human to triage, not an automatic verdict.

The Mover stage doesn’t have dedicated tooling in this lab yet, but it’s the same underlying pattern as `privileged_role_access_review.py`: a Mover who changed teams six months ago and never had their old privileged role assignment removed looks, from Graph’s perspective, identical to a Joiner who was over-provisioned from day one — a directory role member whose actual current job doesn’t justify the access. The access-review script catches both by asking the same question (stale sign-in activity on a privileged role member) regardless of which JML stage produced the mismatch.

4.3 GUID/Object triage methodology

This is one of the most concrete, real pieces of investigative methodology in this whole lab, and it's worth explaining why the problem exists architecturally before walking through the runbook itself — the “why” is what makes the methodology make sense instead of feeling like an arbitrary checklist.

Why this ambiguity exists

In Azure and Entra ID, GUIDs (128-bit UUIDs, formatted like 3b8e4a2f-6c1d-4a9e-8b2a-1d2e3f4a5b6c) are the universal identifier format — and they are used for both identity objects and non-identity resource references, with no type information encoded in the GUID itself. A GUID that identifies a user object, a GUID that identifies an App Registration, a GUID that identifies an Azure DevOps Collection, and a GUID that identifies a service connection inside a DevOps project are all, structurally, the exact same kind of value — 32 hex characters in the same five-group pattern. There is nothing about the string 3b8e4a2f-6c1d-4a9e-8b2a-1d2e3f4a5b6c itself that tells you which of those it is. The only way to know is to ask the system that's supposed to know it — and different systems own different GUID namespaces, so you may have to ask more than one.

This is why a detection that fires on “unrecognized GUID accessing a resource” is, on its own, an incomplete alert — it correctly flags that a GUID has been seen that isn't in your known-identity inventory, but “not in your known-identity inventory” has at least two very different explanations: (a) this really is an unrecognized identity — a real security concern — or (b) this is a legitimate non-identity resource reference (a DevOps Collection ID, a service connection ID, an application's internal resource GUID) that was never going to appear in an identity inventory in the first place, because it was never an identity. Triage exists to tell those two apart, and doing it wrong in either direction has a real cost: false-negative (waving off a real unrecognized identity as “probably just a resource ID”) is a missed intrusion; false-positive (escalating a routine DevOps service connection as an unknown identity) is wasted analyst time and, at scale, alert fatigue that erodes trust in the detection.

The real investigative methodology

The guid-triage runbook in detection-engineering documents the actual sequence an analyst works through to resolve this ambiguity, roughly in order of “cheapest and most likely to resolve it” to “authoritative”:

1. Check Enterprise Applications (Entra ID admin center, or GET `/servicePrincipals/{id}` via Graph). Enterprise Applications is the tenant's list of service principals — instances of applications (first-party, third-party, or your own) that have actually been consented to and can operate in this tenant. If the GUID resolves here, it's a service principal — a real, non-human identity, and worth understanding what permissions it holds and who consented to it.
2. Check App Registrations (GET `/applications/{id}` via Graph). This is the definition layer, distinct from Enterprise Applications/service principals as discussed in 4.1 — an app can be registered (defined) in a tenant without necessarily having an active service principal doing something right now, or the GUID in the alert might be the Application (client) ID rather than the Object ID, which is a common source of confusion since both are GUIDs and both are shown in different places in the Entra portal for the same app.
3. Check Managed Identities (filtered view within Enterprise Applications, or GET `/servicePrincipals?$filter=servicePrincipalType eq 'ManagedIdentity'`). If the GUID belongs to a managed identity, it should trace back to a specific Azure resource (a VM, Function App, Logic App) — and if it does, the next question is simply whether that resource's automation is supposed to be touching the thing it touched, which is a much more mundane investigation than “unknown identity.”
4. Query Microsoft Graph API directly for the object, e.g. GET `https://graph.microsoft.com/v1.0/directoryObjects/{id}`. This is the step that actually resolves the ambiguity authoritatively, because the response code itself is the signal, not just the payload:
 - 200 OK with a directory object body → this GUID identifies a real object in the directory (user, group, service principal, application) that Graph knows about. Now the question shifts to “why

- did this specific object legitimately show up here, and is that expected.”
- 404 Not Found → Graph, which is the authoritative source for every identity object in the tenant, has no record of this GUID as a directory object at all. This is the concrete, decisive evidence that the GUID is very likely a non-identity resource reference — an Azure DevOps Collection ID, a service connection ID, an internal resource identifier from some other Azure or third-party service that happens to use GUIDs for its own object naming, none of which live in the Entra ID directory Graph queries against.

A 404 here isn’t “inconclusive” — it’s the actual finding. It means: whatever this GUID is, it is not an identity, because if it were, the tenant’s own identity directory would know about it. That reframes the original alert from “unrecognized identity accessing a resource” to “a non-identity resource ID got picked up by log parsing/correlation logic that assumed every GUID in that log field was an identity” — which is a detection-tuning problem (see the falsepositives discussion in Part 5.4), not a security incident.

Why this matters beyond one runbook

The deeper lesson is that identity detection logic is only as good as its assumption that a GUID in a given log field is actually an identity — and that assumption is often built into a query or correlation rule without anyone explicitly stating it, because in the common case it’s true. Azure’s architecture doesn’t give you a free type check on a GUID; you have to build one, either into the detection itself (pre-filtering known non-identity GUID namespaces, where that’s feasible) or into the triage process that handles what the detection produces. This runbook is the second approach, done properly: cheap checks first, an authoritative check (Graph’s 200/404) as the tiebreaker, and the result of that tiebreaker treated as real evidence, not a shrug.

4.4 Conditional Access, MFA, and PIM

These three controls are often talked about together because they all sit at the boundary between “an identity is authenticated” and “an identity gets to use a privilege,” but they answer different questions.

Multi-Factor Authentication (MFA) answers “is this really the person the password claims it is?” by requiring a second, independent proof of identity (a push approval, a TOTP code, a hardware key) in addition to something known (the password). MFA is the single highest-leverage control against credential-based attacks generally, including the password-spray pattern this lab’s `password-spray.yml` Sigma rule detects — a spray attack that successfully guesses a password still fails at the MFA challenge, so the practical damage of the underlying weak/reused password is contained by a control at a completely different layer. This is why MFA fatigue attacks (an attacker who already has valid credentials spamming MFA push approvals hoping for an accidental tap) exist as a distinct attack pattern worth its own playbook — it’s specifically targeting the one control standing between a stolen credential and account takeover.

Conditional Access (Entra ID) is policy evaluated at sign-in time (and, with continuous access evaluation, in near-real-time during a session) that decides whether to grant, block, or add friction to an access attempt based on signals: which user, which app, from what location/IP, from what device (and is that device compliant/managed), what the calculated sign-in risk level is. It’s the practical mechanism for “just because you authenticated correctly doesn’t mean this specific access attempt, in this specific context, should be allowed” — e.g. requiring MFA specifically when sign-in risk is elevated rather than every time, or blocking sign-in entirely from locations the organization has no legitimate presence in. The `sentinel-soar-playbooks` designs assume Conditional Access policies exist as the thing that got tampered with in the `conditional-access-policy-tampering.yml` Sigma rule’s scenario — a Conditional Access policy being weakened or disabled is itself a high-value target for an attacker who already has some foothold, because it removes a standing control rather than working around it once.

Privileged Identity Management (PIM) is the mechanism for just-in-time (JIT) privileged access instead of standing privileged access. Without PIM, being a Global Administrator means holding that role permanently — the permission exists on the account 24/7, whether or not anyone is using it in a given moment, which means the account is a maximally attractive target every single moment of every day, and any successful

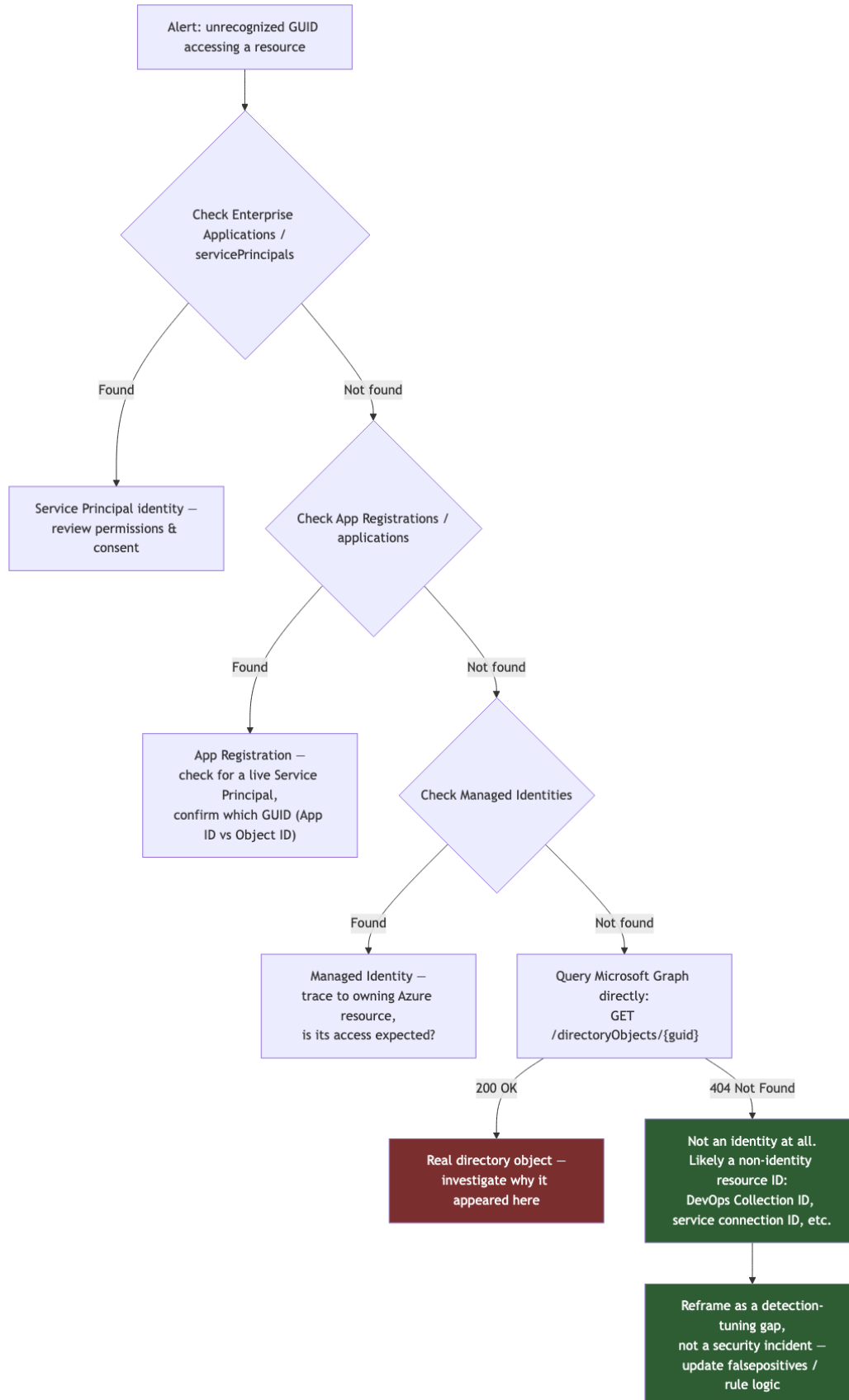


Figure 11: Diagram
46

compromise of it is instantly a full-privilege compromise. With PIM, a user is eligible for the Global Administrator role but doesn't hold it by default; they activate it for a limited time window (commonly requiring MFA re-confirmation and/or a business justification, sometimes requiring another admin's approval), use the privilege for what they needed it for, and it expires back to nothing automatically.

The reason PIM matters as a real control, not a compliance checkbox, is the attack-surface math: a compromised credential for a PIM-eligible-but-not-activated account gets an attacker nothing privileged — they'd additionally have to trigger and survive an activation flow (MFA, approval, justification, all of which can be logged and alerted on) to actually get the privilege. `privileged_role_access_review.py` in this lab is directly downstream of this idea, even without PIM configured: it flags directory-role members who haven't signed in for 30+ days, because a standing privileged role assignment on an account nobody is actively using is exactly the risk profile PIM is designed to eliminate — the role is a liability sitting there with nothing to show for it. The honest state of this lab, per its own README, is that the PIM walkthrough itself is one of the not-yet-done items in the IAM catalog — it needs a real Entra ID P2-licensed tenant (available via the free M365 Developer Program E5 trial) that hasn't been set up yet, so PIM here is explained as the correct control to reach for, not demonstrated end-to-end.

4.5 RBAC and least privilege

Role-Based Access Control (RBAC) means authorization decisions are made by assigning identities (or groups) to roles — named bundles of permissions — rather than granting individual permissions directly to individual identities one at a time. The value is manageability: reviewing “who has the Contributor role” is tractable; reviewing “what is the union of every individually-granted permission across every identity” is not, at any real scale.

Least privilege is the principle that an identity should hold only the permissions it actually needs to do its job, no more — and RBAC is the mechanism most identity systems use to try to approximate that in practice, imperfectly, because roles are necessarily coarser than any one person's actual job.

Role assignment scope in Azure

A critical, easy-to-get-wrong detail in Azure RBAC is that a role assignment isn't just “this role” — it's “this role, at this scope,” and Azure's scopes nest hierarchically:

```
Tenant (Entra ID directory root – Entra roles, not Azure RBAC)
├── Management Group
│   ├── Subscription
│   │   ├── Resource Group
│   │   │   └── Resource (a single VM, storage account, etc.)
```

A role assigned at a higher scope is inherited by everything beneath it. Assigning Contributor at the subscription level grants Contributor on every resource group and every resource inside that subscription — including resource groups and resources that get created after the assignment was made, with no additional action needed. This inheritance is powerful and exactly why over-broad scoping is one of the most common real over-privileging failure modes: an assignment made at subscription scope because it was the fastest way to unblock someone (or because the resource group they actually needed access to didn't exist yet) quietly grants access to everything that gets built in that subscription going forward, and nobody circles back to narrow it once the resource group in question does exist.

The terraform-labs repo's own operational discipline is a direct, if implicit, RBAC-scope-adjacent lesson: its README documents authenticating via `az login` rather than long-lived secrets, and running `terraform destroy` after every exercise — both of which are about minimizing the time window and blast radius that any credential or resource footprint exists for, which is the same underlying instinct as scoping a role assignment as narrowly as workable rather than as broadly as convenient.

Common over-privileging failure modes

- Scope creep by convenience — assigning at subscription or resource-group scope because it's one fewer decision than assigning at the individual resource, even when the actual need is one resource.
- Role creep over time — an identity accumulates role assignments across multiple projects/teams as it moves through an organization (the Mover problem from 4.2) and nothing is ever removed, only added.
- Built-in roles that are broader than the job — Contributor grants nearly everything except access-management actions; it's often assigned because it "obviously covers what's needed" without checking whether a much narrower built-in or custom role would have covered the actual task.
- Standing privileged roles instead of PIM-eligible ones (4.4) — RBAC and PIM are complementary, not competing: RBAC defines what a role can do, PIM (where configured) governs when an identity actually holds it.
- Group-based assignment drift — a role assigned to a group is only as tightly scoped as that group's membership is actively curated; a stale group with broad role assignment is a single point where JML failures (4.2) and RBAC failures compound each other.

4.6 Federation and identity providers (IdP)

An Identity Provider (IdP) is the system that authenticates a user and asserts their identity to other applications, so that individual applications don't each have to build and secure their own password store. Federation is the practice of trusting an external IdP's assertion rather than maintaining a separate identity for the same person in every system — sign in once to the IdP, and downstream applications trust that session.

Two protocols dominate how that trust assertion is actually carried:

- SAML (Security Assertion Markup Language) — an older, XML-based standard where the IdP issues a signed XML assertion ("this browser session belongs to user X, authenticated at time T, with these attributes") that the Service Provider (the application) validates and trusts. Very common in enterprise SSO, especially with legacy or on-prem-adjacent applications.
- OIDC (OpenID Connect) — built as a thin identity layer on top of OAuth 2.0, using JSON and JWTs (JSON Web Tokens) instead of XML. The IdP issues an ID token (a signed JWT asserting who the user is) alongside the OAuth access token that's actually used for API authorization. OIDC is the more modern default for web and mobile applications, and is what most consumer-facing "Sign in with X" flows use under the hood.

Neither protocol is intrinsically "more secure" — both, done correctly (validated signatures, checked audiences/issuers, short-lived tokens), provide a solid trust boundary; both, done incorrectly (accepting unsigned assertions, not validating the audience claim), are trivially bypassable. The practical difference is more about ecosystem fit: SAML shows up more in enterprise/legacy SSO integrations, OIDC in modern API-driven and mobile-first architectures.

Entra ID as an IdP, and why Auth0 was planned as a second one

This lab's identity content — the Sigma rules, the Graph API scripts, the Conditional Access/PIM discussion above — is almost entirely Entra ID-native, for the practical reason that Entra ID is the IdP this lab actually has meaningful, if incomplete, access to (via the free M365 Developer Program path referenced in `intune-endpoint-health-platform's` README).

Auth0 [FREEMIUM] (Okta-owned; free tier caps active users/organizations and gates advanced features like custom domains and some enterprise connections behind paid plans) was identified as a planned second identity provider for this lab specifically to diversify beyond Entra-only content, and to add detections based on Auth0's System Log — Auth0's own event stream (successful/failed logins, breached-password detections, anomaly signals) which has a genuinely different shape from Entra ID's `SignInLogs/AuditLogs`, and would have been a useful second data source to build detection content against rather than writing every rule against the same schema.

Being honest about this, per how this lab is actually documented: Auth0 was never finished. A dashboard URL was shared, but no working Machine-to-Machine application was ever actually configured — no Domain, Client ID, or Client Secret exist for it in this lab. Okta's free developer tier was discussed as an alternative and not pursued, in favor of Auth0, which itself then also wasn't finished. This is an honest, open gap, not a built thing dressed up as one — worth stating plainly rather than implying Auth0 content exists anywhere in this lab's detections when it doesn't. The value of naming it here is instructional: it's a real, common failure mode in lab and portfolio work generally (an ambitious second-platform expansion gets identified as valuable, gets as far as "account exists," and stalls before configuration) and it's more useful to a reader to see that named than papered over.

4.7 BEC and phishing as identity-adjacent threats

Business Email Compromise (BEC) and phishing are usually filed under "email security," but they belong in an identity security discussion because of what they're actually attacking: not a login flow, but trust in an identity itself — specifically, a recipient's trust that a message genuinely came from the sender it claims to. That's identity spoofing/impersonation, functionally the same underlying problem as a stolen credential, just executed without ever touching the target's authentication system at all. This lab's bec-phishing folder (in detection-engineering) documents two real cases that illustrate two distinct ways that trust gets forged.

Case 1: invoice fraud via authenticated bulk-mail services

The first case is invoice fraud delivered through legitimate, authenticated bulk-mail services — services like SendGrid or Mailchimp. The mechanism worth understanding closely: these services maintain their own sender reputation, built from the aggregate behavior of every legitimate customer sending through their infrastructure, and their outbound mail correctly passes SPF and DKIM for their own sending domain (because it genuinely originates from their infrastructure, with their own authentication configured correctly — nothing about the transport is forged). An attacker who signs up for one of these services and sends a fraudulent invoice through it inherits that platform's good reputation and correctly-passing authentication, which is exactly what lets the message bypass reputation-based and basic authentication-based filtering that would otherwise catch a more obviously spoofed message. The fraud isn't in the transport layer at all — SPF/DKIM genuinely pass, because the mail genuinely came from SendGrid/Mailchimp's servers, exactly as configured. The fraud is entirely in the content: an invoice with altered payment details, sent to look like it came from a known vendor, riding on infrastructure that authentication checks were never going to catch because those checks were never lying about where the mail came from.

Case 2: executive impersonation via external personal email

The second case is executive impersonation from an external personal email address, where the SPF/DKIM/DMARC picture is the opposite of case 1: misalignment, not clean pass. This is the more classically recognizable BEC pattern — an email claiming to be from a company executive, sent from a lookalike or unrelated external address, asking for an urgent wire transfer or gift-card purchase, exploiting organizational trust in a name and title rather than any technical trust mechanism. Here, DMARC alignment checks (does the visible From: domain match what SPF/DKIM actually authenticated?) are the relevant control — a message claiming to be from `ceo@company.com` but actually sent from an unrelated personal domain fails alignment, and a DMARC policy set to reject or quarantine (rather than just monitor) on failure is the concrete technical control that stops this specific pattern, if the organization's DMARC policy is actually enforced rather than left at `p=none`.

Why these are identity security problems, not just email problems

The reason both cases belong in an identity security discussion, not a separate "email security" bucket, is the throughline from 4.1: authentication is "who are you, and can you prove it," and both of these attacks are entirely about defeating that question for a human reader instead of for a login system. Case 1 defeats it by riding on infrastructure whose technical authentication is genuinely valid, while the represented identity

(the vendor) is not who actually sent the content. Case 2 defeats it by impersonating a name and title the recipient trusts, from an address that fails the technical authentication check that exists specifically to catch that. Neither attacker needed a password, a token, or an MFA bypass — they needed the recipient to believe an identity claim, which is the exact same target as a credential-stealing phish, reached by a different route. Framed against 4.1's authentication/authorization split, both cases are pure authentication-layer attacks aimed at a human verifier instead of a machine one — which is precisely why technical controls alone (SPF/DKIM/DMARC enforcement) address case 2 but only partially blunt case 1, and why case 1 ultimately still needs human judgment or content-based detection, not just sender authentication, to catch.

Part 5: Detection Engineering Principles

This part explains the discipline behind the content in the detection-engineering repo — not just what a Sigma rule or a KQL query looks like, but why the format and workflow are built the way they are, and where this lab's own detection content ran into the real limits of "write once, run anywhere." The central teaching example of this whole part is the last one: the actual, current gap between what this lab's detection content is written for (Sentinel/KQL) and what its live SIEM actually is (Wazuh) — and why that gap was left open rather than papered over.

5.1 Sigma rule anatomy

Sigma [FREE] (open, community-maintained YAML specification and tooling, [sigmahq/sigma](https://github.com/sigmaHQ/sigma) on GitHub, no license cost) is a generic, source-agnostic signature format for describing detection logic. Its entire reason to exist is stated plainly in this lab's own detection-engineering README: writing the same detection logic three times for three different query languages (KQL, SPL, Lucene, whatever) is a waste of effort, and worse, it means the logic itself — the actual security judgment about what pattern matters — is scattered across multiple platform-specific dialects instead of living in one place as the source of truth. A Sigma rule is written once, in a structured YAML document, and a converter (`sigma-cli` plus a target-specific backend/pipeline) compiles it into whatever query language a specific platform speaks. The Sigma file is the asset; the compiled KQL, SPL, or whatever else is one output of it — which is exactly why this lab keeps both the Sigma source and the generated KQL in the repo rather than just the compiled query, and treats the Sigma file as authoritative when the two could drift.

Here's a real rule from this lab, `sigma-rules/password-spray.yml`, with every field annotated:

```
title: Potential Password Spray Against Azure AD      # human-readable name
id: 4c1a9e3d-5b7f-4a6e-8d2c-3f4a5b6c7d8e           # globally unique UUID –
                                                    # stable identity for this
                                                    # rule across renames/edits
status: experimental                                # maturity: experimental,
                                                    # test, stable, deprecated
description: >
  Flags failed sign-in events that match common password-spray failure codes.
  This rule expresses the per-event condition only – the "many accounts, one
  source, short window" aggregation is added in the KQL layer...
references:
  - https://attack.mitre.org/techniques/T1110/003/
author: Danish Khan
date: 2026-08-29
tags:
  - attack.credential-access
  - attack.t1110.003                                # MITRE ATT&CK mapping –
                                                    # see 5.3
logsource:
  category: authentication                          # WHERE this rule applies –
                                                    # what kind of log source
```

```

product: azure                                # (product/service pair)
service: signinlogs                           # this rule targets, before
                                              # any specific query syntax

detection:
  selection:
    ResultType:
      - 50053 # account locked
      - 50126 # invalid username or password
    condition: selection                      # boolean logic combining
                                              # named selection blocks
falsepositives:                              # honest, specific – see 5.4
  - Shared corporate NAT/VPN egress IP causing many users to appear to share a source IP
  - A single user genuinely mistyping their password multiple times
level: high                                  # severity: informational,
                                              # low, medium, high, critical

```

The fields that do the real work are `logsource` and `detection`. `logsource` is deliberately abstract — it says “this rule needs an authentication-category log from Azure’s sign-in service,” not “query the SigninLogs table in Sentinel specifically.” That abstraction is what makes conversion to different backends possible at all: a pipeline (covered in 5.2) is the piece of configuration that maps `logsource: {category: authentication, product: azure, service: signinlogs}` onto an actual table name in a specific platform. `detection` is where the actual matching logic lives — one or more named blocks (here just `selection`) combined by a `condition` field, which can reference multiple named blocks with `and/or/not` for more complex rules.

`falsepositives` and `level` look like boilerplate metadata but aren’t — 5.4 covers why `falsepositives` in particular deserves real, specific content, not a placeholder.

Where Sigma’s single-rule model genuinely runs out

It’s worth being honest about a real limitation this lab actually hit, because it’s more instructive than a rule that converted cleanly: password spray is fundamentally an aggregation pattern — many distinct accounts failing from one source IP within a short window — and Sigma’s single-rule specification has no clean way to express “count distinct X, grouped by Y, over a time window.” The rule above only captures the per-event condition (which failure codes count as a spray-relevant failure); the aggregation logic is written by hand directly into the generated KQL file (shown in 5.2), with a comment explaining why it lives there instead of in the Sigma source. This is a real, acknowledged drift risk — if the failure-code logic in the Sigma rule changes, whoever updates it has to remember the hand-written KQL aggregation exists downstream and isn’t automatically kept in sync by the converter. No cleaner solution was found within plain Sigma for this rule; it’s flagged in this lab’s own documentation as a known rough edge, not resolved.

5.2 KQL fundamentals

KQL (Kusto Query Language) is the query language for Azure Log Analytics / Microsoft Sentinel (and Application Insights, and a few other Azure data stores built on the same Kusto engine). It’s the compile target this lab’s Sigma rules are converted to, via `sigma-cli` with the `kusto` plugin (`pipx install sigma-cli && sigma plugin install kusto`).

A handful of operators do almost all the real work in detection-relevant KQL:

- **where** — row-level filtering, the direct analog of a Sigma `selection` block. `SigninLogs | where ResultType in~ ("50053", "50126")` filters down to rows matching either failure code (`in~` is the case-insensitive membership operator).
- **summarize** — aggregation: collapsing many rows into grouped statistics. This is exactly the operator Sigma has no equivalent for, which is why it’s the one hand-added to the generated password-spray KQL.

- **bin()** — time-windowing, almost always used inside a `summarize ... by` clause to bucket events into fixed intervals (`bin(TimeGenerated, 10m)` groups rows into 10-minute buckets) — this is what turns “count of failures” into “count of failures per 10-minute window,” which is the difference between a meaningless running total and an actual burst-detection signal.
- **join** — combining rows from two tables on a matching key, used when the detection logic needs to correlate across log types (e.g., joining `SignInLogs` to `AuditLogs` to see whether a suspicious sign-in was followed by a specific administrative action).
- **dcount()** — distinct count, almost always the aggregation function actually being computed inside `summarize` for identity-abuse patterns, because “how many failures happened” is far less meaningful than “how many distinct accounts failed” — a single account retried ten times is a mistyped password; ten distinct accounts each failing once from the same source in the same window is a spray.

Here’s the full generated KQL for the password-spray rule, showing how the Sigma selection/condition maps onto where, and how the hand-written aggregation extends it:

```
// Base condition below is what sigma-cli generates directly from the Sigma
// rule. The aggregation (many distinct accounts, one source, short window)
// is added by hand here – Sigma's single-rule spec has no clean way to
// express "count distinct X grouped by Y over a timeframe", so this part is
// maintained directly in KQL, with the Sigma rule as the source of truth for
// the underlying per-event condition it wraps.
SignInLogs
| where ResultType in~ ("50053", "50126")
| summarize
    FailedAccounts = dcount(UserPrincipalName),
    AccountList = make_set(UserPrincipalName, 20)
    by IPAddress, bin(TimeGenerated, 10m)
| where FailedAccounts >= 10 // tune this threshold against your own traffic before enabling
```

Reading this line by line against the operators above: `where ResultType in~ (...)` is the direct compile of the Sigma selection; `summarize FailedAccounts = dcount(UserPrincipalName) ... by IPAddress, bin(TimeGenerated, 10m)` is the hand-added aggregation, grouping by source IP and 10-minute time bucket and counting distinct accounts (not total failed events) per group; the final `where FailedAccounts >= 10` is a tunable threshold, explicitly commented as needing calibration against real traffic rather than trusted as a universal constant — ten failed accounts in ten minutes might be a loud, obvious spray on a small tenant and background noise on a large one with shared corporate NAT egress, which is exactly the false-positive scenario the rule’s `falsepositives` field already names.

The pipeline problem: why conversion isn’t automatic for every log source

Converting Sigma to KQL isn’t just “run the compiler” — it needs a pipeline, a small piece of configuration telling the converter which Sentinel table a given logsource maps to. The built-in `sentinel_asim` pipeline shipped with `sigma-cli` only knows a limited built-in list of log categories out of the box (things like `network_connection`); anything outside that list fails conversion with `unable to determine table name from rule` — not because the rule is wrong, but because Entra ID sign-in and audit logs simply aren’t in that pipeline’s default mapping. This lab’s fix, `kql-conversions/pipelines/azuread-table-mappings.yml`, is a small (~15-line) custom pipeline that explicitly sets the table name for `SignInLogs`, `AuditLogs`, and `OfficeActivity` before `sentinel_asim`’s own resolution logic runs (it has to run at a lower priority number so it takes effect first). This is a concrete, real example of Sigma’s platform-agnostic promise needing real, platform-specific glue work to actually deliver — “source-agnostic detection logic” doesn’t mean zero platform-specific configuration exists anywhere in the pipeline; it means that configuration is isolated into one small, reusable mapping file instead of being baked into every individual rule.

5.3 MITRE ATT&CK mapping as a discipline

MITRE ATT&CK [FREE] (public knowledge base, MITRE Corporation, no cost, no license) is a structured taxonomy of adversary tactics (the “why” — the attacker’s goal, like Credential Access or Privilege Escalation) and techniques/sub-techniques (the “how” — a specific method, like T1110.003, Password Spraying, a sub-technique of the broader T1110 Brute Force technique).

Tagging a detection rule with an ATT&CK ID — as every rule in this lab does, e.g. `attack.t1110.003` on the password-spray rule, `attack.t1078.004` (Valid Accounts: Cloud Accounts) on the impossible-travel rule — does two real things beyond documentation. First, it gives the rule a stable, shared vocabulary that means the same thing across every detection engineering team on earth; “T1110.003” unambiguously means the same attacker behavior whether you’re reading this lab’s Sigma rule or a completely unrelated vendor’s threat intel report. Second, and more operationally useful, it makes coverage a measurable, queryable thing instead of a vague impression.

This is exactly what `attack-mapping.csv` in this lab is for — a flat table tying every rule to its technique, its data source, and its actual status:

Rule	ATT&CK Technique	Data Source	Status
suspicious-signin-velocity	T1078.004	Azure AD Sign-in Logs (SigninLogs)	converted (KQL) – not yet deployed
password-spray	T1110.003	Azure AD Sign-in Logs (SigninLogs)	converted (KQL) + hand-written aggregation
suspicious-oauth-consent	T1528	Entra ID Audit Logs (AuditLogs)	converted (KQL) – not yet deployed
privileged-role-assignment	T1098.003	Entra ID Audit Logs (AuditLogs)	converted (KQL) – not yet deployed
mass-file-download	T1567	M365 Unified Audit Log (OfficeActivity)	converted (KQL) – not yet deployed
conditional-access-policy-tampering	T1556.009	Entra ID Audit Logs (AuditLogs)	converted (KQL) – not yet deployed
suspicious-powershell-execution	T1059.001	Defender for Endpoint (imProcessCreate/DeviceProcessE	converted (KQL) – not yet deployed

A table like this is what turns “we have detections” into an answerable question: which techniques does this rule set actually cover, against which data sources, and — critically, and honestly captured in the Status column here — which of those are just syntactically valid versus actually validated against real production data. Read across this table, the real coverage claim is narrower than “8 rules covering 7 techniques” sounds: every single row says “not yet deployed/validated,” which is the honest state of this content (all 8 rules convert cleanly and are syntactically valid Sigma/KQL; none have run against a real Sentinel workspace with real sign-in traffic, so false-positive rates against real data are simply unknown). A coverage-tracking file that only recorded the technique mapping and left out the status column would overstate what this lab’s detection content actually proves — the Status column is what keeps `attack-mapping.csv` honest rather than decorative.

At a larger scale, this same discipline is how a real detection engineering team decides where to invest next: cross-reference the techniques a rule set covers against the techniques most relevant to the organization’s actual threat model (informed by industry reporting, red team findings, or a framework like the ATT&CK-based Center for Threat-Informed Defense’s coverage heat maps), and the gap between the two is the backlog. Without ATT&CK mapping, “what are we missing” has no rigorous answer at all — just impressions.

5.4 False positive tuning philosophy

Alert fatigue is not an abstract UX complaint — it’s the mechanism by which a detection stops working even while it keeps firing correctly. An analyst who has seen the same rule fire 40 times for a benign reason stops reading alert #41 carefully, and alert #41 being the one real positive doesn’t change that they’ve been trained, by every prior instance, to expect noise. This is a real cost with a real failure mode attached, and it’s the reason the `falsepositives` field in a Sigma rule deserves to be treated as a living, honestly-maintained piece of the rule’s actual logic, not a boilerplate field filled in once and forgotten.

Look again at the two rules quoted in 5.1 and 5.2. `impossible-travel.yml` names specific, plausible causes: “New device or browser used for the first time by a legitimate user” and “Corporate VPN or anonymizing proxy used intentionally by the organisation.” `password-spray.yml` names “Shared corporate NAT/VPN egress IP causing many users to appear to share a source IP” and “A single user genuinely

mistyping their password multiple times.” Every one of these is specific enough to actually inform a tuning decision — a real analyst reading “shared corporate NAT/VPN egress IP” knows exactly what to check (is this source IP a known corporate egress point? if so, either exclude it or raise the distinct-account threshold for it specifically) rather than being handed a generic “some false positives may occur” that provides zero actionable direction.

The philosophy this implies: a `falsepositives` field should be written by actually thinking through “what legitimate, non-malicious activity would trigger this exact condition” for this specific rule’s exact logic, not copy-pasted from a template. And it should change over time — as a rule actually runs against real traffic (which, honestly, none of this lab’s 8 rules have done yet, per 5.3’s status table), the real false-positive sources that show up in production should get added to this field, and thresholds (like `password-spray.yml`’s `KQL layer FailedAccounts >= 10`) should get tuned against what real traffic actually looks like, not left at a plausible-sounding placeholder forever. A `falsepositives` field that never changes after a rule goes live is a signal that nobody’s actually reviewing what the rule fires on — which is the exact condition that produces alert fatigue in the first place, quietly, over months, until the rule gets muted or ignored rather than fixed.

The `mass-file-download` rule’s status entry is worth calling out specifically here too: it’s explicitly flagged in `attack-mapping.csv` as “high FP risk by design.” That’s an honest acknowledgment that some detections are inherently noisy because the underlying behavior (downloading many files) is extremely common and legitimate, and the rule exists anyway because the alternative — not detecting mass exfiltration via M365 at all — is worse than tuning a noisy rule down over time. Naming that risk explicitly, in the coverage table itself, is exactly the same honesty principle as a well-written `falsepositives` field: say what the rule will actually do in production, don’t let the deployment discover it the hard way.

5.5 The platform translation problem

This is the central teaching example of this whole part, and it’s worth stating plainly rather than softening: this lab’s detection content is written for Microsoft Sentinel’s KQL schema — **SigninLogs**, **AuditLogs**, **OfficeActivity** — but the SIEM actually running in this lab, live, is Wazuh [FREE] (Apache 2.0-licensed, self-hosted, no tier or feature paywall — the full product is free). That’s not an oversight to be fixed by “just converting the KQL to Wazuh’s rule format.” It’s a real architectural mismatch, and understanding why it’s real, not just asserted, is the actual lesson.

Why “just convert it” isn’t trivial

Wazuh is built around host- and network-level telemetry: log files tailed off a monitored endpoint (via its agent or `<localfile>` configuration), Windows Event Log channels, file integrity monitoring, rootkit/vulnerability detection, and its rule engine (decoders + rules, largely XML-based, matching against parsed log fields) is designed around that shape of data. This lab’s own working example of Wazuh doing exactly what it’s good at is the MySQL container monitoring: Wazuh ships built-in decoders and rules for MySQL’s log format out of the box (`ruleset/decoders/0150-mysql_decoders.xml`, `ruleset/rules/0295-mysql_rules.xml`), so four deliberate wrong-password login attempts against the `soc-lab-mysql` container correctly fired Wazuh’s built-in rule 50106 (“MySQL: authentication failure,” level 9) with no custom Wazuh rule needed at all — because MySQL error-log-format authentication failures are precisely the kind of host-adjacent log Wazuh’s ruleset was built to parse. The Cloudflare Pages traffic case is the same pattern one level harder: Wazuh has no built-in decoder for arbitrary JSON web logs, but it does auto-decode top-level JSON keys into `data.<field>` once told the log format is JSON, so only a custom rule (id 100011, matching `data.url` against `/login.html`) was needed, not a custom decoder — still fundamentally the same kind of task Wazuh is built for: matching structured fields inside a log line that arrives, one event at a time, from something that resembles a host or a web request.

Entra ID sign-in and audit logs are a completely different shape of problem. **SigninLogs** and **AuditLogs** are cloud identity provider telemetry — structured JSON records describing authentication and directory-management events at the tenant level, with no host, no agent, and no local log file to tail in

the first place; the data lives in Microsoft's cloud and is queried through Log Analytics/KQL, not read off a filesystem. More importantly, the detection logic itself — the password-spray rule's `summarize dcount(UserPrincipalName) by IPAddress, bin(TimeGenerated, 10m)` aggregation covered in 5.2 — is a cross-event, cross-account, time-windowed aggregation query, which is exactly the kind of query Kusto/KQL is built to express fluently and Wazuh's per-event decoder-and-rule matching engine is not built to express at all in the same way. Wazuh does have some correlation capability (rule-to-rule triggering within a time window via `<if_matched_sid>/frequency` rules), but reproducing “count distinct accounts failing from one IP in a 10-minute window, across a stream of cloud identity events that were never designed to land on a Wazuh-monitored host in the first place” would mean building an entirely separate ingestion pipeline (something to pull Entra ID logs out of Azure and reformat them into a form Wazuh could parse, conceptually similar to the MySQL relay script but for a fundamentally different, higher-volume, schema-rich data source) and then re-deriving the aggregation logic inside Wazuh's much more limited correlation primitives.

Why forcing the fit would be worse than admitting the gap

It would be possible to technically write something that superficially resembles this detection logic as a Wazuh rule — matching on some subset of fields, approximating the aggregation with a frequency rule, producing an alert that looks like it's “the Wazuh version” of the password-spray detection. That would be worse than the honest gap, for a specific reason: it would produce a rule that looks deployed and functioning while actually evaluating a materially different, weaker condition than the one the Sigma rule and its `falsepositives` field were actually reasoned about — and nobody reading “password-spray: covered” in a status table would know that without reading the Wazuh rule's actual logic line by line. That's a worse state than an honest gap, because an honest gap is visible and can be prioritized; a fake-fit rule creates false confidence that quietly erodes the moment it's actually tested against real traffic (or, worse, never gets tested and just sits there being wrong).

This is why the actual decision in this lab was to pick one honest target platform per piece of content rather than force universal coverage: the Sigma/KQL detection content stays written and reasoned about against Sentinel's real schema, because that's the schema Entra ID identity telemetry actually has, and it's ready to point at a real Sentinel workspace the moment one exists — while Wazuh, running live in this lab today, is used for exactly what it's actually good at and actually has real data for right now (MySQL auth failures, HTTP request patterns from the Cloudflare Pages site), and is explicitly not asked to pretend it's evaluating cloud identity provider telemetry it was never built to receive.

The generalizable lesson

The teaching point here generalizes well past this one lab: “our detection content covers X” is a claim about logic, and that claim is only true for the platform the logic was actually reasoned about against. A Sigma rule's portability is real — the same rule genuinely can target Splunk, Elastic, or Sentinel by swapping backends, because those are all query engines built to answer the same kind of question (structured search and aggregation over structured events) even if the syntax differs. But portability to a platform built for a different kind of question entirely — host telemetry versus cloud IdP telemetry — isn't a backend-swap problem, it's a “this platform cannot ask this question the way it needs to be asked” problem, and no amount of syntax conversion fixes that. Recognizing which kind of gap you're looking at — and saying so plainly in a status table or a README, the way `attack-mapping.csv`'s status column and this lab's own documentation do — is the actual discipline. Pretending broad coverage exists because two rules technically both have the word “rule” in them is the failure mode this whole section exists to name and avoid.

Part 6: Incident Response & Triage Workflows

6.1 The alert → investigation → case → resolution lifecycle

Every incident response process, no matter how mature the organization, moves through the same four stages. Understanding what each stage is actually for — and where the boundaries between them sit — matters more than memorizing tool names, because the tools change (this lab uses Wazuh [FREE] and TheHive/Cortex [FREE]; a production shop might use Splunk [PAID] and ServiceNow SecOps [PAID]) but the stages don't.

Alert is a machine-generated signal: a correlation rule matched a pattern in log data and produced a record with a severity level, a source, and some context. An alert is a hypothesis, not a finding. Wazuh's rule 100011 firing on a request to `/login.html` on the lab's Cloudflare Pages site is an alert: it says "this pattern is worth a human look," nothing more. It does not mean an attack happened.

Triage is the stage most often confused with "investigation," and the distinction matters enough to state plainly: triage is a rapid initial assessment, not a full investigation. Its job is to answer a small, fixed set of questions fast — who or what is involved, how many events, is this expected activity, does this warrant escalation — using only the evidence already sitting in the alert plus one or two quick supporting queries. Triage that takes twenty minutes and touches six different log sources has stopped being triage; it has quietly become an investigation with a light disguise. A SOC that cannot triage in under a few minutes per alert cannot keep up with alert volume, which is precisely why triage exists as a distinct, disciplined step rather than everyone just "starting to investigate."

Investigation is what happens after triage concludes an alert is worth real time: full evidence gathering across every relevant log source, scoping ("what else did this actor touch"), timeline reconstruction, and — critically — it happens inside a case, not inside the SIEM's alert queue. This is where a case management system earns its keep. A SIEM like Wazuh is built to ingest, correlate, and surface events at scale; it is not built to hold a running narrative of "here is everything we know about incident #47, assigned to analyst X, evidence attached, decisions logged with timestamps and reasoning." That is a fundamentally different data model — structured casework, not log correlation — and it is exactly the gap TheHive [FREE] (case management) and Cortex [FREE] (automated observable enrichment, running analyzers against IOCs attached to a case) are designed to fill. TheHive gives a case a lifecycle (open → in progress → closed), tasks, observables, and an audit trail; Cortex takes an observable dropped into a case — an IP, a hash, a domain — and runs it through configured analyzers automatically.

Resolution closes the loop: root cause determination, containment verification, and — the step most SOCs skip under time pressure but shouldn't — feeding what was learned back into detection engineering. If an alert turned out to be a true positive that took too long to catch, that's a detection gap to close. If it turned out to be a noisy false positive, that's a tuning task. Resolution without that feedback loop just means the same class of incident gets triaged from scratch again next time.

The honest gap in this lab

Wazuh and TheHive/Cortex both run in this lab (`~/securitylab/wazuh-docker` for Wazuh; TheHive 5.4 + Cortex 3.1.7 backed by Cassandra 4.1 and Elasticsearch 7.17.24 for the case management side), but they are not wired together. TheHive and Cortex's containers exist and have been built, but were stopped/idle during this lab session, and there is no active integration — no webhook, no Wazuh-to-TheHive connector, nothing — turning a Wazuh alert into a TheHive case automatically. This is a real, documented gap, not a detail being glossed over: today, when Wazuh's rule 100011 or rule 50106 fires, that alert lives and dies in the Wazuh dashboard's alert list. Nobody and nothing opens a case for it. In a production SOC this integration is usually one of the first pieces of plumbing built (Wazuh can forward alerts via its integration framework — a shell/Python script hook, or a Filebeat output — to a webhook that creates a TheHive case, and Cortex-Wazuh-Connector is a real open-source project built for exactly this), but in this lab it remains a next step, not a finished capability. Anyone reading this document and reproducing the lab should treat

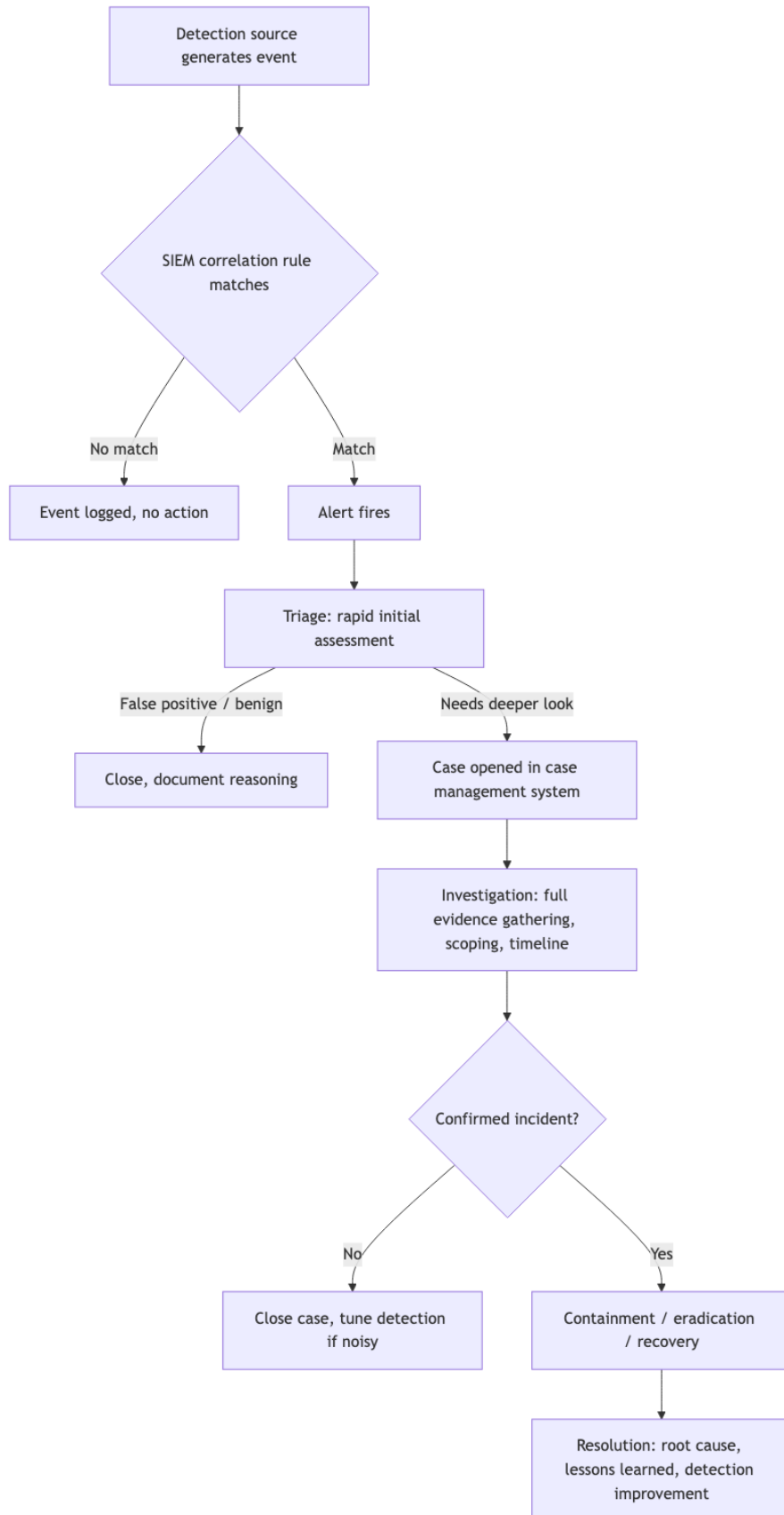


Figure 12: Diagram
57

“wire Wazuh alerts into TheHive automatically” as an explicit, valuable extension exercise, not something to assume already exists.

When to use a dedicated case management system / when not to

Use one the moment more than one person needs to work an incident, or the moment “what did we do and why” needs to survive longer than someone’s memory of a Slack thread. Use one when observables need repeatable automated enrichment (Cortex analyzers) rather than an analyst manually pasting a hash into five different lookup sites every time. Use one when compliance or after-action review requires a defensible audit trail of who decided what, when.

Don’t bother for a one-person lab where every alert gets looked at within minutes and closed the same session — the overhead of opening and maintaining a case for every single alert exceeds the value, which is part of why this lab’s current single-analyst workflow (a human looking directly at the Wazuh dashboard) hasn’t urgently needed the TheHive integration finished yet. The gap is real and worth closing for the sake of completeness and practice, but it isn’t blocking anything today, precisely because alert volume here is low and there’s only one person triaging.

6.2 Real case walkthroughs

The following walkthroughs are graded by how real they are. Some are fully worked runbooks from this lab’s actual repos. Some are real alerts that actually fired during this lab session, with real evidence. One category is explicitly hypothetical — planned but not yet run — and is labeled as such throughout; it is included because it teaches what triage would look like once the missing piece (a Windows VM with telemetry) exists.

Walkthrough 1: The GUID triage runbook (**detection-engineering/guid-triage/**)

This is the fullest worked example in the portfolio, built specifically to teach triage discipline for a hard identity-security problem: a GUID (a service principal ID, an object ID, an application ID) shows up in a sign-in log or audit log, and the analyst has to figure out what it actually is before deciding whether the activity is a threat.

What the analyst sees. A Sentinel-schema SigninLogs or AuditLogs entry where the actor or target is a raw GUID rather than a friendly username — e.g. a service principal authenticating, or an application ID appearing as the actor in an audit event. Unlike a human sign-in, there’s no immediately readable “who,” and that opacity is exactly what makes GUIDs a favorite blind spot: analysts under time pressure sometimes wave a GUID through as “probably some system thing” without checking, which is the failure mode this runbook exists to prevent.

What the analyst checks, in order:

1. Resolve the GUID’s identity first. Query Entra ID (via `az ad sp show --id <guid>` or the Graph API `/servicePrincipals/{id}` endpoint) to determine what the GUID actually is: a first-party Microsoft service principal, a third-party enterprise application, a custom-registered app your org owns, or — if the lookup returns nothing — a GUID that no longer resolves to any object, which is itself a signal worth flagging (deleted app, or a fabricated/spoofed ID that never existed).
2. Check the application’s owner and creation date. A service principal created yesterday behaving unusually is a very different story than one that’s been running unchanged for two years.
3. Check what permissions/scopes it holds. A GUID with `Directory.Read.All` or high-privilege Graph scopes acting oddly is a materially higher-severity finding than one scoped to read a single mailbox.
4. Check the pattern of activity, not just the single event. Is this service principal authenticating from its usual pattern (same conditional access policy results, same IP ranges/ASNs, same frequency), or is this a deviation — a new location, a burst of activity, a permission grant it’s never made before?
5. Cross-reference against known-good inventories. Does this GUID appear in an approved application registry / CMDB? An unlisted GUID performing directory changes is far more suspicious than a listed one.

Conclusion the runbook teaches the analyst to reach: a GUID is not inherently suspicious — most sign-in and audit noise involving GUIDs is completely legitimate automation (backup jobs, sync connectors, monitoring tools) — but a GUID is also not inherently trustable by virtue of being a machine identity, which is the actual bias this runbook is correcting for. The triage verdict is a function of identity resolution + privilege level + behavioral deviation, evaluated in that order, and an analyst who skips step 1 (resolving what the GUID even is) cannot honestly reach any verdict at all.

Walkthrough 2: The two BEC/phishing cases (**detection-engineering/bec-phishing/**)

These two cases model the alert-to-verdict path for business email compromise, using Sigma rules built against email/identity telemetry.

Case A — Invoice fraud via bulk-mail infrastructure. The pattern: an attacker sends a fraudulent invoice or payment-redirect email that is not sent from a compromised internal mailbox, but from bulk commercial email infrastructure (mass-mailer services, marketing platforms) — mail servers with real deliverability reputation, making the mail more likely to bypass spam filtering than a fly-by-night sender domain would. The detection logic centers on the mismatch between the claimed sender identity (a name/display matching a known vendor or internal finance contact) and the actual sending infrastructure (a bulk-mail provider's IP/domain that has no legitimate business relationship with that claimed identity), combined with payment-redirect language patterns in the body/subject. Alert-to-verdict: the alert fires on the infrastructure mismatch; the analyst confirms by checking whether the claimed sender has ever legitimately used that bulk-mail provider before (usually no) and whether the message contains bank-detail- change or urgent-payment language (usually yes) — verdict reached without needing a full mailbox compromise investigation, because the sending infrastructure alone tells the story.

Case B — Executive impersonation via personal email. The pattern: an attacker registers or uses a personal free-webmail address (Gmail, Outlook.com style) and sets the display name to match a real executive's name, then emails staff — typically finance or HR — with an urgent, authority-pressured request (wire transfer, gift cards, sensitive data). The detection logic here centers on display-name spoofing detection: a message where the display name matches an internal executive but the underlying envelope/from address domain is an external free-webmail domain, especially when combined with urgency/authority-pressure language and a request type (financial, credential, data) that the real executive would not normally make by email to that recipient. Alert-to-verdict: the alert fires on the display-name/ domain mismatch; the analyst confirms by checking the actual sending domain against the executive's real corporate domain (trivial mismatch check) and by noting the classic BEC social-engineering markers (urgency, secrecy requests, off-channel payment instructions) — verdict reached quickly because the domain mismatch is a near-deterministic signal once you know to look at envelope-from rather than display-name.

Both cases teach the same underlying lesson from a different angle than the GUID runbook: in BEC, the infrastructure/domain is almost always the highest-signal thing to check first, because attackers can spoof a display name trivially but cannot spoof legitimate sending infrastructure without actually compromising it.

Walkthrough 3 (hypothetical — planned, not run yet): T1078.004 and T1059.001

The `atomic-red-team-validation` repo contains two case study folders — T1078.004 (Valid Accounts: Cloud Accounts) and T1059.001 (Command and Scripting Interpreter: PowerShell) — that are honestly documented in that repo as “planned, not yet run,” because no Windows test VM existed at the time they were written. That VM now exists (see Part 9, Lab 4), but Sysmon and a Wazuh agent have not yet been installed on it, so these two case studies remain unrun. What follows is explicitly a hypothetical walk of what triage would look like once real telemetry from an actual atomic test exists — not a record of anything that has happened.

T1078.004 (Cloud Accounts) — would-run triage shape: the Atomic Red Team test for this technique typically involves authenticating with a valid cloud-account credential from an atypical context (e.g., programmatic sign-in via CLI credentials in a way that mimics an attacker reusing stolen tokens). If real telemetry existed, the triage an analyst would perform: check the sign-in log for the authentication method and client

app used (does it match how this account normally authenticates — interactive browser vs. CLI/token), check the source IP/ASN against the account's normal pattern, and check what the session did immediately after authenticating (the “how many” and “what” of triage). The hypothetical verdict path mirrors the GUID runbook's logic: identity + context + behavioral deviation.

T1059.001 (PowerShell) — would-run triage shape: the Atomic Red Team test for this technique executes PowerShell with patterns commonly associated with malicious use (encoded commands, download-and-execute patterns, AMSI- bypass-adjacent syntax). If Sysmon were installed and forwarding Event ID 1 (process creation) and Event ID 4104 (PowerShell script block logging) to Wazuh, the triage an analyst would perform: read the decoded/deobfuscated command line, check the parent process (was PowerShell spawned by a normal admin session, or by something like an Office document process — a strong signal either way), and check for any follow-on network connections or process spawns from that PowerShell session. This is explicitly presented as “here is what the triage would look like,” not as an event that has occurred — the honest state of this lab is that this remains future work, tracked as Lab 5 in Part 9.

Walkthrough 4: The real live-fired alerts from this lab session

Unlike the previous walkthrough, these two are genuinely real: they fired during this lab session, against real (synthetic-but-live) traffic, and produced real alert JSON in the Wazuh dashboard. These are deliberately kept as micro-triage examples — real alert, real one-line assessment — because that is what most day-to-day triage actually looks like: fast, terse, and conclusive without needing an open case.

Alert: Cloudflare **/login.html** rule (custom rule ID 100011, level 7). Fired when a request hit `/login.html` on the lab's public Cloudflare Pages site (`soc-lab-target.pages.dev`), captured via the Pages Function → `wrangler tail` → Python relay → Wazuh pipeline (see Part 9, Lab 2 for the full data path).

- Who: the client IP surfaced via Cloudflare's `cf-connecting-ip` header in the alert JSON, plus country and user-agent.
- What: an HTTP request to `/login.html`, a page that does nothing but log to console — there is no real authentication behind it.
- How many: the request count for that source in the observed window (a single curl-driven test in this case, since the traffic generating this alert was the analyst's own verification curl requests).
- Is this expected: yes — this was deliberate verification traffic generated by the analyst to confirm the detection pipeline worked end-to-end, not an actual external actor. One-line verdict: “Expected — self-generated test traffic confirming rule 100011 fires correctly on `/login.html` requests; no real actor involved, pipeline verified functional.” In a live scenario with unrecognized source IPs hitting this same page, the verdict would instead flag it as possible scanner/ credential-testing reconnaissance, per the rule's framing, and warrant a quick IP reputation check as the next triage step.

Alert: MySQL failed-auth rule (built-in Wazuh rule ID 50106, level 9, “MySQL: authentication failure”). Fired against the `soc-lab-mysql` container via the `general/error log` → Python relay → Wazuh localfile → Wazuh's built-in `mysql_log` decoder/rule pipeline (no custom rule needed — see Part 9, Lab 3).

- Who: the MySQL client connection recorded in the error log for the failed authentication attempt.
- What: four deliberate wrong-password login attempts against the container, generated by the analyst as a verification test.
- How many: 4 failed attempts in the observed window.
- Is this expected: yes — deliberately generated to confirm the built-in MySQL detection pipeline worked, and it did: rule 50106 fired correctly and auto-mapped to PCI DSS/GDPR/HIPAA/NIST 800-53 compliance controls out of the box, which is one of the real advantages of using a SIEM's built-in ruleset over a custom one — that compliance mapping came for free. One-line verdict: “Expected — 4 self-generated failed logins confirming rule 50106 and MySQL log ingestion pipeline work correctly; no real brute-force actor involved.” In a live scenario, the same rule firing against a source that isn't the analyst's own testing would prompt an immediate check of attempt volume and source — a handful of failures from one internal IP is very different from hundreds from an external one.

6.3 A generic triage runbook template

The structure below is deliberately modeled on the shape of the GUID triage runbook — identity resolution first, context and privilege second, behavioral deviation third, verdict last — generalized so it applies to any new alert type, not just identity/GUID alerts. Use this as a starting skeleton when a detection fires on something with no existing runbook yet.

1. Context and scope

- What exactly fired the alert — which rule ID, which detection logic, what raw event triggered it?
- What is the blast radius if this is real — one account, one host, one application, or something broader?
- What time window does this alert cover, and is there a natural window to extend the query into (a few minutes before/after) to catch related activity the single event doesn't show?

2. Initial hypothesis

- State, in one sentence, what this alert is claiming might be happening. Being explicit here (rather than jumping straight to evidence-gathering) forces clarity about what you're actually trying to confirm or deny — exactly the discipline the GUID runbook enforces by making “resolve the GUID's identity” the mandatory first step rather than an optional one.

3. Evidence gathering (the “who/what/how many” of triage)

- Who/what is involved — resolve raw identifiers (GUIDs, IPs, hashes, usernames) to something meaningful before doing anything else. An unresolved identifier is not evidence, it's a placeholder.
- How many — is this a single event or a pattern? Volume changes the verdict more than almost anything else.
- Context checks — does this match a known-good inventory, an approved application list, a normal behavioral baseline for this identity/host? Pull only what's needed to answer the hypothesis — triage stops here, investigation is what happens if more is needed.

4. Confirm or deny

- Based on steps 1–3 only, reach one of three verdicts: benign/expected (close with documented reasoning), needs investigation (escalate to a case), or inconclusive (a legitimate third outcome — document what's missing and either escalate anyway on caution, or set a follow-up).
- Do not let “I'm not sure” default silently to “close it” — an inconclusive triage that gets closed without escalation is how real incidents get missed.

5. Containment decision

- If escalating: does this warrant an immediate containment action (disable an account, isolate a host) before the full investigation completes, or can containment wait until the case has more evidence? This decision mirrors the human-approval-gate logic covered in Part 7 for automated response — the same reasoning about irreversibility and blast radius applies whether the containment action is manual or automated.

6. Documentation

- Record the verdict and the one or two pieces of evidence that drove it, even for a benign close — especially for a benign close, because that's the record that lets someone later ask “why was this alert dismissed” and get a real answer instead of “the analyst probably checked and it was fine.” This is the exact discipline a case management system like TheHive is built to enforce structurally; until Wazuh

and TheHive are wired together in this lab, that documentation has to happen manually, which is itself a good argument for finishing that integration.

Part 7: SOAR & Automation Principles

SOAR — Security Orchestration, Automation, and Response — is the practice of turning repeatable response actions into code instead of a runbook a human executes by hand every time. This lab's SOAR content lives in `sentinel-soar-playbooks`, three playbook designs written for Microsoft Sentinel [PAID] (consumption-based; a free trial tenant is reachable via the M365 Developer Program) using Logic Apps as the automation engine. It is important to be precise about what “playbook design” means here: these are architecturally complete designs — trigger, logic, permissions, approval gates all specified — but none of them are deployed, because no real Sentinel tenant exists in this lab. Section 7.2 covers exactly what that honestly means as a portfolio artifact.

7.1 When to automate vs. keep a human in the loop

The real principle, and the one these three playbooks are each built around, is a function of two variables: reversibility and blast radius.

- Low-risk, high-volume, reversible actions are exactly what automation is for. If an action can be undone with no lasting harm, and a human doing it manually a hundred times a day is pure toil, automate it without a gate. Enrichment — looking things up, attaching context to an alert — is the canonical example: it changes nothing in the environment, so there is no failure mode where automating it does damage.
- Irreversible or high-blast-radius actions — anything that changes a real user's or real system's state in a way that isn't trivially undone, or that affects someone's ability to do their job — need a human explicitly in the loop before execution, not just notified after the fact. The cost of a false positive here (disabling the wrong account, isolating the wrong device) is not “an analyst wastes five minutes reviewing a lookup” — it's an actual business disruption, which is a categorically different risk than automation-for-enrichment carries.

This is not a binary “automate everything cautious, gate everything risky” rule applied uniformly — it's a spectrum, and the three playbooks in this lab sit at three different points on it deliberately, which is exactly why they're useful to study together rather than in isolation:

Playbook	Reversibility	Blast radius	Human gate?
auto-enrich-signin-alert	Fully reversible (read-only)	None	No
disable-compromised-user	Reversible, but disrupts a real person immediately	High (one user's access)	Yes — Teams approval
isolate-infected-device	Reversible (selective isolation), un-isolation is the risky reverse	High (one device's network access)	Partial — isolate is automated, un-isolate is manual-only

7.2 Playbook design walkthroughs

Playbook 1: auto-enrich-signin-alert — no approval gate, and why

What it does. Triggered by a Sentinel sign-in risk alert, this playbook performs read-only lookups: resolving the signing-in user's normal behavioral baseline, checking the source IP against threat intelligence / geolocation, pulling recent sign-in history for the same account, and attaching all of it back onto the alert as enrichment before a human analyst ever looks at it.

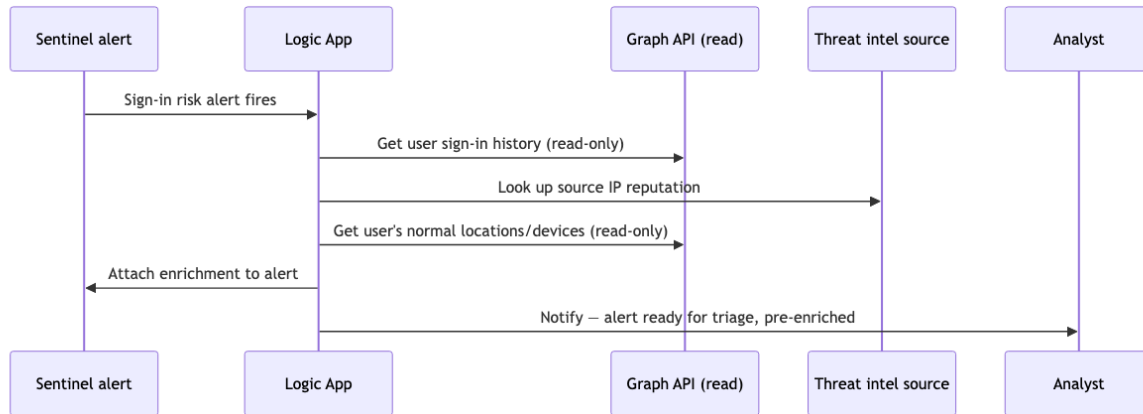


Figure 13: Diagram

Why no approval gate is needed here. Every action this playbook takes is a read. It queries state; it changes nothing. There is no scenario in which running this playbook against the wrong alert, or running it twice, or running it against a legitimate sign-in, causes any harm — the worst case is wasted API calls. This is precisely the “low-risk, high-volume, reversible” category from 7.1, and it’s also the highest-value automation target in practice, because enrichment is exactly the kind of repetitive lookup work that eats analyst time without requiring analyst judgment — the judgment still happens, it just happens with the evidence already assembled instead of the analyst spending the first five minutes of triage gathering it manually. This maps directly to the “evidence gathering” stage of the generic triage runbook in Part 6.3 — this playbook exists to make that stage faster, not to replace the “confirm or deny” judgment that comes after it.

Playbook 2: disable-compromised-user — Teams approval gate, and why here specifically

What it does. Triggered manually by an analyst (or by a high-confidence automated detection, depending on deployment posture) once a user account is believed compromised, this playbook would: revoke all active sessions for the user via the Microsoft Graph API’s session-revocation endpoint (invalidating refresh tokens so existing sign-ins are forced to re-authenticate), and disable the account (`accountEnabled: false` via Graph). Critically, before either action executes, the playbook posts an approval card into a Microsoft Teams channel and blocks until a designated approver responds.

Why an approval gate here specifically. This is the playbook that sits squarely in the “irreversible-ish, affects a real user’s ability to work” category. Disabling an account and revoking sessions is technically reversible — you can re-enable the account and the user can sign back in — but the impact is immediate and real: the person cannot work, cannot access email, cannot access anything gated by that identity, from the moment this executes. If the automation trigger is wrong (a false positive on compromise, or the wrong user identified), the damage isn’t undone by flipping the flag back — the person already lost working time, and if it’s an executive or someone in a time-sensitive role, that cost is not trivial. That asymmetry — fast to trigger, slow and disruptive to be wrong about — is exactly the shape of risk that argues for a human approval gate rather than full automation, even though the individual actions (revoke session, disable account) are simple, well-understood Graph API calls that could technically run unattended. The gate is not there because the automation is hard to build; it’s there because the cost of an automated false positive here is categorically higher than in Playbook 1.

Playbook 3: isolate-infected-device — selective isolation, and why un-isolation is manual-only

What it does. Triggered by a Defender for Endpoint detection, this playbook would isolate a device suspected of infection using Defender for Endpoint’s selective isolation capability rather than full isolation. Selective isolation cuts the device off from general network communication while preserving specific allowed traffic —

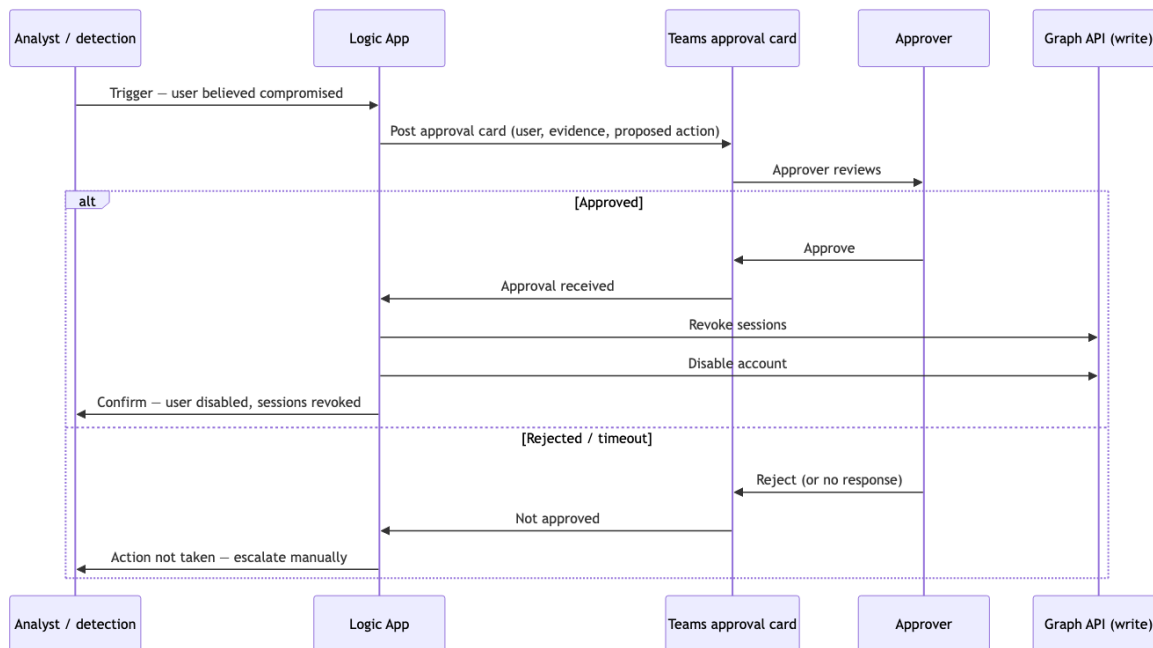


Figure 14: Diagram

notably, continued communication with the Defender for Endpoint management infrastructure itself, so the device stays manageable and telemetry keeps flowing during containment. Full isolation, by contrast, cuts everything, including the security tooling’s own management channel, which can leave a device isolated but unmanageable — unable to receive the un-isolate command remotely, unable to keep reporting telemetry that would help confirm the infection is actually contained.

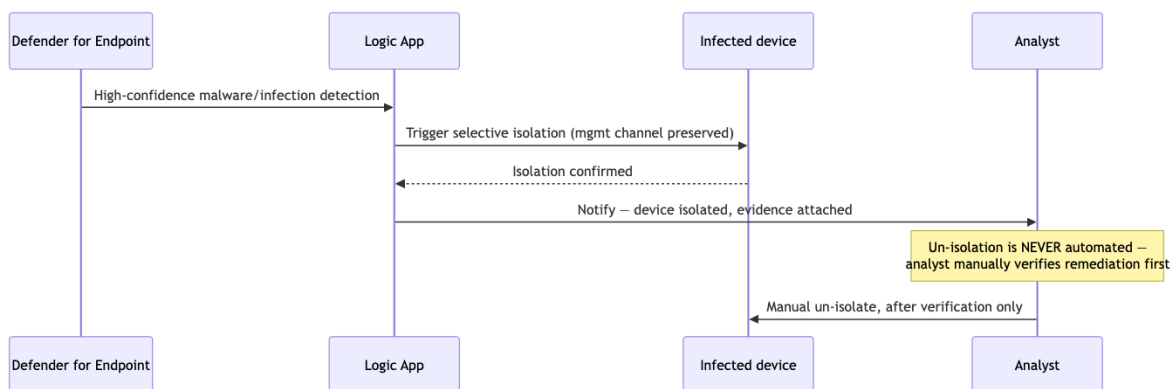


Figure 15: Diagram

Why selective is the safer automated default. Selective isolation is a narrower, more conservative action than full isolation — it removes the device’s ability to do damage (spread laterally, exfiltrate data, communicate with C2) while deliberately keeping the one channel that matters for staying in control of the device open. That makes it a better default for automated, unattended triggering specifically: if the detection turns out to be a false positive, the device is still reachable and manageable, and the disruption to the user is contained-but-recoverable in a way that a device gone completely dark over the network is not. This is the same reversibility logic as 7.1 — selective isolation is the more reversible of the two options for the same containment goal, so it’s the one that gets to run without a human in the loop first, whereas full isolation (or the un-isolation step below) is not.

Why un-isolation is kept strictly manual, always. Isolating a possibly- infected device automatically errs safe: worst case, a clean device gets briefly inconvenienced. Un-isolating a device automatically errs unsafe: the entire reason to isolate a device in the first place is that it might be compromised, and reconnecting it to the network without a human confirming remediation actually happened would just hand a possibly-still-compromised device its network access back on the automation's schedule rather than on verified evidence. There is no volume problem un-isolation-automation would solve that's worth that risk — un-isolation happens far less often than isolation, so the "high-volume" justification for automating simply isn't present, and the downside if wrong is severe. This is the cleanest example in the whole portfolio of the 7.1 principle applied asymmetrically to two sides of what looks like "the same" action: isolate and un-isolate are logical opposites, but they are not symmetric in risk, so they are not treated symmetrically in the design.

What "designed but not deployed" honestly means

None of these three playbooks have executed a single real action, because there is no real Sentinel tenant in this lab to deploy them into — only a free-trial-tenant possibility that has not been pursued. That is stated plainly, not softened. As a portfolio artifact, what "designed but not deployed" honestly represents is design thinking made concrete and reviewable — the trigger conditions, the exact API calls, the permission scopes, the approval-gate placement and reasoning, all specified precisely enough that a reviewer (or a future version of this lab with a real tenant) could deploy them largely as-is. What it does not represent is operational proof: nobody can point to a log showing this playbook actually revoked a session or isolated a device, because it never has. Claiming otherwise — implying these ran when they didn't — would be exactly the kind of dishonesty this whole document is built to avoid. The honest framing is "here is evidence I can design SOAR automation correctly, including judgment about where human gates belong," not "here is evidence this automation works in production," and those are genuinely different, both-valuable claims that should never be blurred together.

7.3 A general SOAR playbook design checklist

Use this checklist structure when designing a new playbook, whether or not it will ever be deployed. It's the same structure underlying all three playbooks above, made explicit and reusable.

Trigger conditions

- What exact event/alert fires this playbook? Be specific — a vague trigger ("suspicious activity") either never fires or fires constantly; a precise one (a specific Sentinel analytics rule ID, a specific Defender detection category) is testable and tunable.
- What's the expected trigger volume? This matters directly for the automate-vs-gate decision in 7.1 — a trigger that fires twice a year doesn't need the same automation investment as one that fires fifty times a day.

Required permissions/scopes (principle of least privilege for the automation's own identity)

- What is the minimum Graph API / Defender API / cloud provider scope this playbook's service identity needs to do its job — not what's convenient, what's minimum? `auto-enrich-signin-alert`'s identity needs read-only scopes (`AuditLog.Read.All`, `User.Read.All`) and nothing else — it should be structurally incapable of writing to anything, so that even a bug in the playbook's logic can't cause a write it was never supposed to make. `disable-compromised-user`'s identity needs the specific write scopes for session revocation and account disable (`User.RevokeSessions.All`, `User.ReadWrite.All` or a narrower equivalent) — and nothing broader like full Directory admin, even though a broader grant might be more "convenient" to set up once.
- A playbook's own service identity is a real attack surface — if that identity's credentials leak, the blast radius of that leak is bounded by exactly the scopes granted. Scope minimally for that reason alone, not just for tidiness.

Approval gates

- Does this action's reversibility and blast-radius combination (per 7.1) cross the line where a human needs to explicitly approve before execution, not just be notified after? If yes, specify exactly where in the flow the gate sits, who the valid approvers are, and what happens on timeout or rejection (never leave "approver doesn't respond" undefined — `disable-compromised-user's` design treats a timeout the same as a rejection: no action taken, escalate manually).

Rollback path

- If this action executes and turns out to be wrong, what's the exact reverse action, and is it itself automated or manual? State this even for actions that feel obviously reversible — "re-enable the account" is a real step with its own required permissions and its own decision about whether it needs a gate (per the isolate/un-isolate asymmetry in 7.2, the reverse of a gated or ungated action is not automatically the same category as the forward action).

What stays manual, and why

- Explicitly list what this playbook does not automate, and state the reason — not as an afterthought, but as a first-class design decision. For `isolate-infected-device`, un-isolation staying manual is not a gap to fill later, it's the correct permanent design given the risk asymmetry. Writing this down prevents a later well-meaning contributor from "finishing" the automation by removing a safeguard that was placed deliberately.

Part 8: Cloud Security & Infrastructure as Code

This part covers the `terraform-labs` and `aws-identity-detection` repos — the cloud/IaC-facing half of this lab. Two facts matter for reading this part honestly: none of the `terraform-labs` exercises have ever been `apply'd` against a real Azure subscription (they are `terraform fmt/validate` clean and honestly documented as `untested-in-anger`), and the AWS side is deliberately tested against a local emulator, not a real AWS account. That is not a weakness to apologize for — it is itself a security decision, and this part explains why.

8.1 Terraform + Azure: security principles beyond "how IaC works"

Terraform [FREE] (the open-source CLI, Mozilla Public License 2.0) is used here as the IaC tool of record. It is worth separating two things that get conflated: Terraform-the-CLI, which is free and self-contained, and Terraform Cloud, which is a hosted service with its own free and paid tiers (covered in Part 12). The `terraform-labs` repo's exercise 04 is specifically a Terraform Cloud integration exercise, but the CLI itself — what actually runs `plan/apply/destroy` — costs nothing.

Secrets hygiene in IaC: why `.tfstate` and `.tfvars` must never be committed

Two files are the recurring landmines in any Terraform repo, and both are landmines for the same underlying reason: Terraform's whole job is to describe and track real infrastructure, and real infrastructure has real identifiers, real network ranges, and sometimes real credentials baked into the description of what should exist.

- `.tfstate` (and `.tfstate.backup`) is Terraform's record of every resource it created and every attribute of that resource as last known — resource IDs, computed values, and in many providers' schemas, secret values that were set as resource arguments (a generated database password, a storage account key, a connection string) get written into state in plaintext by default, because state is what lets Terraform compute a diff on the next `plan`. A `.tfstate` file is not "metadata about infrastructure" in the way a Kubernetes manifest is — it is frequently a de facto credentials dump. Committing

it to a public repo is equivalent to publishing whatever secrets happened to exist in any resource Terraform managed, at any point in the project's history (git history matters here too — deleting the file in a later commit does not remove it from history).

- **.tfvars** files (`terraform.tfvars`, `*.auto.tfvars`) are where a user supplies input values to a module — and it is extremely common, especially early in learning Terraform, to put a real subscription ID, a real admin password variable, or a real API key directly into one of these files because it is the path of least resistance to “just get it running.” The fix is not “be careful” — the fix is that `.tfstate` and any `*.tfvars` file with real values in it belong in `.gitignore` from the very first commit of a Terraform repo, before a single `apply` is ever run, and `example/template` variants (`terraform.tfvars.example`) with placeholder values are what actually gets committed for other readers.

When to use this / when NOT to use this: the “never commit state or tfvars” rule is universal for any repo that is or might become public. It is slightly relaxable only in a fully private repo with restricted access and a remote backend already handling state (see exercise 02, the remote state Azure backend) — but even then, defaulting to “gitignore it anyway” costs nothing and removes an entire class of future accidents if the repo visibility ever changes.

gitleaks as a second-layer safety net — and its real, documented limitation

`gitleaks` [FREE] (open source, Apache 2.0) is a secret-scanning tool that greps commits (and optionally full history) for patterns that look like credentials — AWS access keys, private key headers, high-entropy strings adjacent to variable names like `password` or `secret`. It was run as part of this lab's confidentiality audit process (see Part 10) and is genuinely useful as a second layer of defense.

The honest limitation, documented in this lab's `terraform-labs` README: during testing, a password string containing a `$` character slipped past `gitleaks` undetected. Shell and Terraform interpolation syntax both use `$` heavily (`${var.name}`, `$VARIABLE`), which means pattern- and entropy-based scanners tuned to avoid flagging every interpolation expression in a codebase can end up under-flagging a real secret that happens to contain the same character. The lesson that matters here is specific and important: “the hook didn't complain” is not proof a commit is clean. A `.gitignore` that never lets the sensitive file reach `git` add in the first place is the actual control; `gitleaks` (or any secret scanner, pre-commit hook, or CI gate) is a backstop that catches mistakes the primary control missed — not a substitute for the primary control, and not something to trust blindly because it stayed silent.

When to use this / when NOT to use this: run `gitleaks` (or an equivalent scanner) on every repo before making it public, and ideally as a pre-commit hook so it never gets forgotten under deadline pressure. Do not treat a clean `gitleaks` run as a substitute for manually confirming `.tfstate/.tfvars/.env` files were never staged, and do not treat it as a substitute for a full-history audit before a repo's first public push (see Part 10's confidentiality audit case study) — a scanner tuned for common secret shapes will always have blind spots for the shapes it wasn't tuned for.

Least-privilege auth for IaC: **az login** vs. long-lived service principal secrets

Terraform needs to authenticate to Azure to do anything. There are two common ways to give it credentials, and they carry very different risk profiles:

- **az login** (interactive, browser-based, uses the Azure CLI's own token cache) authenticates as the actual logged-in human, using short-lived OAuth tokens that expire and require re-authentication — and critically, whatever MFA policy is on that account applies to this login too.
- A service principal with a long-lived client secret is a non-interactive credential (a client ID + client secret pair) meant for unattended/CI use. It works from any machine that has the secret, indefinitely, until manually rotated or expired — which is exactly what makes it dangerous if it leaks: unlike an interactive session, there is no human-in-the-loop MFA check standing between “someone has this string” and “someone can create/destroy real Azure resources.”

This lab's terraform-labs README states the deliberate choice: auth only via **az login**, never long-lived secrets, precisely because a single human doing exploratory, non-automated learning work has no real need for an unattended credential, and every unattended credential that exists is one more thing that can leak. A service principal with a scoped-down role and a short secret rotation window is the right answer for genuine CI/CD automation; it is the wrong answer for "I want to run terraform apply from my laptop while I'm sitting at it."

The other concrete guardrail documented in the same README: a spending cap is set on the Azure subscription before creating anything, and MFA is required on the account doing the az login. Neither of these is IaC-specific — they are account-level controls — but they matter specifically because IaC can create billable resources faster than a human reviewing a portal UI would, and a misconfigured for_each loop or an accidentally-applied module can create far more resources, far faster, than manual clicking ever would. A spending cap turns "I made an expensive mistake" into "I made a mistake that got capped," which is the entire point of setting it before the first resource exists, not after a surprise bill.

When to use this / when NOT to use this: use az login for any individual, interactive, exploratory Terraform work — which is all this lab's Terraform work is, since nothing has been applied yet. Use a service principal only when there is a genuine unattended automation need (a CI pipeline running terraform plan/apply on merge, for example) — and even then, scope its role assignment as narrowly as the pipeline's actual job requires, not Owner on the subscription out of convenience.

8.2 Case study: security-header baseline at the Application Gateway

terraform-labs exercise 06 defines Application Gateway [PAID] (Azure Application Gateway is a billed, consumption-plus-instance-priced Azure resource — there is no free tier; it is included here as IaC design, not as something that has been deployed) rewrite rules that add a baseline set of security response headers — Content-Security-Policy, Strict-Transport-Security, X-Content-Type-Options, X-Frame-Options — and that strip version-disclosure headers like Server and X-Powered-By before a response leaves the gateway.

```
# Illustrative shape of the exercise 06 rewrite rule set – generic
# example, not verified output, since nothing here has been applied.
resource "azurerm_application_gateway" "main" {
  # ... gateway core config ...
```

```
  rewrite_rule_set {
    name = "security-header-baseline"

    rewrite_rule {
      name          = "add-security-headers"
      rule_sequence = 100

      response_header_configuration {
        header_name = "Strict-Transport-Security"
        header_value = "max-age=31536000; includeSubDomains"
      }
      response_header_configuration {
        header_name = "X-Content-Type-Options"
        header_value = "nosniff"
      }
      response_header_configuration {
        header_name = "X-Frame-Options"
        header_value = "DENY"
      }
      response_header_configuration {
```

```

        header_name = "Content-Security-Policy"
        header_value = "default-src 'self'"
    }
}

rewrite_rule {
    name          = "strip-version-disclosure-headers"
    rule_sequence = 110

    response_header_configuration {
        header_name = "Server"
    }
    response_header_configuration {
        header_name = "X-Powered-By"
    }
}
}
}

```

The principle: centralize the control at a chokepoint

The reason this exercise exists is not “how do you set an HTTP header in Terraform” — it is a specific architectural argument for where a security control should live. If every application team behind the gateway is individually responsible for configuring their own app’s security response headers correctly, the realistic outcome across a portfolio of a dozen apps is a dozen slightly different, mostly-forgotten header configurations — some teams get it right, some ship the framework default (which often actively leaks version information, e.g. an unmodified `Server: nginx/1.18.0` or `X-Powered-By: Express`), and none of it is enforced anywhere.

Putting the rewrite at the Application Gateway — the single chokepoint every request and response already has to pass through — turns “twelve teams each configuring headers correctly” into “one gateway config, correct once, applying to everything behind it.” This is the same logic behind putting a WAF at a gateway rather than asking every app to implement its own input validation for every known attack pattern: a shared enforcement point scales better than distributed, per-team discipline, because it does not depend on every team remembering to do the right thing every time a new service is deployed.

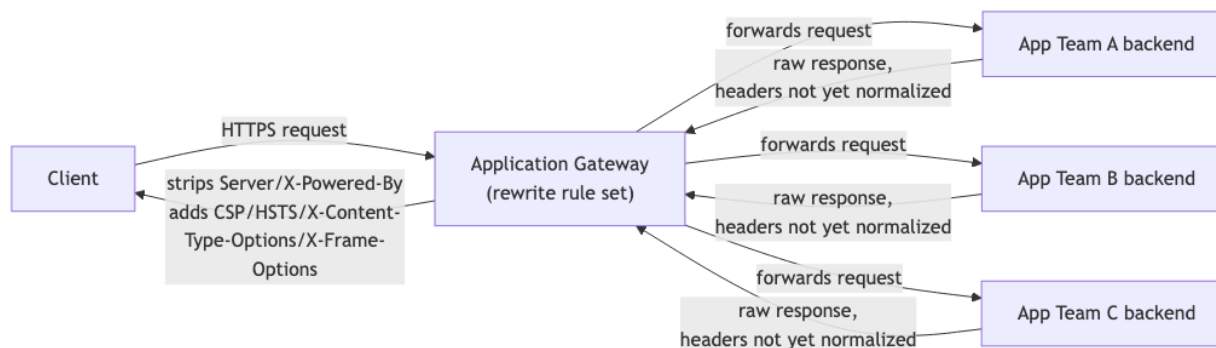


Figure 16: Diagram

The honest limitation

A gateway rewrite rule normalizes what leaves the gateway. It cannot verify two things that actually matter for whether the control is doing its job:

1. Whether the CSP value is actually a good policy for the app behind it. `default-src 'self'` is a reasonable generic baseline, but a real app might legitimately need to load fonts from a CDN, submit forms to a third-party payment processor, or embed a widget — none of which a blanket gateway-level CSP can know about. Setting the header is not the same as setting the right header for every app behind the gateway; it is a floor, not a tailored policy per app.
2. Whether the app has inline scripts that will break under it. A strict CSP without `'unsafe-inline'` or a nonce/hash mechanism will silently break any app that relies on inline `<script>` tags or inline event handlers (`onclick="..."`) — which is common in older or less disciplined frontend code. The gateway has no way to know this in advance; the failure mode shows up as “the app stopped working” in a browser console, not as a Terraform error, and it shows up after the rewrite rule is live, not before.

The realistic operating model this implies: a gateway-level header baseline is a good default floor, not a “set it and forget it” control. It still needs per-app validation (does this app actually work under this CSP?) and the CSP value itself still needs tuning per application over time — the gateway just makes sure something reasonable is applied everywhere by default, instead of nothing being applied anywhere until an app team gets around to it.

8.3 Case study: WAF-in-front-of-APIM, detection mode pilot

terraform-labs exercise 07 puts an Azure Web Application Firewall [PAID] (bundled with Application Gateway’s WAF SKU or as a policy attached to Front Door — billed, no free tier) in front of Azure API Management (APIM) [PAID], configured in detection mode rather than prevention mode.

Detection mode vs. prevention mode

A WAF evaluates every request against a ruleset (commonly the OWASP Core Rule Set) looking for patterns associated with SQL injection, XSS, path traversal, and similar attack classes. It has two fundamentally different operating modes:

- Prevention mode: a request that matches a blocking rule is actually blocked — the client gets a 403 (or similar), the request never reaches the backend.
- Detection mode: a request that matches a blocking rule is logged as if it would have been blocked, but is allowed through to the backend unmodified. Nothing about the live traffic changes; the only output is a log entry saying “this rule would have fired here.”

Why this staged-rollout pattern matters

This is a genuinely important, transferable security engineering principle, not a WAF-specific quirk: any new enforcement control should be piloted in a mode that observes without acting, before it is allowed to act on real traffic.

The reason is that a WAF ruleset — especially a broad managed ruleset like OWASP CRS — is tuned against generic attack patterns, not against this specific API’s actual, legitimate traffic shape. A ruleset that is excellent at catching SQL injection attempts can also flag a legitimate API request that happens to contain a string like `SELECT` or `OR 1=1` in a free-text field (a search box, a support ticket description, a code snippet a user is legitimately submitting) as if it were an attack. If that ruleset is switched straight to prevention mode on day one, every one of those false positives becomes a real outage — a legitimate user’s request gets silently dropped, and unless someone is watching very closely, the first sign of trouble is a support ticket, not a security log.

Running in detection mode first inverts that risk. The same ruleset evaluates the same live traffic and produces the same “this would have matched” log entries — but nothing actually breaks. That log becomes a tuning input: a security engineer can review which rules are firing, correlate them against known-legitimate traffic, and either accept a rule as a genuine true-positive detector or add a targeted exclusion for the specific parameter/pattern that a legitimate workflow happens to trigger. Only once that tuning pass has driven the

false-positive rate down to an acceptable level does the policy get flipped to prevention mode — at which point blocking real traffic is a calculated, informed decision instead of a blind bet.

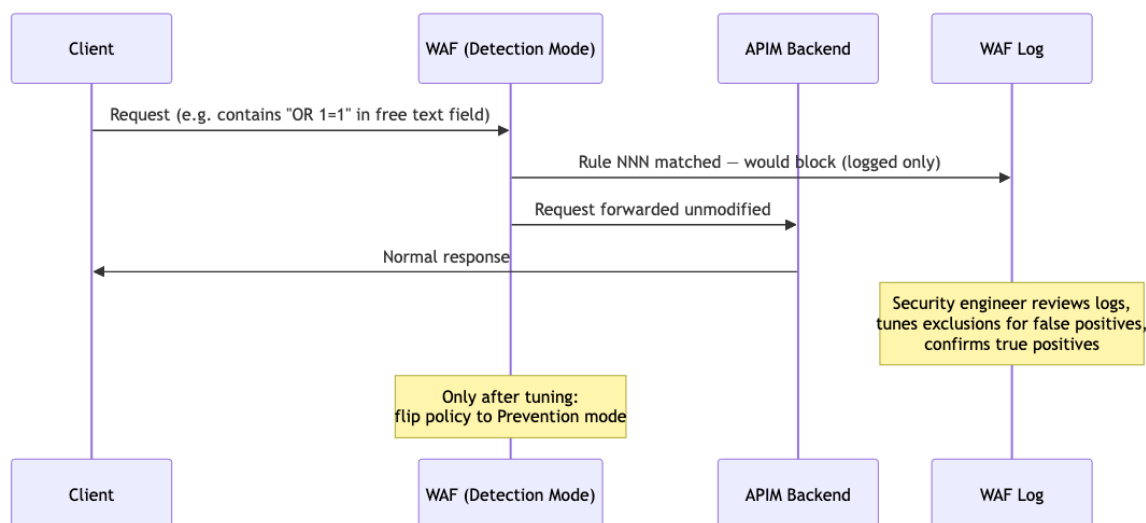


Figure 17: Diagram

When to use this / when NOT to use this: detection mode is the right starting point for any new WAF policy, or any change to an existing policy's ruleset (a CRS version bump, a new custom rule), being applied to traffic whose full shape isn't already well understood. It is the wrong place to stay indefinitely — a WAF sitting in detection mode forever provides visibility but zero actual blocking, which is a detection capability, not a protection capability; leaving it there permanently because "prevention mode might break something" just defers the tuning work rather than avoiding it, while leaving real attack traffic unblocked the whole time. The pilot in this lab's exercise 07 is explicitly a pilot — its own honest status, like every other terraform-labs exercise, is "designed and validated by terraform plan, never apply'd against a real APIM instance," so the detection mode log-and-tune loop described above has not actually been exercised against live traffic in this lab yet. What is real here is the design decision to build the exercise around a staged rollout instead of a straight-to-prevention deployment.

8.4 AWS side: IAM privilege escalation detection against LocalStack

The `aws-identity-detection` repo pairs AWS IAM privilege-escalation detection content with a local lab built on LocalStack [FREEMIUM] (the community edition is free and covers core services including IAM, S3, and Lambda; the paid Pro tier adds more services and features like persistent state across restarts and additional AWS API coverage — the community tier is what this lab uses).

IAM privilege escalation: what the detections are looking for

AWS IAM privilege escalation is a well-documented class of technique where a principal with a seemingly limited set of permissions uses those permissions to grant themselves broader ones — for example, a user who can call `iam:CreatePolicyVersion` on their own attached policy, or `iam:AttachUserPolicy` on their own user, or `iam:PassRole` combined with the ability to launch a Lambda or EC2 instance with an attacker-chosen role attached. None of these individually looks like a dramatic breach — each is a permission that might reasonably exist on a legitimate identity or automation role — but specific combinations of permissions on the same principal add up to "this identity can make itself an administrator." Detecting this class of technique means watching CloudTrail-equivalent event logs for the specific API call sequences and permission combinations associated with each known privesc path (there are roughly twenty-plus documented IAM privesc methods in the public research this space is built on), not just watching for any single

suspicious-looking call in isolation.

Why LocalStack instead of a real AWS account for this kind of testing

Testing privilege-escalation techniques — not just detections for them, but actually exercising the technique to confirm the detection fires — means deliberately creating IAM users, policies, and role chains designed to escalate privilege, and then actually escalating it. Doing that against a real AWS account carries two distinct risks that LocalStack removes:

1. Cost risk. Real AWS resources cost money the moment they exist, and it is easy to forget to tear something down after a test session — an EC2 instance, an RDS instance, or even just accumulated API call volume on some services can produce a real, unplanned bill days later when nobody is watching. LocalStack runs entirely as local Docker containers emulating the AWS API surface; nothing billable exists anywhere, and there is no “did I forget to destroy something” follow-up risk.
2. Blast-radius risk. A successful privilege escalation test, by definition, results in a principal gaining more access than it should have. Against a real account, that is a genuine security event — the escalated principal really can now do whatever the escalated permissions allow, against real infrastructure, real data, and potentially real production resources if the test account is not as isolated as assumed. Against LocalStack, the “account” is an emulated API surface with no real underlying infrastructure to actually affect — escalating privilege there proves the detection logic works without ever creating a real over-privileged identity anywhere.

The trade-off is fidelity: LocalStack emulates the AWS API surface, not every nuance of real AWS’s internal behavior, IAM policy evaluation edge cases, or service-specific quirks, so a detection validated purely against LocalStack should still be considered “logic validated,” not “proven identical to real AWS behavior” — but for iterating on detection logic and confirming a technique produces the expected log signature, it is the correct, safe, free tool for the job, not a compromise made only because a real AWS account wasn’t available.

When to use this / when NOT to use this: use LocalStack for developing and validating detection logic against IAM privesc technique shapes — confirming the right API calls get logged and the right patterns get flagged — cheaply and destructively, without any real-world consequence. Do not treat a LocalStack-validated detection as fully production-ready without a final validation pass against real AWS CloudTrail output (even read-only, non-destructive validation against a real, properly-isolated test account) before relying on it in production, since emulation fidelity gaps are a real, if narrow, risk.

Part 9: End-to-End Practical Labs

The labs below are reproducible steps for what was actually built and verified in this lab session, on a MacBook (Apple Silicon, 16GB RAM), macOS, using Homebrew, Docker Desktop, and QEMU. Each lab states its goal, prerequisites, exact steps, expected result, and what it teaches — and each is explicit about what was actually verified with real evidence versus what was merely assumed to work. Labs 1 through 3 were fully verified live. Lab 4 was verified up to a specific, stated point and no further. Lab 5 is explicitly not done yet and is included as a documented next step, not a completed lab.

Lab 1: Stand up Wazuh [FREE] locally via Docker

Goal

Get a working single-node Wazuh 4.9.2 SIEM/XDR stack (manager, indexer, dashboard) running locally via Docker, as the foundation every other lab in this document depends on.

Prerequisites

- Docker Desktop installed and running (macOS, Apple Silicon).

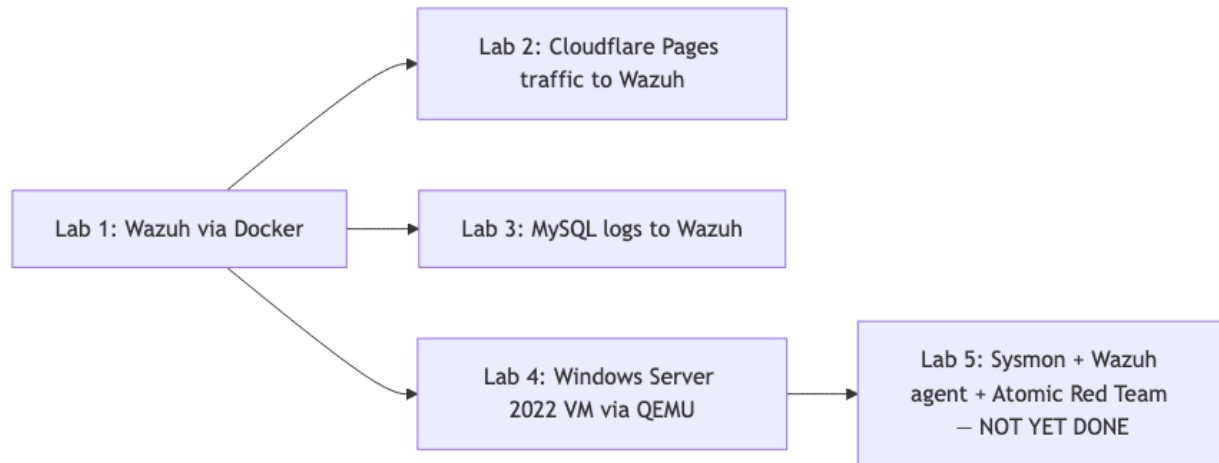


Figure 18: Diagram

- Git.
- At least a few GB of free disk for the indexer's data volume.

Steps

1. Clone the official Wazuh Docker repository to a location outside any git repo you intend to push publicly — this stack generates real certs and local data that should not end up in version control:

```

mkdir -p ~/securitylab
cd ~/securitylab
git clone https://github.com/wazuh/wazuh-docker.git -b v4.9.2
cd wazuh-docker/single-node
  
```

2. Generate the certificates the stack needs for the indexer/manager/ dashboard to talk to each other over TLS:

```

docker compose -f generate-indexer-certs.yml run --rm generator
  
```

The real bug hit and its fix: this cert-generator step failed on first run with a permissions error — the container attempts to write certificate files into a bind-mounted local directory, and depending on how Docker Desktop on macOS has mapped file ownership between the container's user and the host filesystem, the container's write can be rejected. The fix was ensuring the target certs directory was writable by the container's user before running the generator — on macOS with Docker Desktop this typically means checking the bind-mount path's permissions and, if needed, adjusting them (chmod on the local config/wazuh_indexer_ssl_certs/ directory) before re-running the generator step, rather than fighting Docker's user-namespace mapping directly. Once the target directory was writable, re-running the exact same generate-indexer-certs.yml command succeeded and populated the certs directory.

3. Bring the stack up:

```

docker compose up -d
  
```

4. Confirm all three containers are healthy:

```

docker compose ps
  
```

Expected: wazuh.manager, wazuh.indexer, and wazuh.dashboard containers all in a running/healthy state.

Expected result

- Wazuh dashboard reachable at `https://localhost` in a browser (self-signed cert — browser will warn, accept to proceed).
- Default login `admin / SecretPassword` — documented here explicitly as needing to be changed before this stack is ever exposed beyond localhost; it was left as default for this local-only lab session, which is fine for a lab bound to localhost and unacceptable for anything reachable externally.
- Indexer API reachable on port 9200.

What it teaches

- The shape of a real, self-hosted SIEM deployment: three cooperating services (ingestion/correlation, search/storage, visualization), not a single monolith — the same architectural split you'll see in Elastic's stack and most SIEM products, just open-source and free (Apache 2.0, no paid tier) instead of a managed/paid equivalent like Sentinel.
- That “the docs say run this compose file” and “it works on the first try” are not the same thing — a permissions mismatch between a bind-mounted host directory and a container's expected write access is one of the most common real-world Docker friction points, and diagnosing it (check what the container is trying to write, check whether the host path allows that) is a transferable skill well beyond Wazuh specifically.

Lab 2: Wire a real Cloudflare Pages site's traffic into Wazuh with a custom detection rule

Goal

Get live, real HTTP traffic from a real public website into Wazuh, and have a custom rule fire a real alert on a specific suspicious path — end to end, verified with real curl-generated evidence.

Prerequisites

- Lab 1 completed (Wazuh running locally).
- A Cloudflare [FREEMIUM] account (static Pages hosting is free; note Logpush/continuous log streaming is a paid-plan feature, which is exactly why this lab needed a different capture path — see step 3).
- `wrangler` (Cloudflare's CLI) installed via npm.
- Python 3 for the relay script.

Steps

1. Create and deploy a minimal static Cloudflare Pages site with a `functions/_middleware.js` Pages Function that logs every request:

```
// functions/_middleware.js
export async function onRequest(context) {
  const { request } = context;
  console.log(JSON.stringify({
    method: request.method,
    path: new URL(request.url).pathname,
    ip: request.headers.get("cf-connecting-ip"),
    country: request.headers.get("cf-ipcountry"),
    user_agent: request.headers.get("user-agent"),
  }));
  return context.next();
}
```

Deploy with:

```
wrangler pages deploy ./public --project-name soc-lab-target
```

Include a clearly labeled dummy `/login.html` (does nothing but log — no real authentication logic behind it) and `/contact.html`, tag pages `noindex,nofollow`, and put a visible on-page banner disclosing this is a synthetic lab fixture collecting nothing — required before pointing any real public traffic at it.

2. Authenticate wrangler — real lesson from this session: a hand-scoped Cloudflare API token is easy to under-scope (multiple tokens created and discarded in this session: one scoped only to R2 storage, others missing Pages Edit permission specifically). `wrangler login`'s browser-based OAuth flow grants broader scope by default and was faster than fighting the custom-token permission-group UI for this one-off admin task:

```
wrangler login
```

3. Capture live traffic. Cloudflare's Free tier has no Logpush/continuous log retention, so live traffic is captured by tailing the deployment directly:

```
wrangler pages deployment tail --format=json <deployment-id>
```

Documented limitation: this command binds to a specific deployment ID, which means it breaks on every redeploy — a new deployment ID means the tail command must be re-run with the new ID. This is a real, ongoing limitation of this capture approach, not a one-time setup cost.

4. Run a small Python relay that consumes the `wrangler tail` JSON output and appends each parsed event as a line to a file bind-mounted into the Wazuh manager container, e.g. `/var/ossec/logs/cloudflare-relay.log`.
5. Configure Wazuh to watch that file as JSON (in `wazuh-docker/single-node/config/wazuh_cluster/wazuh_manager/ossec.conf`):

```
<localfile>
  <log_format>json</log_format>
  <location>/var/ossec/logs/cloudflare-relay.log</location>
</localfile>
```

Wazuh auto-decodes top-level JSON keys into data. `<field>` for rule matching — no custom decoder was needed, since the relay already emits flat JSON with sensible field names.

6. Add a custom local rule in `wazuh-docker/single-node/config/wazuh_cluster/wazuh_manager/etc/rules/localrules` (a custom rule was needed here, unlike Lab 3, because Wazuh has no built-in decoder or ruleset for arbitrary JSON web logs the way it does for known log formats like MySQL's):

```
<rule id="100011" level="7">
  <if_sid>...</if_sid>
  <field name="data.path">^/login\.html$</field>
  <description>Possible scanner/credential-testing activity: request to /login.html</description>
</rule>
```

7. Restart the manager to load the new rule, then generate real traffic and confirm:

```
curl https://soc-lab-target.pages.dev/login.html
```

Expected result

Real request appears via the `wrangler tail` → relay → Wazuh pipeline, and rule 100011 fires a level-7 alert visible in the Wazuh dashboard, with the real client IP, country, and user-agent from the actual curl request attached in `data.*` fields. This was verified live in this session — real curl traffic produced real alerts, not a simulated/assumed result.

What it teaches

- The general pattern for feeding any source that doesn't have a native Wazuh integration: capture → relay/reformat → localfile → custom rule. This four-stage pattern generalizes far beyond Cloudflare.
- The distinction between a decoder and a rule: Wazuh's JSON auto-decode handled field extraction for free; the detection logic (what pattern in those fields matters) still had to be hand-written, because detection logic is inherently source- and use-case-specific in a way generic field parsing isn't.
- A real operational limitation (the deployment-ID binding breaking on redeploy) that a from-scratch reader would hit and should expect, not be surprised by.

Lab 3: Wire a real MySQL container's logs into Wazuh using its built-in `mysql_log` decoder/rules

Goal

Feed a real MySQL 8.0 container's authentication logs into Wazuh and confirm Wazuh's built-in MySQL ruleset detects failed authentication — deliberately contrasted with Lab 2, which needed a custom rule because no built-in ruleset existed for that source.

Prerequisites

- Lab 1 completed.
- Docker, for running the MySQL container.

Steps

1. Stand up a MySQL 8.0 container with general query logging and error logging both enabled:

```
docker run -d --name soc-lab-mysql \
  -e MYSQL_ROOT_PASSWORD=<lab-only-password> \
  -v soc-lab-mysql-logs:/var/log/mysql \
  mysql:8.0 \
  --general-log=1 \
  --general-log-file=/var/log/mysql/general.log \
  --log-error=/var/log/mysql/error.log
```

2. Run a small Python relay script that tails `error.log` and `general.log`, and reformats matching authentication-related lines to fit Wazuh's expected MySQL log: ... prematch format (the format Wazuh's built-in `0150-mysql_decoders.xml` decoder expects to match against), appending them to a file bind-mounted into the Wazuh manager container.
3. Configure Wazuh to pick the relayed file up as `syslog` format (not `json` this time — the relay's job is specifically to produce lines that match the built-in decoder's expected prematch, so plain syslog-style ingestion is correct here):

```
<localfile>
  <log_format>syslog</log_format>
  <location>/var/ossec/logs/mysql-relay.log</location>
</localfile>
```

4. Confirm the built-in decoder and rules are present in the manager's ruleset (no custom rule file needed for this lab — this is the whole point of the contrast with Lab 2):

```
docker exec wazuh.manager grep -l "MySQL" /var/ossec/ruleset/decoders/0150-mysql_decoders.xml
docker exec wazuh.manager grep -l "50106" /var/ossec/ruleset/rules/0295-mysql_rules.xml
```

5. Generate real failed-login traffic against the container:

```
for i in 1 2 3 4; do
    mysql -h 127.0.0.1 -u root -pWrongPassword$i 2>&1
done
```

Expected result

Four deliberate wrong-password login attempts correctly fired Wazuh's built-in rule 50106 ("MySQL: authentication failure", level 9). Verified live in this session. As a side benefit of using the built-in ruleset rather than a custom one, the alert auto-mapped to PCI DSS, GDPR, HIPAA, and NIST 800-53 compliance controls out of the box — mappings that come baked into Wazuh's official ruleset and that a hand-written custom rule would not have had unless the compliance tags were manually added.

What it teaches

- The direct contrast with Lab 2: when a log source has a well-known, standard format (MySQL's error/general log format is stable and documented), a mature SIEM ships decoders and rules for it already — the work is entirely in getting the log lines into the expected shape and location, not in writing detection logic from scratch. When the source is arbitrary/custom (a JSON web log with lab-specific field names), no such ruleset can exist ahead of time, and custom rule-writing is unavoidable.
- The real value of using built-in rulesets where they exist: compliance mapping, MITRE ATT&CK tagging, and tuning that the Wazuh project maintains centrally — reasons to prefer "adapt the log to the existing rule" over "write a new rule to match the log" whenever a built-in option exists.

Lab 4: Build a Windows Server 2022 evaluation VM via QEMU with a fully unattended install

Goal

Get a working Windows Server 2022 desktop running under QEMU on Apple Silicon, installed with zero interactive input, as the target host for future Atomic Red Team testing (Lab 5).

Prerequisites

- QEMU installed via Homebrew (`brew install qemu`).
- `hdiutil` (built into macOS) for building the answer-file ISO.
- The Windows Server 2022 Evaluation ISO, downloaded from Microsoft's own stable fwlink direct-download URL for the Server evaluation channel — chosen specifically because it requires no interactive business-info web form, unlike the Windows 11 Enterprise evaluation channel.
- Roughly 1+ hour of wall-clock time — TCG software emulation of an x86_64 guest on an Apple Silicon host is genuinely slow; this is not a KVM/HVF accelerated install.

Steps

1. Create the virtual disk:

```
qemu-img create -f qcow2 ~/securitylab/vms/winserver2022.qcow2 60G
```

2. Build an `autounattend.xml` answer file covering the `windowsPE`, `specialize`, and `oobeSystem` passes: disk partitioning, product key (evaluation edition), administrator password, computer name, and a `FirstLogonCommands` block.

The real bug hit and its fix: an initial attempt at the `specialize` pass included a `Networking-MPSSVC-Svc` component intended to enable the firewall and RDP declaratively via unattend XML properties. This failed during setup with:

Windows could not parse or process the unattend answer file for pass [specialize]. The settings specified in the answer file cannot be applied. The error was detected while processing settings for component [Networking-MPSSVC-Svc].

The property names used for that component were wrong/unsupported for this build. Rather than continuing to fight exact unattend.xml component schemas from memory against undocumented edge cases, the fix was to remove that component from the specialize pass entirely and instead enable the firewall rule and RDP via plain netsh advfirewall and registry commands inside FirstLogonCommands in the oobeSystem pass — simpler, more reliable, and far easier to verify correct:

```
<FirstLogonCommands>
  <SynchronousCommand wcm:action="add">
    <Order>1</Order>
    <CommandLine>netsh advfirewall firewall set rule group="remote desktop" new enable=yes</CommandLine>
  </SynchronousCommand>
  <SynchronousCommand wcm:action="add">
    <Order>2</Order>
    <CommandLine>reg add "HKLM\System\CurrentControlSet\Control\Terminal Server" /v fDenyTSConnections /t REG_DWORD /d 0 /f</CommandLine>
  </SynchronousCommand>
</FirstLogonCommands>
```

3. Build a small hybrid ISO containing just autounattend.xml at its root, using macOS's built-in hdiutil:

```
hdiutil makehybrid -o ~/securitylab/vms/autounattend.iso \
  ~/securitylab/vms/unattend-src/ -iso -joliet
```

Windows Setup automatically scans attached removable media for a file named autounattend.xml during the windowsPE pass — this is why the answer file doesn't need to be baked into the install ISO itself, just present on any attached CD-ROM.

4. Boot the install with two CD-ROMs attached — the Windows install ISO and the answer-file ISO — using BIOS/IDE emulation rather than UEFI/q35:

```
qemu-system-x86_64 \
  -machine pc \
  -accel tcg \
  -m 4096 \
  -smp 2 \
  -drive file=~/securitylab/vms/winserver2022.qcow2,if=ide,format=qcow2 \
  -drive file=~/path/to/WinServer2022-eval.iso,media=cdrom,if=ide \
  -drive file=~/securitylab/vms/autounattend.iso,media=cdrom,if=ide \
  -boot d \
  -vnc :1
```

Why BIOS/IDE (pc machine type) instead of q35/UEFI: IDE disk and CD-ROM emulation is understood natively by Windows Server 2022's inbox drivers with no virtio driver injection needed at install time — trading some disk/network performance for install-time simplicity and compatibility, which was the right tradeoff for getting a working install without fighting driver injection on top of everything else. Why TCG (software emulation): an x86_64 guest cannot use Apple Silicon's hardware virtualization (HVF accelerates only same-architecture guests), so this install runs fully emulated — genuinely slow (~1hr+), a real, expected cost of this specific host/guest architecture combination, not a misconfiguration to fix.

5. Connect via VNC (:1 → localhost:5901) to watch/confirm the unattended install proceeds without any prompts, through to a working desktop with the 10-boot autologon oobeSystem setting takes it to.

Expected result

A booted Windows Server 2022 desktop, reached with zero interactive input during setup, confirmed by connecting over VNC and seeing a logged-in desktop. This was confirmed — the install completed and a working desktop was reached.

What was NOT verified — an honest, stated gap

Reaching a working desktop over VNC is not the same claim as “this VM is reachable and manageable the way a real lab target needs to be.” RDP and WinRM reachability from the host were never independently verified end-to-end in this session. The `FirstLogonCommands` block enables the firewall rule and flips the `fDenyTSCconnections` registry key, and there is every reason to expect RDP works given those settings applied cleanly with no error — but no actual RDP client connection from the host to the guest was tested and confirmed working, and WinRM (needed for a lot of remote Windows administration and would matter for future automation against this VM) was not configured or tested at all. This is a real, stated gap, not an assumption dressed up as a result — a reader reproducing this lab should treat “confirm RDP/WinRM actually connect from the host” as an unverified next step, separate from and in addition to Lab 5’s Sysmon/agent work.

The open, unresolved tension worth naming

QuartzVM, the GUI VM manager the user separately built (a real open-source, MIT-licensed Rust/Tauri app, `quartzvm-releases`), is explicitly designed around a “create → snapshot → destroy” disposable-lab philosophy. The Windows Server VM built in this lab — hardcoded administrator password baked into the answer file, 10-boot autologon configured for convenience — is, as built, closer to a persistent asset than a disposable lab VM. This is a genuinely open, undecided question in this lab, not something resolved by this document: should this VM be rebuilt to fit the disposable philosophy before it’s used for anything sensitive, or does its role (a fixed Atomic Red Team target) argue for persistence being fine here? Both positions have merit; no decision has been made.

What it teaches

- The real mechanics of a fully unattended Windows install via raw QEMU — not through a GUI wizard, but through the actual answer-file/ISO mechanism Windows Setup itself uses.
- A concrete lesson in debugging `unattend.xml` failures: when a declarative component’s exact property schema is uncertain, falling back to an imperative equivalent (`netsh/reg` commands in `FirstLogonCommands`) is often more reliable than guessing at exact schema syntax from memory — a generally transferable lesson about knowing when to stop fighting a declarative config format and use an imperative escape hatch instead.
- The specific architecture tradeoffs (BIOS/IDE vs. UEFI/q35, TCG vs. hardware acceleration) that matter when running an `x86_64` guest on an Apple Silicon host, and why those tradeoffs point toward BIOS/IDE for an install-time-simplicity-first VM.
- The discipline of stating what “the VM booted” does and doesn’t prove — a working desktop over VNC is real evidence of one thing (the install succeeded) and zero evidence of another (host-to-guest network management reachability), and conflating the two is exactly the kind of overclaiming this document is built to avoid.

Lab 5 (NOT YET DONE — a documented next step, not a completed lab): install Sysmon + a Wazuh agent, run one real Atomic Red Team test

Status

This lab has not been performed. It is included here specifically so the gap is documented rather than silently missing, and so a reader knows exactly what remains between “a Windows VM exists” (Lab 4) and “the

atomic-red-team-validation repo's T1078.004 and T1059.001 case studies are real, run, evidenced case studies" rather than the "planned, not yet run" state they're honestly labeled with today (see Part 6, Walkthrough 3).

Goal (planned)

Install Sysmon and a Wazuh agent on the Windows Server 2022 VM from Lab 4, confirm the agent registers with the Wazuh manager from Lab 1 and Sysmon telemetry (process creation, PowerShell script block logging) flows into Wazuh, then run exactly one Atomic Red Team test — most naturally T1059.001 (PowerShell), since it requires no cloud identity setup, unlike T1078.004 which involves cloud account context — and confirm Wazuh produces a real alert from real Sysmon telemetry of that real technique execution.

Prerequisites this depends on

- Lab 4's VM, plus resolving the stated RDP/WinRM reachability gap from Lab 4 first — Sysmon and agent installation will need either a working RDP session or working WinRM/PowerShell remoting from the host, and neither has been confirmed working yet.
- Sysmon (Microsoft Sysinternals, [FREE]) with a configuration file (commonly a public baseline like SwiftOnSecurity's or Olaf Hartong's Sysmon-modular config) tuned to log process creation (Event ID 1) and, for PowerShell-specific visibility, PowerShell script block logging (Event ID 4104) enabled via Group Policy or registry.
- The Wazuh Windows agent package, installed and pointed at this lab's Wazuh manager (wazuh.manager container from Lab 1) with the manager's address and an enrollment key.
- The Atomic Red Team PowerShell module (Invoke-AtomicRedTeam) installed on the guest to actually execute the T1059.001 atomic test.

What it would teach, once done

- Whether Wazuh's default Windows/Sysmon ruleset detects the atomic test's execution pattern out of the box, or whether — as with the Cloudflare case in Lab 2 rather than the MySQL case in Lab 3 — a custom rule is needed to catch it, which itself would be a useful, real data point about Wazuh's out-of-box Windows detection coverage versus its Linux/network coverage demonstrated in Labs 2 and 3.
- The full, real version of the hypothetical T1059.001 triage walkthrough in Part 6 — turning that "here is what triage would look like" narrative into an actual alert with actual evidence, the same transition Labs 2 and 3 already made for their respective detections.

Until this lab is performed, any description of what its results would look like remains explicitly hypothetical, exactly as framed in Part 6.

Part 10: Real Troubleshooting Log

This part is a set of genuine troubleshooting case studies from actually building this lab — not illustrative examples. Each one follows the same shape: symptom, investigation, root cause, fix, and a one-line lesson that generalizes past this specific bug. The point of including these at all is that this is what real lab-building (and real production operations) actually looks like — not a clean sequence of commands that worked on the first try, but a series of specific things that broke, diagnosed and fixed one at a time.

10.1 The Wazuh certificate generator permissions bug

Symptom: Wazuh 4.9.2 [FREE] (Apache 2.0, self-hosted) ships an official certificate generation step as part of the wazuh-docker single-node setup — a container run that generates the internal TLS certificates (root CA plus per-node certs) used between the manager, indexer, and dashboard containers. Partway through

this run, the output folder holding the generated certificates got locked read-only, before the generator had finished writing the last two certificate files. Subsequent writes into that folder failed.

Investigation: the certificate generator writes its output into a bind-mounted host directory, and at some point during the run it (or a step in the container's entrypoint chain) applies restrictive permissions to that directory as a hardening step — intended to run after all files are written, but in this run it took effect while two certificate files were still outstanding. This left the output directory in a half-finished, now-locked state: most of the expected certs present, two missing, and the directory no longer writable to let the generator finish naturally on a re-run without first fixing permissions.

Root cause: a permissions-hardening step in the cert generator's run sequence executed before, rather than strictly after, all output files were written — a timing/ordering issue in the generator's own process, not a misconfiguration on the host side.

Fix: rather than fight the generator's permission state to coax it into finishing the run, the two missing certificate files were created by hand — copying the already-generated root CA cert/key pair to serve in place of the two missing files, since in this single-node setup topology the missing files were, functionally, just further copies of the same root CA material the generator had already successfully produced for the other nodes. This is a pragmatic, single-node-specific fix — it works because a single-node Wazuh deployment's internal certs all chain to the same root CA and don't need distinct per-node key material to function correctly in this topology; it would not necessarily be the right fix in a multi-node cluster where each node legitimately needs its own distinct certificate identity.

Lesson: when a generator tool fails partway through a multi-file output with a permissions error, check whether the already-completed output is actually sufficient to hand-complete the remaining files before assuming the whole run needs to be reset and re-attempted from scratch.

10.2 The Windows **autounattend.xml** specialize-pass component error

Symptom: during the fully unattended install of the Windows Server 2022 Evaluation VM (built via raw qemu-system-x86_64, BIOS/i440fx machine type, autounattend.xml supplied via a second CD-ROM built with macOS's hdiutil makehybrid), Windows Setup surfaced this error during the specialize pass:

```
Windows could not parse or process the unattend answer file for pass
[specialize]. The settings specified in the answer file cannot be
applied. The error was detected while processing settings for component
[Microsoft-Windows-Networking-MPSSVC-Svc].
```

Investigation: the autounattend.xml included a Networking-MPSSVC-Svc component (the component that governs the Windows Firewall/MPSSVC service) in the specialize pass, intended to pre-enable the firewall and the specific rules needed for RDP access without any manual step after first boot. The error message names the exact component and pass where parsing failed, which narrowed the search immediately to that one component block rather than the file as a whole — the property names used for that component in the answer file did not match the schema Windows Setup actually expects for that component (wrong/unsupported property names for that specific unattend component), so Setup could not apply the specialize-pass settings and halted there.

Root cause: incorrect property names in the Networking-MPSSVC-Svc component block of autounattend.xml — the exact unattend.xml component schema for this particular firewall-service component was reconstructed from memory/general knowledge rather than validated against the actual schema, and it didn't match what Windows Setup expects.

Fix: rather than continue trying to get the exact property-name schema for that component right, the component was removed from autounattend.xml entirely, and the equivalent outcome (firewall enabled, RDP allowed) was achieved a simpler, more reliable way: plain netsh advfirewall commands run inside FirstLogonCommands, which execute as ordinary shell commands after the OS has already booted for the first time, rather than needing to be expressed as unattend.xml XML schema during the specialize pass at all.

```

<!-- Illustrative shape of the FirstLogonCommands replacement – generic
      example of the approach, not a verified full unattend.xml excerpt -->
<FirstLogonCommands>
  <SynchronousCommand wcm:action="add">
    <Order>1</Order>
    <CommandLine>netsh advfirewall set allprofiles state on</CommandLine>
  </SynchronousCommand>
  <SynchronousCommand wcm:action="add">
    <Order>2</Order>
    <CommandLine>netsh advfirewall firewall set rule group="remote desktop" new enable=yes</Com
  </SynchronousCommand>
</FirstLogonCommands>

```

Lesson: when an exact declarative-config schema (unattend.xml components, in this case) is hard to get right from memory and a simpler imperative equivalent (a shell command run after boot) achieves the same outcome, prefer the imperative path — it fails with an ordinary, readable command error instead of an opaque XML-schema parse error that only surfaces deep into a lengthy install process.

10.3 The Cloudflare API token permission-scoping saga

Symptom: getting `wrangler` (Cloudflare’s CLI) authenticated with enough permission to actually deploy to Cloudflare Pages [FREEMIUM] (static hosting is free; Logpush/continuous log retention needs a paid plan) took several failed attempts with hand-scoped API tokens before succeeding.

Investigation: Cloudflare’s custom API token system is built around granular permission groups — a token is scoped to specific combinations of a permission group (e.g. “Cloudflare Pages”, “Workers R2 Storage”, “Zone DNS”) crossed with a resource scope (account-level or specific zone). The first token created for this session’s work was scoped to R2 storage permissions — the wrong permission group for what was actually needed (deploying to Pages), a leftover from earlier, unrelated R2-focused work. A second and third token were created, attempting to correct this, but still landed without the specific “Pages — Edit” permission actually included in the permission group selection. Compounding this, the Cloudflare account itself had zero zones and no site actually configured at the point these tokens were first being tested, which made it harder to tell, from a failing `wrangler` command alone, whether the problem was token scope, account state, or both at once.

Root cause: Cloudflare’s permission-group system for custom API tokens is granular enough that it is genuinely easy to under-scope a token — there is no single “Pages: full access” checkbox that obviously covers everything Pages deployment needs; the correct combination of permission group and access level has to be selected deliberately, and it is easy to select a plausible-looking but incomplete combination (as happened across the first three tokens here) without an obvious error telling you which specific permission is missing.

Fix: rather than continue iterating on hand-scoped custom tokens, `wrangler login` was used instead — Cloudflare’s browser-based OAuth flow, which authenticates `wrangler` with the actual logged-in user’s account access rather than a manually-assembled permission set. This flow grants broader scope by default (matching whatever the logged-in account can actually do in the dashboard) and succeeded immediately where the hand-scoped custom tokens had not.

Lesson: Cloudflare’s granular custom-token permission system is easy to under-scope, and for a one-off or individual admin task, the browser-based OAuth login flow is often faster and more reliable than iterating on a hand-built token’s exact permission groups — save precisely-scoped custom tokens for cases that actually need least privilege (a CI pipeline, a shared automation credential), not for one-person interactive use.

10.4 The `wrangler tail` deployment-ID binding limitation

Symptom: `wrangler pages deployment tail` (used to stream live request logs from the Cloudflare Pages Function middleware logging every real request to `soc-lab-target.pages.dev`, since Cloudflare's free tier has no Logpush/log retention) needs to be run in a way that keeps working across redeployments of the site, and it doesn't.

Investigation: `wrangler pages deployment tail` in non-interactive mode (the mode needed to run it unattended, piped into the Python relay script that reformats and forwards log lines to Wazuh) requires an explicit deployment ID argument — it does not implicitly tail “whatever the current/latest deployment is.” In interactive mode, `wrangler` can prompt for or infer a target deployment, but that interactivity is exactly what has to be avoided for something meant to run unattended in the background.

Root cause: the `tail` command's non-interactive contract is bound to one specific, immutable deployment ID at invocation time, not to the site or project as a moving target. There is no “tail whatever is currently live” mode available non-interactively.

Downstream consequence: because every deploy to Cloudflare Pages creates a new deployment with a new ID, any `wrangler tail` process already running against the previous deployment's ID silently stops being relevant the moment a new deploy goes out — the old deployment may still exist and even still serve some traffic during a transition, but new traffic to the newly-live deployment is invisible to a tail process still bound to the old ID. This directly breaks the goal of “persistent” monitoring: any redeploy of the Pages site — even a trivial content change — silently breaks the log pipeline until someone notices and manually restarts the tail process with the new deployment's ID.

Fix: documented as a known, real, currently-unresolved operational limitation in this lab, not something that was engineered around with, for example, an auto-restart wrapper script that queries the latest deployment ID and re-launches `tail` on each redeploy — that would be the natural next fix, but it had not been built as of this document being written.

Lesson: a monitoring/log-tailing tool bound to a specific resource identifier (a deployment ID, here) rather than to a stable, persistent resource (the project/site itself) is not actually persistent monitoring — it is monitoring that silently expires on the next change to the thing it's watching, and that expiry needs to be handled explicitly (automatic re-binding, alerting on staleness, or both) or it will be discovered the hard way, as a silent gap.

10.5 The MySQL log-format mismatch with Wazuh's decoder

Symptom: MySQL 8.0 [FREE] (community edition; Oracle also sells a paid Enterprise edition with additional features not used here) was stood up as a monitored, deliberately-attacked target (`soc-lab-mysql`) with general query logging and error logging both enabled. Wazuh already ships built-in decoders and rules for MySQL log formats (`ruleset/decoders/0150-mysql_decoders.xml`, `ruleset/rules/0295-mysql_rules.xml`) — no custom Wazuh rule should have been needed. But MySQL's raw `general_log/error_log` output, fed directly into Wazuh, did not get decoded or matched by those built-in rules.

Investigation: Wazuh's built-in `mysql_log` decoder is written against a specific expected line prefix — it uses a prematch pattern looking for lines that begin with `MySQL log:` before it will even attempt the rest of the decode. MySQL's actual raw log output — both `general_log` and `error_log` in their native on-disk format — does not begin with that literal string; it uses MySQL's own native timestamp and field layout, which never matches the decoder's prematch condition at all. Every line was arriving at Wazuh, but every line was being silently ignored at the very first decoder-matching step, before any rule logic ever had a chance to run — meaning no amount of tuning the rule would have fixed this, since the decoder was never even matching to hand anything to the rule stage.

Root cause: a format mismatch between the source log's native output shape and the specific prematch string Wazuh's shipped decoder expects. The decoder and rule content were fine as-is (built-in, no custom

rule needed, exactly as expected) — the problem was entirely that raw MySQL log lines never matched the shape the decoder was written to recognize.

Fix: a small Python relay script was written to tail `error.log/general.log`, reformat each matching line so it starts with the literal `MySQL log:` prefix (and otherwise preserves/reshapes the rest of the line into the field order the decoder expects), and append the reformatted lines to a separate file bind-mounted into the Wazuh manager container. That file is picked up via a Wazuh `<localfile>` stanza configured with `<log_format>syslog</log_format>`. Verified live: four deliberate wrong-password MySQL login attempts correctly fired Wazuh's built-in rule 50106 ("MySQL: authentication failure", level 9), which auto-mapped to PCI DSS/GDPR/HIPAA/NIST 800-53 compliance controls out of the box, once the relay was in place.

Lesson: this is a specific instance of a very general, very common real-world detection engineering problem — the source log format almost never matches your detection platform's expected format out of the box, even when the platform ships purpose-built content for that exact log source. "Just add a log source" sounds like a config change; in practice, log normalization and reformatting — getting the bytes on disk to actually match what the parsing layer expects — is very often the majority of the real work involved, and it is worth budgeting time for that step explicitly rather than assuming a built-in decoder/rule pair means the log source will "just work" once the file is being read.

10.6 The repo-wide markdown-header convention drift

Symptom: across this portfolio's public repos, there is a consistently applied rule of never using literal `#` markdown headers — using bold text instead — everywhere except this specific knowledge document (which needs real `###/####` headers because it gets converted to Word/PDF via pandoc, and pandoc needs real heading levels to build a table of contents). Despite that rule already being consistently present in the existing files of every repo, multiple background AI agents, each individually tasked with writing new content "matching this repo's existing style," still reintroduced literal `#` headers in the new files they wrote.

Investigation: each agent was working from an instruction along the lines of "match the existing style/tone of this repo" — which relies on the agent correctly inferring the no-`#`-headers convention purely by reading the surrounding files and generalizing from them, rather than being told the rule directly. Because the instruction never stated the formatting rule explicitly, whether any individual agent actually picked it up depended on how much of the existing file content it happened to read carefully enough to notice the pattern, and how strongly it weighted "match style" against its own default habit of using standard markdown headers (which is the normal, unremarkable default for markdown generally — there was nothing about a `#` header that would read as obviously wrong to an agent that hadn't specifically registered this repo's specific deviation from that default). The result was inconsistent: some new files matched the convention, others didn't, requiring a manual audit pass afterward to find and fix every file that had drifted.

Root cause: an implicit convention ("match the existing style") was relied on to carry a specific, non-default formatting rule (no `#` headers) that needed to be stated as an explicit instruction to be reliably followed, especially across multiple independent writers working in parallel with no shared checklist between them.

Fix: a manual audit-and-fix pass across the affected files, correcting any reintroduced `#` headers back to the bold-text convention — and, going forward, stating the header-style rule explicitly in any instruction given to a writer (human or agent) working on these repos, rather than relying on it being inferred from context.

Lesson: when delegating content-writing work — to another person or to an automated agent, and especially when multiple writers are working in parallel without visibility into each other's output — state non-default conventions (formatting rules, naming rules, structural rules) as explicit, repeated instructions every time, rather than trusting them to be correctly inferred from "match the existing style." Explicit contracts survive parallel, fast-moving work in a way that inferred conventions don't; this exact document's own formatting rule (use real `#` headers, the opposite of the portfolio's usual convention) is itself only followable because it was stated explicitly in the brief each writer received, which is the direct, deliberate fix for the failure mode described in this case study.

Part 11: Glossary

Alphabetical reference for terms and acronyms used throughout this document. Definitions are written to be precise and specific to how the term is actually used in this lab, not generic dictionary text.

A

Answer file (**autounattend.xml**) — an XML file that supplies Windows Setup with pre-determined install-time answers (disk partitioning, locale, admin password, first-boot commands) so an installation can run fully unattended, with no human clicking through the installer GUI. Windows Setup automatically scans attached removable media (including a CD-ROM) for a file with this exact name during its early (windowsPE) install pass.

ATT&CK (MITRE ATT&CK) — a publicly maintained knowledge base of real-world adversary behavior, organized as a matrix of tactics (the adversary's goal at a given stage, e.g. Initial Access, Persistence, Privilege Escalation), techniques (a specific method used to achieve a tactic, e.g. T1078 Valid Accounts), and sub-techniques (a more specific variant of a technique, e.g. T1078.004 Cloud Accounts). Used in this lab both to map detection rules to the behavior they catch (`attack-mapping.csv`) and to structure adversary-emulation test plans (Atomic Red Team).

B

BEC (Business Email Compromise) — a fraud technique where an attacker compromises or spoofs a business email account/identity (often an executive or finance-team member) to trick a target into taking a harmful action, most commonly an unauthorized wire transfer or a credential/gift-card request. Distinct from generic phishing in that it usually skips malware entirely and relies purely on social engineering and a convincing sender identity.

C

Cortex — the analyzer/responder automation engine paired with TheHive, used to run enrichment and response actions (e.g. reputation lookups, IOC checks) against observables attached to a case.

CSP (Content-Security-Policy) — an HTTP response header that tells a browser which sources (scripts, styles, images, frames, etc.) a page is allowed to load content from, used as a defense-in-depth control against cross-site scripting and data-injection attacks by restricting what a compromised or injected script is actually allowed to do.

D

Decoder (Wazuh) — see “Decoder vs. rule.”

Decoder vs. rule (Wazuh-specific distinction) — in Wazuh, a decoder is responsible for parsing a raw log line into structured fields (extracting a source IP, a status code, a username, etc. from unstructured text) — it decides whether a line is even recognized at all, often gated by a prematch pattern that the start of the line must match before decoding proceeds further. A rule then evaluates the already-decoded fields against conditions (this field equals X, this decoder's output combined with another recent event, etc.) to decide whether to generate an alert, at what severity level. A log line that never matches any decoder's prematch pattern never reaches rule evaluation at all, regardless of how well-written the applicable rule is — this is exactly the failure mode documented in Part 10's MySQL log-format case study.

Detection engineering — the discipline of building, testing, tuning, and maintaining the actual detection logic (rules, queries, correlation searches) that turns raw telemetry into meaningful security alerts — distinct from simply “having a SIEM,” since a SIEM ingesting logs with no tuned detection content on top of it produces either silence or noise, not signal.

Detection mode (WAF) — a WAF operating mode where a request matching a rule is logged as if it would have been blocked, but is still allowed through to the backend unmodified — used to observe and tune a ruleset’s false-positive rate against real traffic before switching to prevention mode. See Part 8.3.

F

False positive / false negative — a false positive is an alert that fires on activity that was not actually malicious or was not the condition the detection was meant to catch (wastes analyst time, and enough of them causes real alerts to get ignored — “alert fatigue”). A false negative is malicious activity that a detection should have caught but didn’t (a silent, undetected gap — generally the more dangerous of the two failure modes, since nothing draws attention to it).

G

GUID (Globally Unique Identifier) — a 128-bit identifier (xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx format) used, among many other places, by Microsoft Entra ID/Azure AD to identify users, groups, applications, and roles uniquely and immutably — unlike a display name or UPN, a GUID doesn’t change if an object is re-named, which is why identity investigations often need to resolve a GUID back to a human-readable identity as an explicit triage step (see the `guid-triage` runbook referenced in the Identity Security section).

H

HSTS (HTTP Strict-Transport-Security) — an HTTP response header that tells a browser to only ever connect to this host over HTTPS for a specified duration, protecting against protocol-downgrade and SSL-stripping attacks on subsequent visits by refusing to even attempt a plain-HTTP connection.

hvf (Hypervisor.framework) — Apple’s built-in hardware virtualization acceleration framework on macOS, analogous to KVM on Linux and WHPX on Windows — lets a hypervisor (like QEMU) use the CPU’s own virtualization extensions instead of software emulation, which is dramatically faster but only works when guest and host CPU architectures match.

I

IaC (Infrastructure as Code) — managing infrastructure (cloud resources, networking, compute) by describing it in version-controlled configuration files (e.g. Terraform HCL) and applying that description through a tool, rather than manually configuring resources by hand through a console/UI — enables review, repeatability, and a diffable history of infrastructure changes.

IdP (Identity Provider) — a system that authenticates users and issues identity assertions/tokens that other applications (relying parties) trust, rather than each application handling its own username/ password authentication independently. Microsoft Entra ID, Auth0, and Okta are all IdPs.

IOC (Indicator of Compromise) — an observable artifact (an IP address, file hash, domain name, email address) associated with known or suspected malicious activity, used to search for or match against telemetry to find evidence of that activity elsewhere.

J

JML (Joiner-Mover-Leaver) — the identity lifecycle framework covering the three key transition points of an identity’s relationship with an organization: joining (provisioning access for a new hire), moving (adjusting access when someone changes role/department), and leaving (deprovisioning access when someone departs) — a common source of identity security gaps is a broken or incomplete Mover or Leaver process leaving stale, unnecessary access in place.

K

KQL (Kusto Query Language) — the query language used by Microsoft Sentinel (and Azure Data Explorer/Log Analytics more broadly) to search and analyze log data — the language this lab's detection-engineering repo's Sentinel-facing detection content is written in, targeting tables like SigninLogs and AuditLogs.

L

Localfile (Wazuh config term) — a `<localfile>` block in Wazuh's agent/manager configuration (`ossec.conf`) that tells Wazuh to monitor a specific file or log source (a file path, a Windows Event Channel, a command's output) and feed its content into the decoder/rule pipeline — the mechanism this lab uses, with `<log_format>syslog</log_format>` and `<log_format>json</log_format>` respectively, to ingest the relayed MySQL logs and the relayed Cloudflare Pages traffic logs.

Logpush — Cloudflare's feature for continuously streaming logs (HTTP requests, firewall events, etc.) to an external destination (storage, a SIEM) — a paid-tier feature, which is why this lab's Cloudflare Pages traffic is instead captured via `wrangler ... tail` rather than Logpush.

M

MFA (Multi-Factor Authentication) — requiring more than one independent proof of identity to authenticate (typically something you know, like a password, plus something you have, like an authenticator app or hardware key) — a baseline control against credential-only compromise, and required on this lab's Azure account specifically as a condition of doing any IaC work against it.

O

OIDC (OpenID Connect) — an identity/authentication layer built on top of OAuth 2.0, adding a standardized way for a client to verify a user's identity and obtain basic profile information via an ID token — commonly used alongside or instead of SAML for modern application sign-in flows.

P

Playbook — in this document's usage, a documented, structured response procedure for handling a specific type of security event or alert (e.g. this lab's MFA fatigue and impossible-travel playbooks) — distinct from a SOAR automation playbook (see `sentinel-soar-playbooks`), though the two terms overlap: an analyst-facing playbook can be the basis a SOAR automation is later built from.

PIM (Privileged Identity Management) — a Microsoft Entra ID feature for just-in-time, time-bound activation of privileged roles, rather than standing/permanent privileged role assignment — reduces the window an elevated-privilege identity is actually usable, and creates an approval/audit trail each time a role is activated.

Prematch (Wazuh decoder term) — see "Decoder vs. rule."

Prevention mode (WAF) — a WAF operating mode where a request matching a blocking rule is actually blocked in real time, as opposed to detection mode's log-only behavior. See Part 8.3.

R

RBAC (Role-Based Access Control) — an access control model where permissions are granted to roles, and identities are granted membership in roles, rather than permissions being assigned to individual identities directly — makes access easier to reason about and audit at scale than per-identity permission assignment.

S

SAML (Security Assertion Markup Language) — an XML-based standard for exchanging authentication and authorization assertions between an IdP and a service provider, commonly used for enterprise single sign-on — older than OIDC but still widely deployed, especially in enterprise SaaS integrations.

Sigma — a generic, platform-agnostic format for writing detection rules, designed to be convertible into the native query language of many different SIEM/log platforms (Splunk SPL, Elastic Query DSL, Sentinel's KQL, and others) from a single source rule — this lab's detection-engineering repo authors detection logic in Sigma first, then maintains KQL conversions for Sentinel's schema.

SIEM (Security Information and Event Management) — a platform that centrally collects, normalizes, and correlates log/event data from many sources and applies detection logic to it to generate alerts — Wazuh and Microsoft Sentinel are both SIEMs (Wazuh also markets itself as XDR; see below).

SOAR (Security Orchestration, Automation and Response) — a platform or capability for automating and orchestrating incident response actions — enrichment, containment, notification — often chained together into playbooks triggered by a SIEM alert. TheHive/Cortex here, and the Sentinel Logic Apps playbook designs, are both SOAR-category tooling.

T

TCG (Tiny Code Generator, QEMU context) — QEMU's built-in software CPU emulation backend, used when hardware-accelerated virtualization (hvf/KVM/WHPX) isn't available or applicable — notably, running an x86_64 guest (Windows Server) on an Apple Silicon (ARM64) host requires TCG, since there is no hardware acceleration path for emulating a different CPU architecture than the host's own — which is exactly why this lab's Windows Server VM install took over an hour: it is genuinely, substantially slower than hardware-accelerated virtualization.

Triage — the initial assessment step of incident response: given a raw alert or report, quickly determining its likely severity, whether it is a true or false positive, and what (if anything) needs to happen next — distinct from full investigation, which goes deeper once triage has established the alert is worth investigating further.

TTP (Tactics, Techniques, and Procedures) — the general term for how an adversary operates, spanning MITRE ATT&CK's tactic/technique hierarchy down to the specific, granular implementation details ("procedures") a particular threat actor or campaign actually used.

W

WAF (Web Application Firewall) — a filtering layer, typically sitting in front of a web application or API, that inspects HTTP(S) requests against a ruleset (commonly OWASP Core Rule Set-based) looking for patterns associated with common web attacks (SQLi, XSS, path traversal), and blocks or logs matches depending on its configured mode. See Part 8.3 for detection vs. prevention mode.

WHPX (Windows Hypervisor Platform) — Windows' built-in hardware virtualization acceleration API, the Windows analog to hvf (macOS) and KVM (Linux) — not directly relevant to this Mac-hosted lab, but included here because it is the third member of the same acceleration-backend family referenced when discussing why TCG was necessary on this particular host.

X

XDR (Extended Detection and Response) — a SIEM-adjacent product category that extends detection and response beyond log correlation alone into more active endpoint-level visibility and response capability (process monitoring, file integrity monitoring, active response actions) — Wazuh markets itself as SIEM/XDR, reflecting that it includes both a log-correlation detection engine and an agent-based endpoint monitoring/active-response capability, not just log ingestion.

Part 12: Appendix — Paid vs. Free Summary Table

A single reference table for every tool/service/product mentioned across this document, covering what's actually free, what the real limits of a free tier are, and what role each one plays in this specific lab. Tiers and limits below reflect general public vendor documentation as of this writing (2026) — vendors change pricing and free-tier limits over time, so treat specific numbers as “accurate as of when this was written,” not permanent facts, and re-verify before relying on them for a real budget decision.

Tool / Service	Free tier or fully free?	What the paid tier adds / real limit of the free tier	Used in this lab as
Wazuh	Fully free (Apache 2.0, self-hosted, no paid tier at all for the core product)	No feature gating — Wazuh's business model is paid support/services on top of a genuinely free product, not a crippled free tier. Real “cost” is self-hosting/ops effort (you run and maintain the containers/indexer yourself).	Core SIEM/XDR for the lab — manager, indexer, dashboard containers; ingests MySQL and Cloudflare Pages traffic logs.
TheHive	Free (community edition, self-hosted)	TheHive 5's commercial “Strategic Blue”/enterprise offerings add SSO, multi-tenancy and support contracts beyond the community edition; the community edition used here is fully functional for case management on its own.	SOAR/case management — built, containers exist, currently stopped/idle, not yet wired to Wazuh alerts.
Cortex	Free (self-hosted, paired with TheHive)	Individual analyzers may themselves call paid third-party services (e.g. a paid threat-intel API) — Cortex itself is free; some analyzers it runs are only as free as the service they call.	Analyzer/responder engine for TheHive; stopped/idle alongside TheHive.
Cassandra	Fully free (Apache 2.0)	No paid tier — DataStax sells a managed/commercial offering built on Cassandra, but the open-source database itself is free.	Backing datastore for TheHive.

Tool / Service	Free tier or fully free?	What the paid tier adds / real limit of the free tier	Used in this lab as
Elasticsearch	Free (Apache 2.0 as of recent versions / prior Elastic license depending on version)	Elastic's commercial tiers (via Elastic Cloud or the platinum/enterprise self-managed subscriptions) add features like advanced security, ML, and support — not required for TheHive's use here.	Backing search/index store for TheHive.
LocalStack	Freemium — community edition is free	Free (community) edition covers core services (IAM, S3, Lambda, and more) used for this lab's testing; the paid Pro tier adds broader AWS service coverage and persistent state across restarts.	Local AWS emulation for <code>aws-identity-detection</code> 's IAM privesc detection testing; built previously, stopped/idle this session.
Suricata	Fully free (open source, GPLv2)	No paid tier for the engine itself — commercial vendors sell managed rule feeds/support around it.	Network IDS, built previously (<code>jasonish/suricata:latest</code>), stopped/idle, documented as the obvious next log source to feed into Wazuh.
MySQL	Freemium — Community Edition is free	Community Edition (used here) is fully free and fully functional; Oracle's paid Enterprise Edition adds features like advanced auditing, encryption add-ons, and support contracts, none needed for this lab's use as a monitored target.	The <code>soc-lab-mysql</code> container — a real, deliberately attacked, logged target feeding Wazuh via a relay script.
Cloudflare Pages	Freemium	Static hosting itself is free with generous limits (unlimited static requests, a monthly build-minutes allowance); Logpush/continuous log retention specifically requires a paid Cloudflare plan — which is exactly why this lab captures live traffic via <code>wrangler ... tail</code> instead.	Hosts the real public <code>soc-lab-target.pages.dev</code> synthetic lab site, plus its <code>_middleware.js</code> request-logging Function.

Tool / Service	Free tier or fully free?	What the paid tier adds / real limit of the free tier	Used in this lab as
Cloudflare Workers	Freemium	Free tier includes a substantial daily request allowance for Workers; higher volume, longer CPU time per invocation, and some advanced features (e.g. Durable Objects at scale) require a paid plan.	Referenced as the underlying platform Cloudflare Pages Functions run on (the <code>_middleware.js</code> logging function is a Pages Function, which runs on the Workers runtime).
Cloudflare Logpush	Paid only — no free-tier access	Requires a paid Cloudflare plan; this is precisely the gap that makes <code>wrangler tail</code> necessary in this lab instead of continuous native log streaming.	Not used in this lab — explicitly worked around due to being unavailable on the free tier.
Cloudflare R2	Freemium	Free tier includes a monthly storage and operations allowance with zero egress fees (R2's headline differentiator vs. S3); usage beyond the free allowance is billed.	Mentioned only as the scope of this session's earlier, wrong-for-Pages API token (R2-storage-scoped, not Pages-scoped) — see Part 10.3.
Docker Desktop	Freemium	Free for personal use, education, small business, and non-commercial open-source use; larger commercial organizations (above a headcount/revenue threshold) require a paid subscription.	Container runtime for the entire locally-running stack — Wazuh, TheHive/Cortex, LocalStack, Suricata, MySQL.
QEMU	Fully free (open source, GPL)	No paid tier — a free, general-purpose machine emulator/virtualizer.	Runs the Windows Server 2022 Evaluation VM directly via raw <code>qemu-system-x86_64/qemu-img</code> commands.
QuartzVM	Fully free (MIT-licensed, open source)	No paid tier — a free desktop GUI wrapper around QEMU the user built themselves.	Available as an optional GUI for managing QEMU VMs; the Windows Server VM in this lab was actually built via raw QEMU commands, not through QuartzVM's GUI.

Tool / Service	Free tier or fully free?	What the paid tier adds / real limit of the free tier	Used in this lab as
Terraform (CLI)	Fully free (Mozilla Public License 2.0)	The open-source CLI itself has no paid tier or feature gate — it runs and manages state entirely locally/against whatever backend you configure, for free.	The IaC tool used across all terraform-labs exercises (01–07).
Terraform Cloud	Freemium	Free tier covers a limited number of users and includes remote state management and remote runs; paid tiers add features like Sentinel policy-as-code enforcement, SSO, and higher usage limits.	Integration target of terraform-labs exercise 04 specifically — distinct from the free CLI used in every other exercise.
Azure	Freemium	A free-tier subscription includes a limited credit amount plus a set of always-free service allowances for the first 12 months and beyond; real usage beyond those allowances is billed — which is exactly why this lab's Azure README documents a spending cap set before creating anything.	The target cloud for all terraform-labs exercises; a free-tier subscription exists, but no exercise has actually been apply'd against it.
Microsoft Sentinel	Paid (consumption-based)	Billed per GB of data ingested/analyzed and per analytics rule run — no standing free tier for real production use, though a trial tenant with some allowance can be obtained via the M365 Developer Program. This is the explicit, stated reason Wazuh was chosen for the actual live lab instead.	Not deployed anywhere in this lab — the target schema/query language (SigninLogs, AuditLogs KQL) that detection-engineering's Sentinel-facing content and sentinel-soar-playbooks' Logic App designs are written for, without ever having a real tenant to deploy them into.

Tool / Service	Free tier or fully free?	What the paid tier adds / real limit of the free tier	Used in this lab as
Microsoft Graph API	Freemium (access tier depends on the tenant/licensing behind it)	The API itself has no separate price — what's billed (or not) is the underlying Microsoft 365/Entra ID tenant and license tier behind the identity/data it's querying; a developer tenant (see M365 Developer Program) provides free access for this kind of scripting/testing work.	Used by the private intune-endpoint-health-platform repo's IAM scripts (privileged role access review, orphaned account detection) and referenced as the mechanism behind the Sentinel playbook designs' account-disable/session-revoke actions.
M365 Developer Program (E5 trial)	Free	Provides a free, renewable sandbox Microsoft 365 E5 tenant (typically on a 90-day renewable cycle, subject to Microsoft's current program terms) intended for development/testing, not production use — a real but time-boxed and non-production-grade free tier.	The realistic free path to a Sentinel-adjacent/Entra ID trial tenant referenced above; not confirmed as actually set up and exercised end-to-end in this lab.
Auth0	Freemium (Okta-owned)	Free tier covers a meaningful number of monthly active users/logins for development and small projects; paid tiers add enterprise features (advanced MFA policies, custom domains at higher tiers, SSO connections beyond the free allowance, System Log retention length).	Planned as a second identity provider for the lab (to diversify beyond Entra ID-only content) — a dashboard URL was shared, but a working Machine-to-Machine app (Domain/Client ID/Client Secret) was never actually configured. Open gap, not a built thing.
Okta Developer Edition	Free	Free developer tier intended for building/testing integrations, with usage caps appropriate to development rather than production traffic.	Discussed as an alternative identity provider option, not pursued — Auth0 was chosen instead (and, as noted above, not finished either).

Tool / Service	Free tier or fully free?	What the paid tier adds / real limit of the free tier	Used in this lab as
GitHub	Freemium	Free tier includes unlimited public repositories and (with reasonable limits) private repositories, free CI minutes on a monthly allowance, and core collaboration features; paid tiers add more CI minutes, advanced security features (e.g. certain code-scanning capabilities), and organization-level administrative controls.	Hosts every repo in this lab's portfolio — all public repos are free-tier public repos; <code>intune-endpoint-health-platform</code> is a private repo, also within the free tier's private-repo allowance.
Windows Server 2022 Evaluation	Free, but explicitly not a real production license	The evaluation build is time-limited — Microsoft's Server evaluation media is generally documented as a 180-day evaluation period (this document notes 180 days as the commonly cited figure; the brief for this lab did not itself specify a number, so treat this as accurate-to-general-Microsoft-documentation rather than lab-verified — confirm against the actual installed build's <code>s\mgr /dlv</code> output if the exact remaining evaluation window ever matters operationally). After the evaluation period, the OS requires conversion to a licensed edition or will begin forcing periodic shutdowns.	The Windows Server 2022 VM built via raw QEMU in this lab, intended to eventually run Sysmon + a Wazuh agent + Atomic Red Team tests — evaluation-licensed, not production-licensed, by design (it's a disposable-in-principle lab VM, though see Part 9's note on the tension between that and its current hardcoded-password/10-boot-autologon configuration).

Notes on accuracy and drift

Free-tier limits, evaluation license durations, and pricing structures are exactly the kind of detail vendors change without much notice. The figures above reflect this document's best understanding at time of writing and, where the underlying lab material didn't specify an exact number itself (the Windows Server evaluation period, specifically), that uncertainty is stated plainly rather than presented as a confirmed lab fact. Before

making a real budget or licensing decision based on any row in this table, re-check the vendor's current published terms directly — this table is a study aid and lab reference, not a procurement source of truth.